

Packing oranges at random



João Ribeiro

IT + Math @ IST



Funded by
the European Union



European Research Council
Established by the European Commission



Credit: https://www.reddit.com/r/oddlysatisfying/comments/b75dgf/this_display_of_oranges/



Kepler (1611):

This is the optimal way of packing oranges in 3 dimension.



Kepler (1611):

This is the optimal way of packing oranges in 3 dimension.

More generally, we can ask:

What is the largest fraction of $\mathbb{R}^n$ covered by non-overlapping spheres all of the same radius?

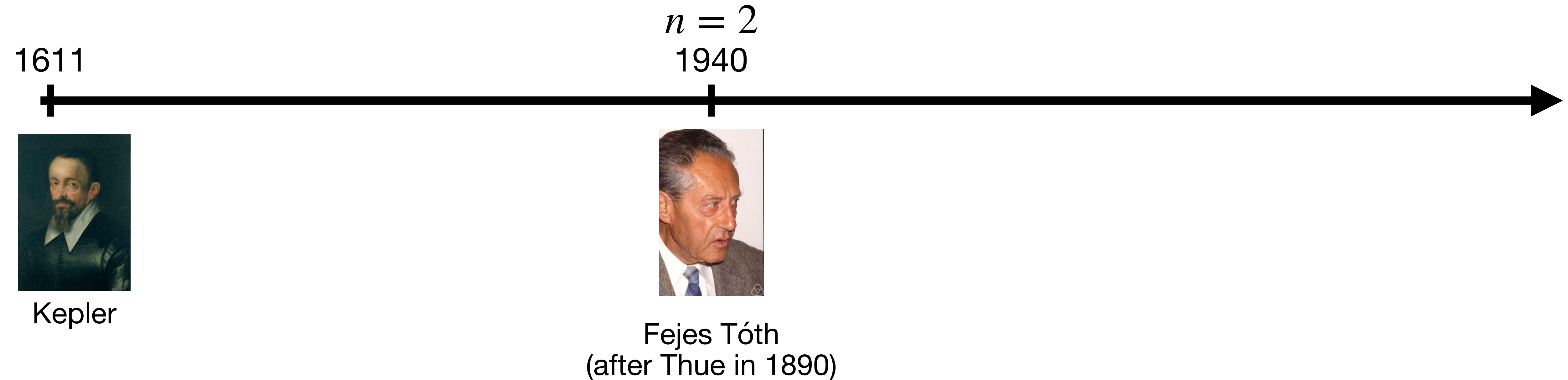
A lightning-fast history of sphere packing (not to scale)

1611

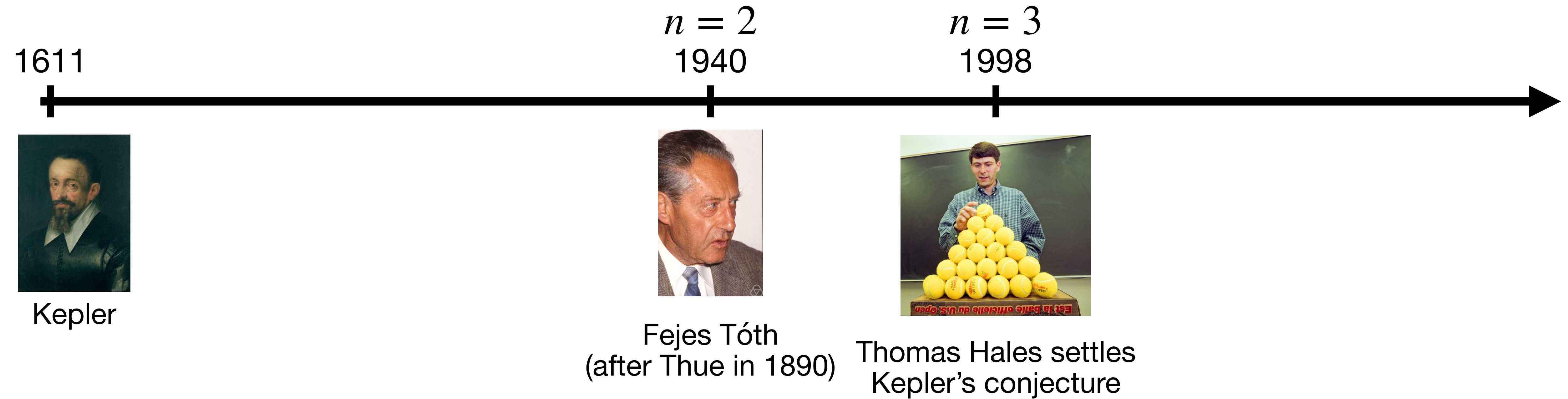


Kepler

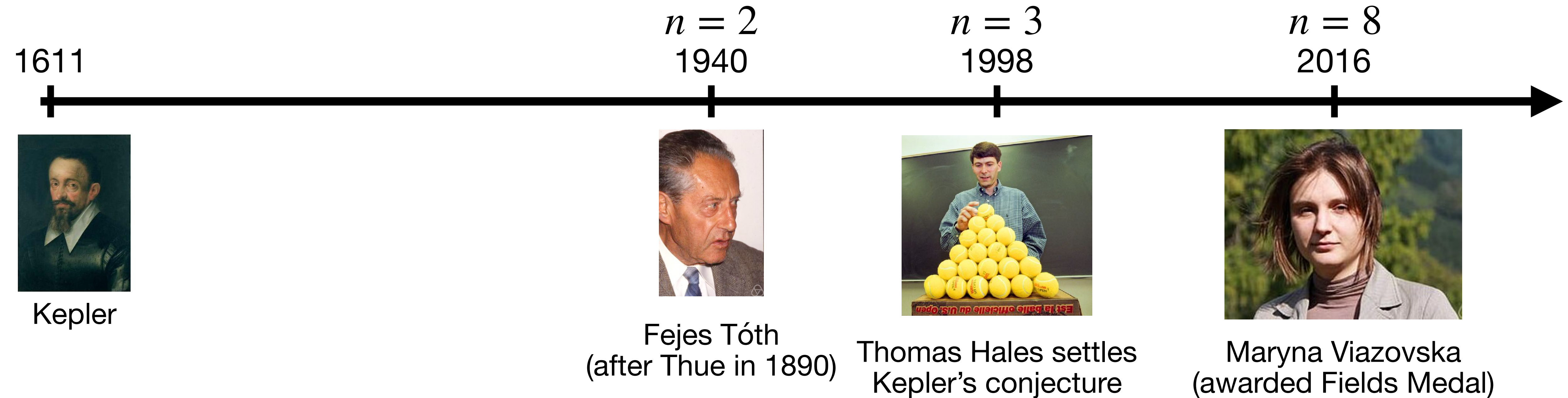
A lightning-fast history of sphere packing (not to scale)



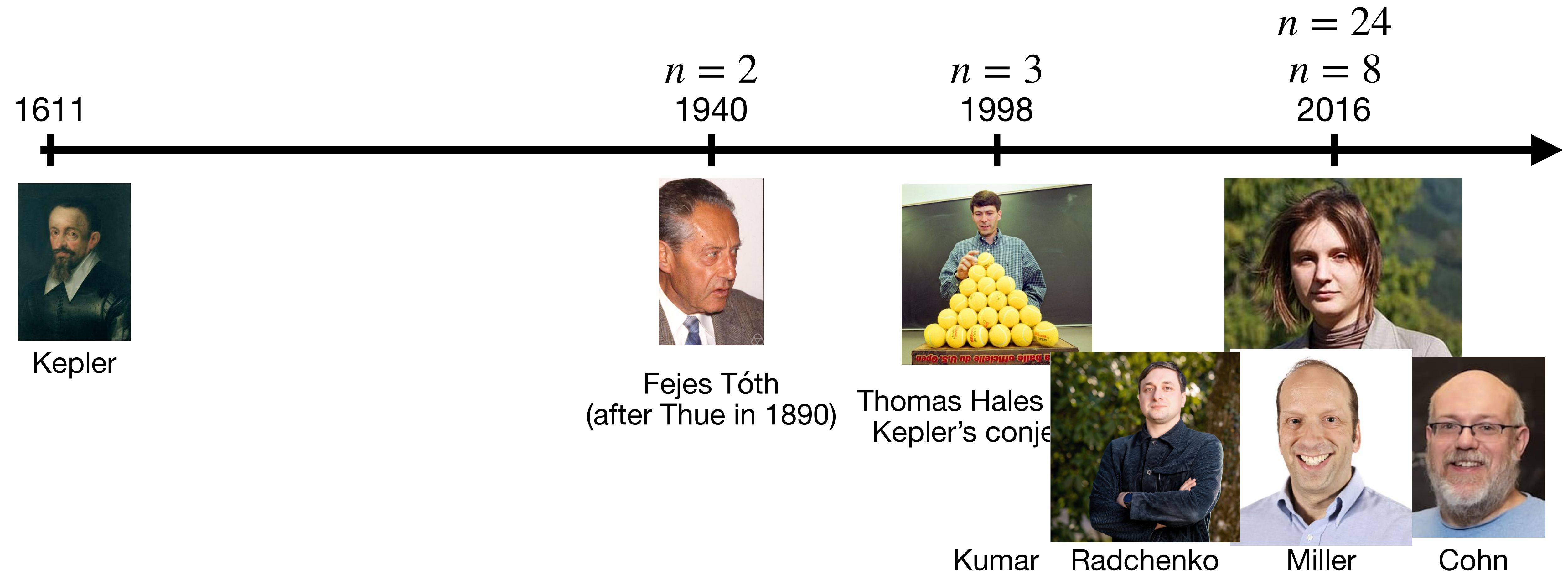
A lightning-fast history of sphere packing (not to scale)



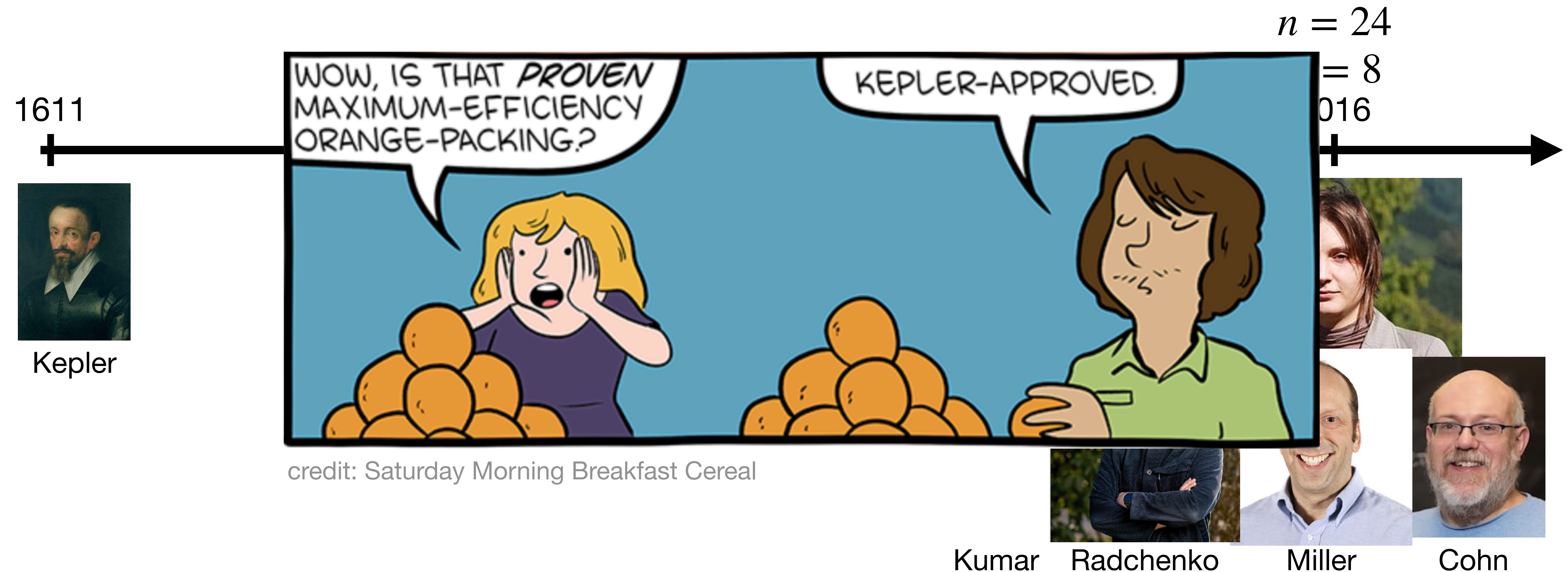
A lightning-fast history of sphere packing (not to scale)



A lightning-fast history of sphere packing (not to scale)



A lightning-fast history of sphere packing (not to scale)



A lightning-fast history of sphere packing (not to scale)

Other dimensions??

Asymptotics as $n \rightarrow \infty$??

$n = 24$

$= 8$

016

1611



credit: Saturday Morning Breakfast Cereal



Kepler



Kumar



Radchenko



Miller

Cohn

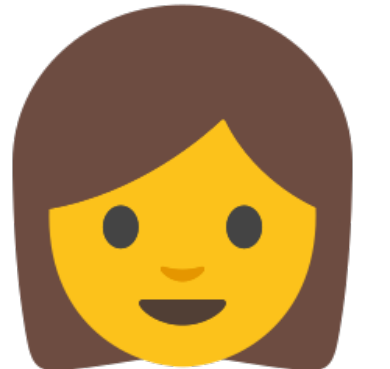
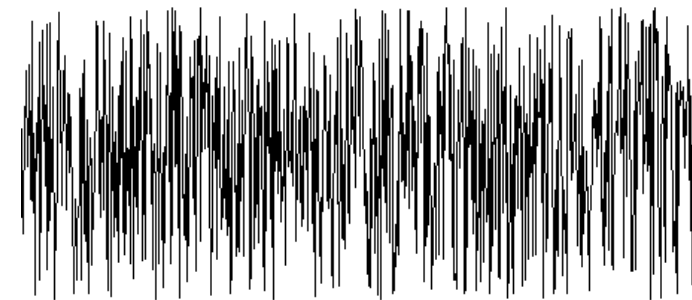
Now, something... completely unrelated?



Dinner at 8!



noisy channel

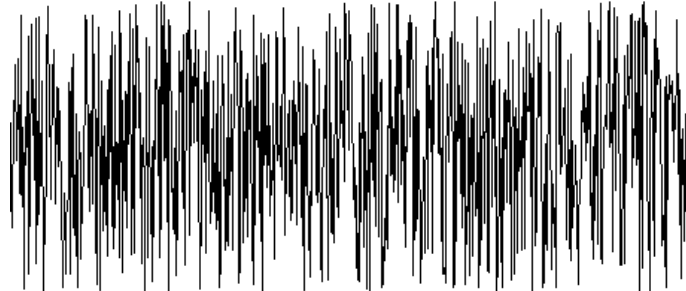


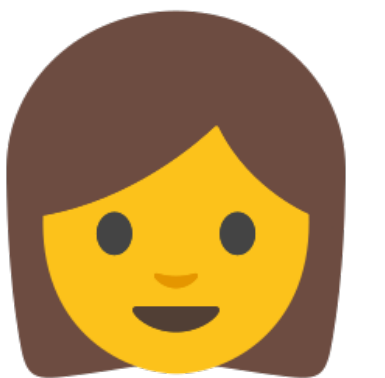
Dinpr at 9!

Now, something... completely unrelated?

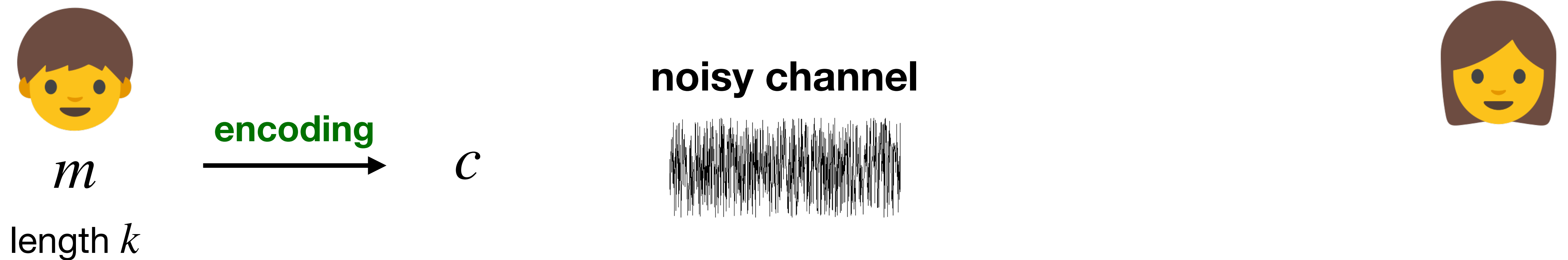



 m
length k

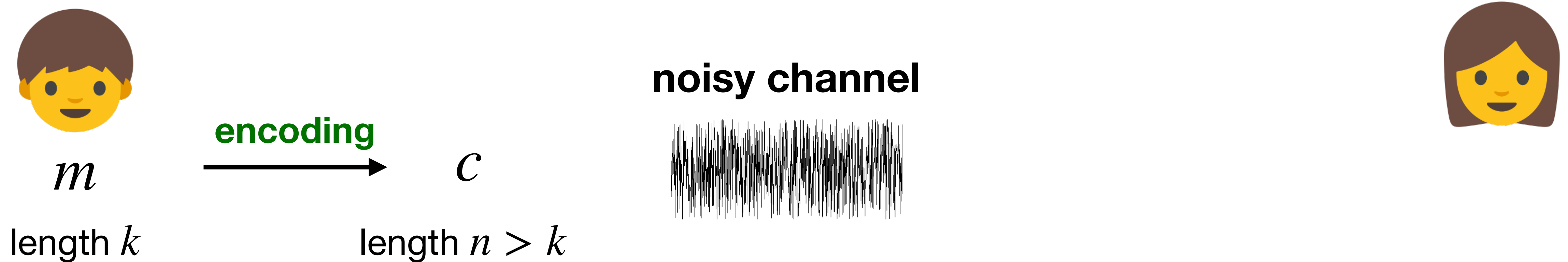
noisy channel




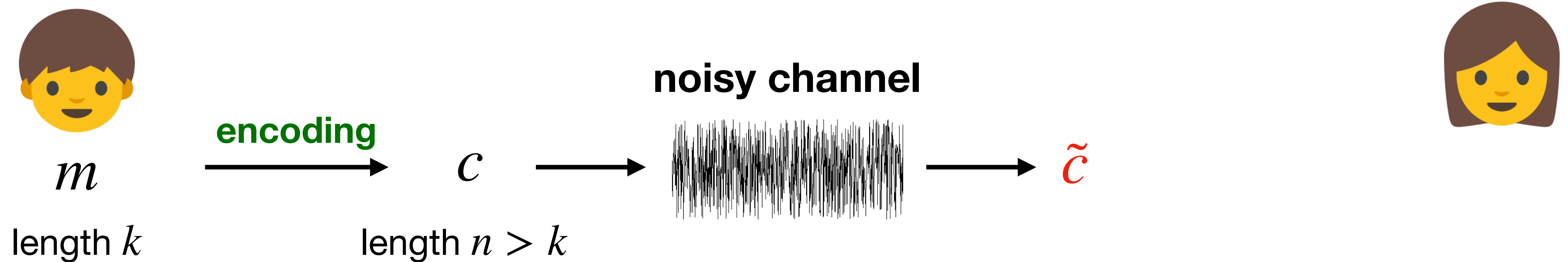
Now, something... completely unrelated?



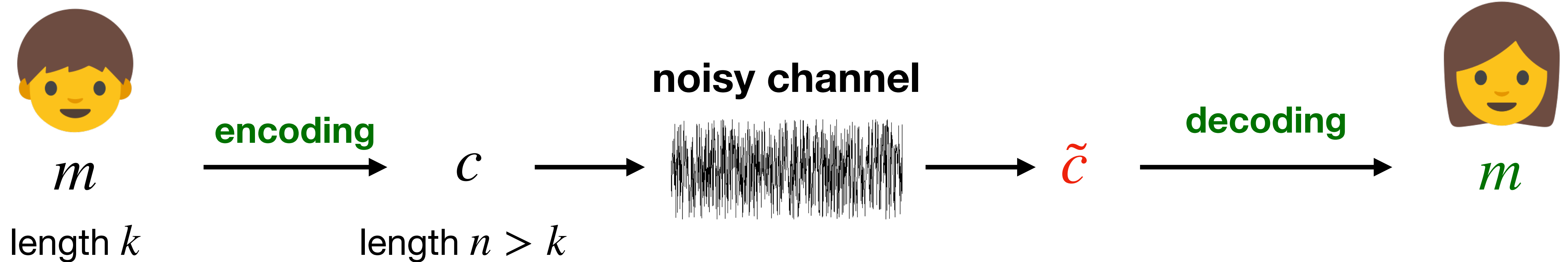
Now, something... completely unrelated?



Now, something... completely unrelated?

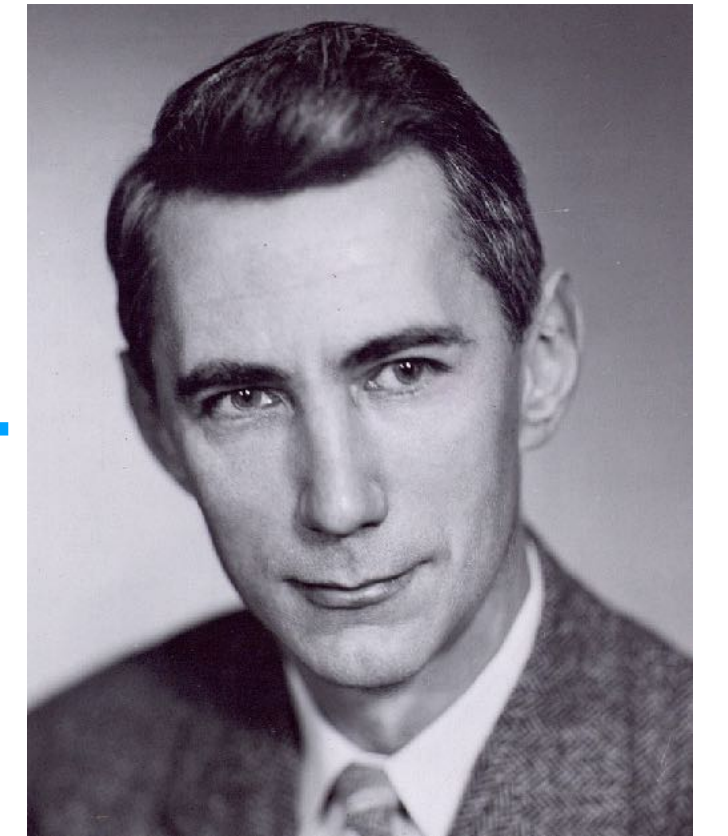


Now, something... completely unrelated?



How to model noise?

How to model noise?



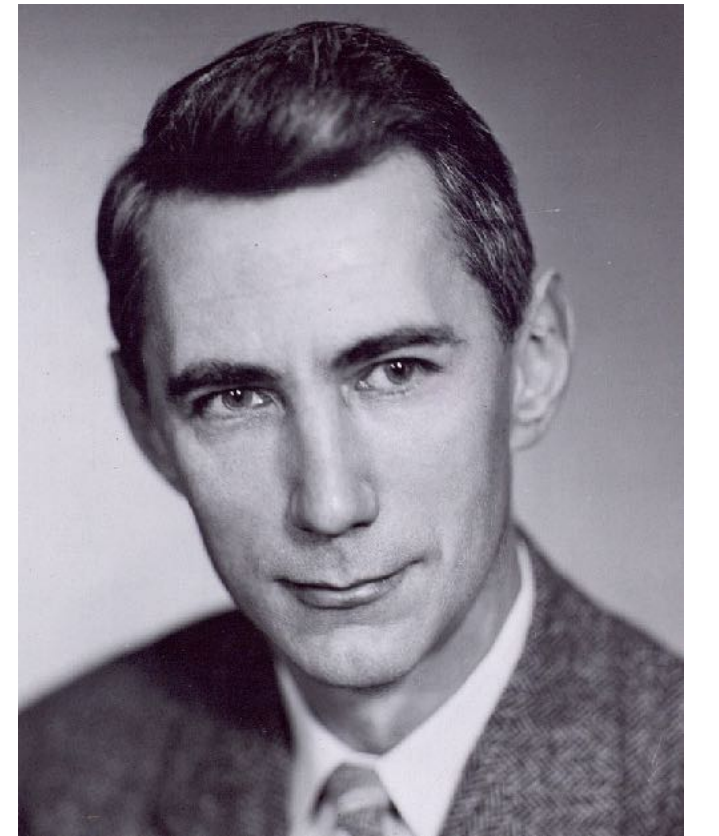
The probabilistic perspective (Shannon 1948)

- Noisy channel introduces random errors.
 - Each symbol independently replaced by a different one with probability p .

How to model noise?

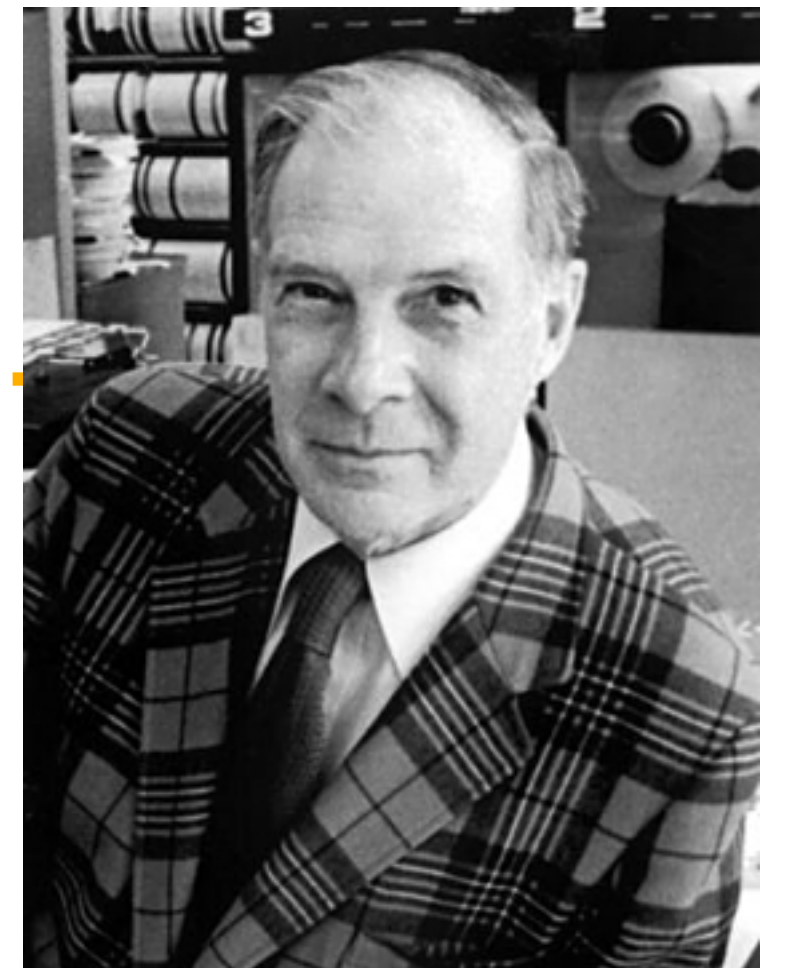
The probabilistic perspective (Shannon 1948)

- Noisy channel introduces random errors.
 - Each symbol independently replaced by a different one with probability p .



The combinatorial perspective (Hamming 1950)

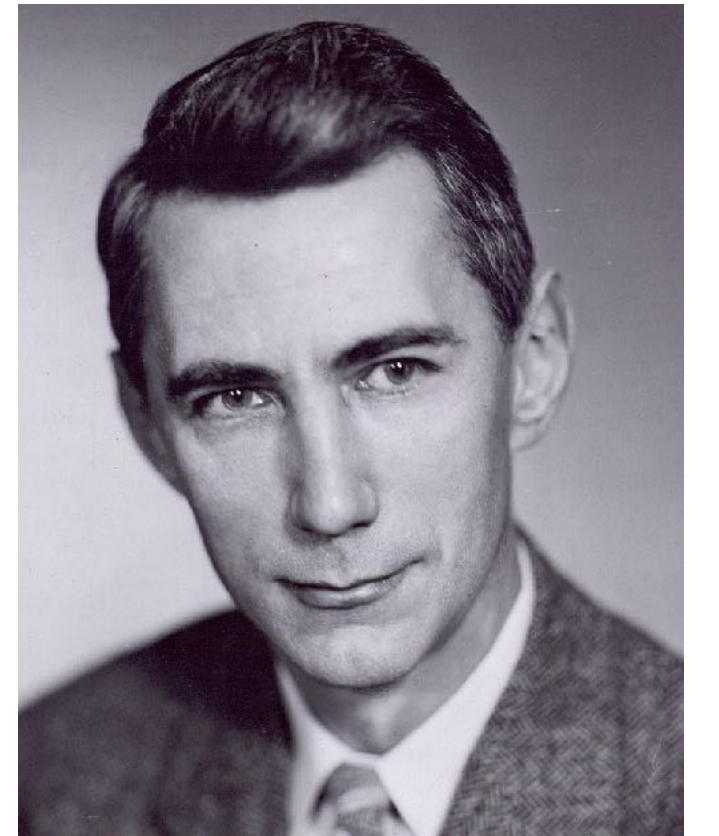
- Noisy channel introduces a bounded number of errors adversarially.
 - Channel may replace any t symbols by different symbols.



How to model noise?

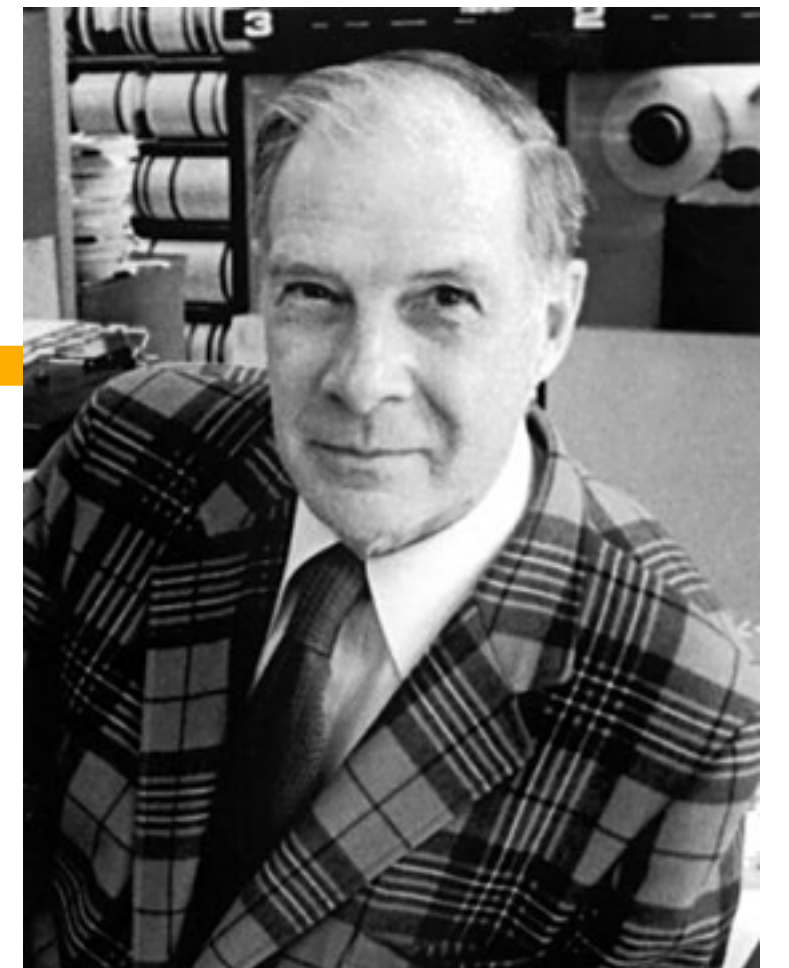
The probabilistic perspective (Shannon 1948)

- Noisy channel introduces random errors.
 - Each symbol independently replaced by a different one with probability p .

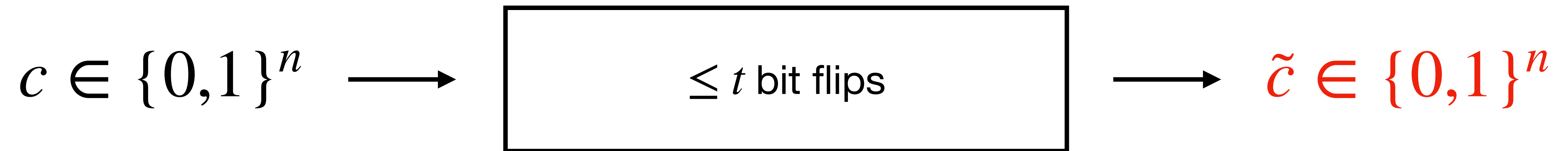


The combinatorial perspective (Hamming 1950)

- Noisy channel introduces a bounded number of errors adversarially
 - Channel may replace any t symbols by different symbols.



The geometry of errors



The geometry of errors



Hamming distance: $d(c, \tilde{c}) = |\{i : c_i \neq \tilde{c}_i\}| = \text{nr. coordinates on which } c \text{ and } \tilde{c} \text{ differ}$

The geometry of errors



Hamming distance: $d(c, \tilde{c}) = |\{i : c_i \neq \tilde{c}_i\}| = \text{nr. coordinates on which } c \text{ and } \tilde{c} \text{ differ}$

Hamming ball: $B_t(c) = \{\tilde{c} \in \Sigma^n : d(c, \tilde{c}) \leq t\}$

The geometry of errors



Hamming distance: $d(c, \tilde{c}) = |\{i : c_i \neq \tilde{c}_i\}|$ = nr. coordinates on which c and $\tilde{c}$ differ

Hamming ball: $B_t(c) = \{\tilde{c} \in \Sigma^n : d(c, \tilde{c}) \leq t\}$

= set of strings that can be obtained from c via $\leq t$ bit flips.

The geometry of errors



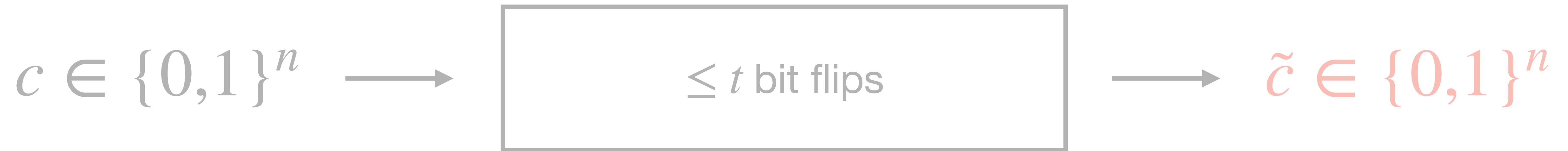
Hamming distance: $d(c, \tilde{c}) = |\{i : c_i \neq \tilde{c}_i\}|$ = nr. coordinates on which c and $\tilde{c}$ differ

Hamming ball: $B_t(c) = \{\tilde{c} \in \Sigma^n : d(c, \tilde{c}) \leq t\}$

= set of strings that can be obtained from c via $\leq t$ bit flips.

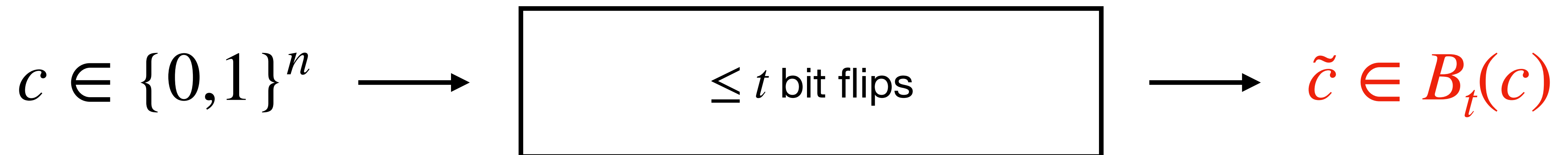
Example: $d(10\textcolor{red}{1}00, 100\textcolor{red}{1}0) = 2$

The geometry of errors

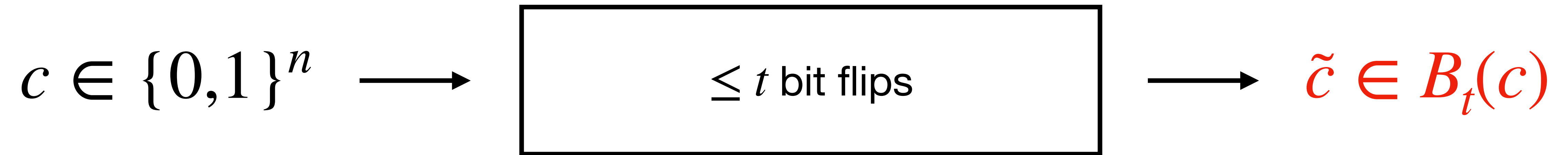


Hamming distance: $d(c, \tilde{c}) = |\{i : c_i \neq \tilde{c}_i\}|$ = nr. coordinates on which c and $\tilde{c}$ differ

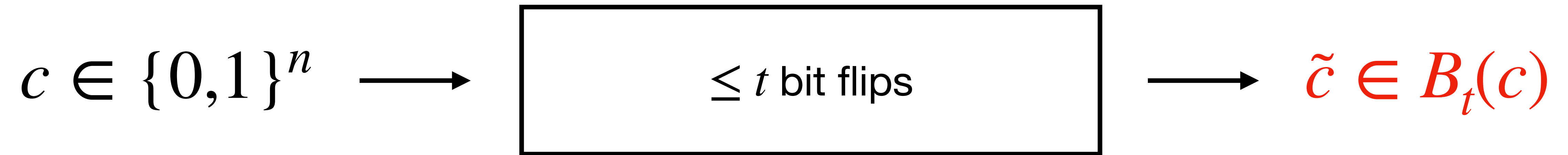
Hamming ball: $B_t(c) = \{\tilde{c} \in \Sigma^n : d(c, \tilde{c}) \leq t\}$ = set of strings at distance $\leq t$ from c .



Error-correcting codes = Sphere packings

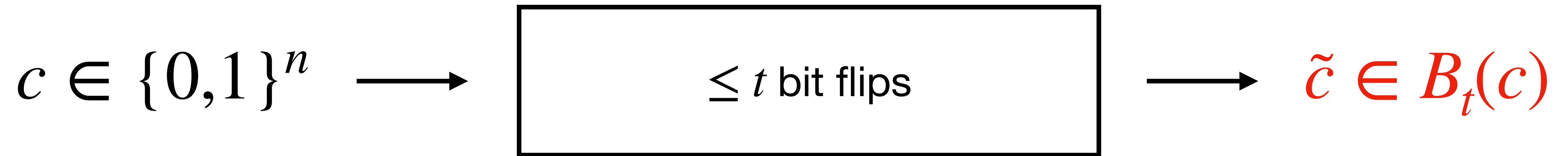


Error-correcting codes = Sphere packings



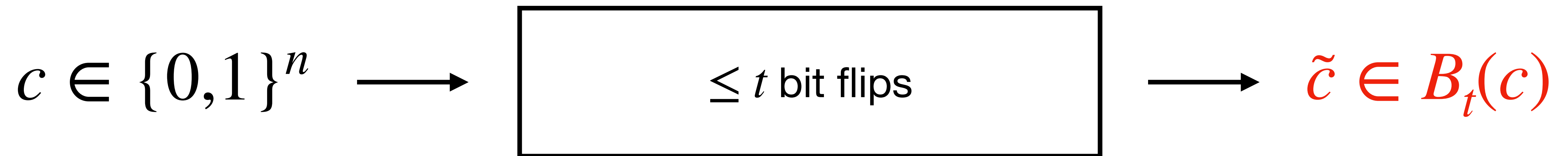
- We call a subset $C \subseteq \{0,1\}^n$ a ***code of block length n*** ;

Error-correcting codes = Sphere packings



- We call a subset $C \subseteq \{0,1\}^n$ a **code of block length n** ;
- We say that C **corrects t errors** if $B_t(c) \cap B_t(c') = \emptyset$ for any distinct codewords $c, c' \in C$.

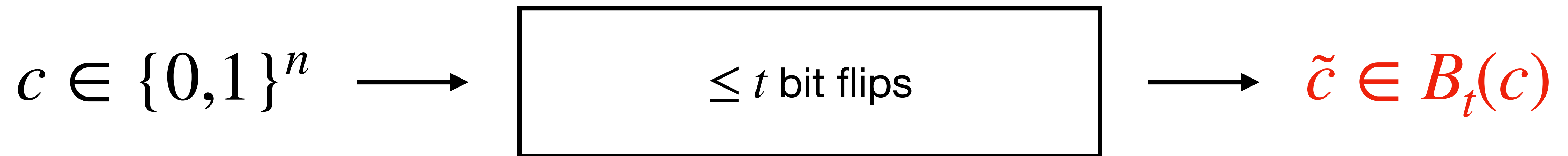
Error-correcting codes = Sphere packings



- We call a subset $C \subseteq \{0,1\}^n$ a **code of block length n** ;
- We say that C **corrects t errors** if $B_t(c) \cap B_t(c') = \emptyset$ for any distinct codewords $c, c' \in C$.

$$C \text{ corrects } t \text{ errors} \iff \min_{c, c' \in C: c \neq c'} d(c, c') > 2t.$$

Error-correcting codes = Sphere packings



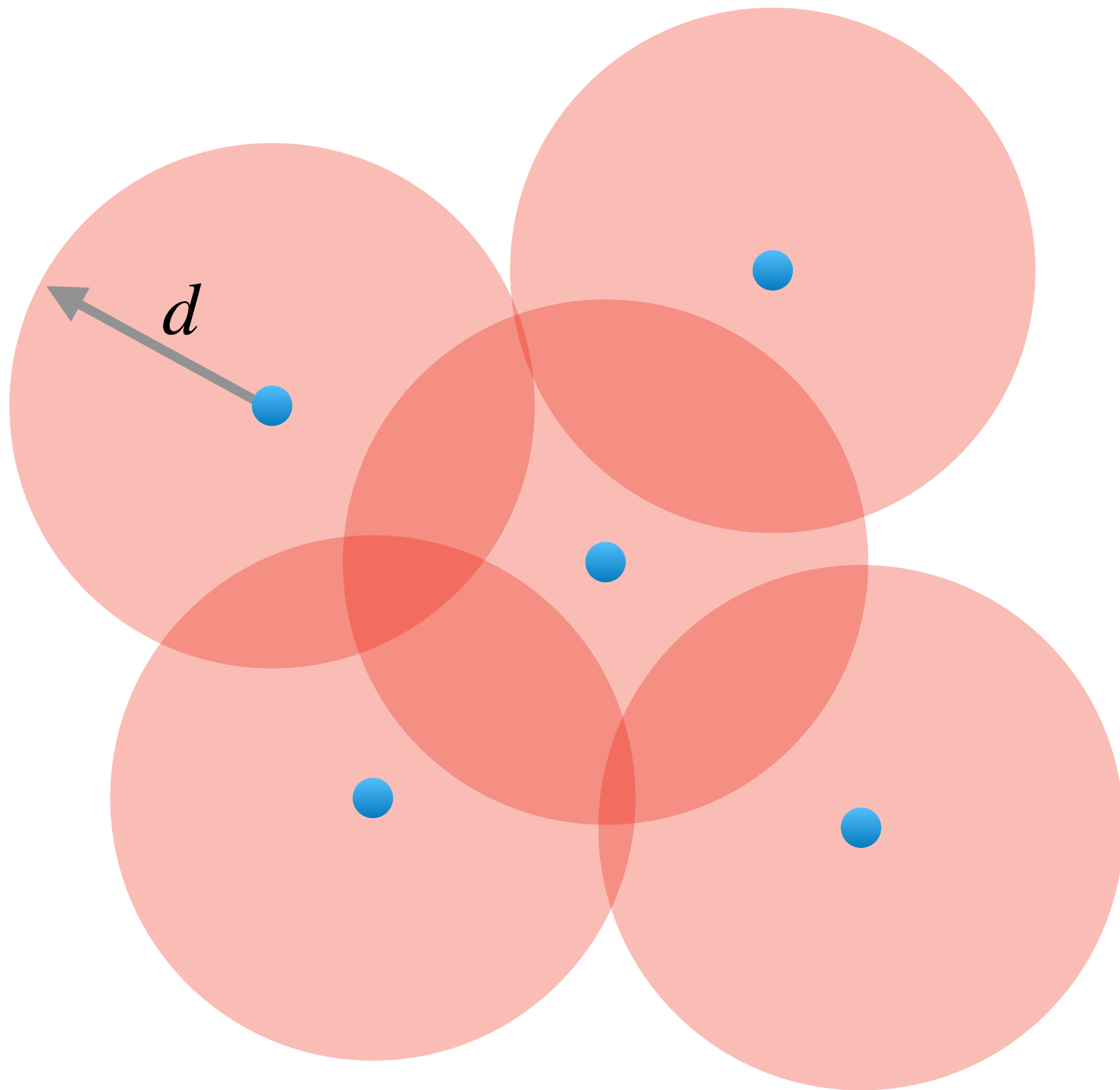
- We call a subset $C \subseteq \{0,1\}^n$ a **code of block length n** ;
- We say that C **corrects t errors** if $B_t(c) \cap B_t(c') = \emptyset$ for any distinct codewords $c, c' \in C$.

$$C \text{ corrects } t \text{ errors} \iff \min_{c, c' \in C: c \neq c'} d(c, c') > 2t.$$

minimum (Hamming) distance of C

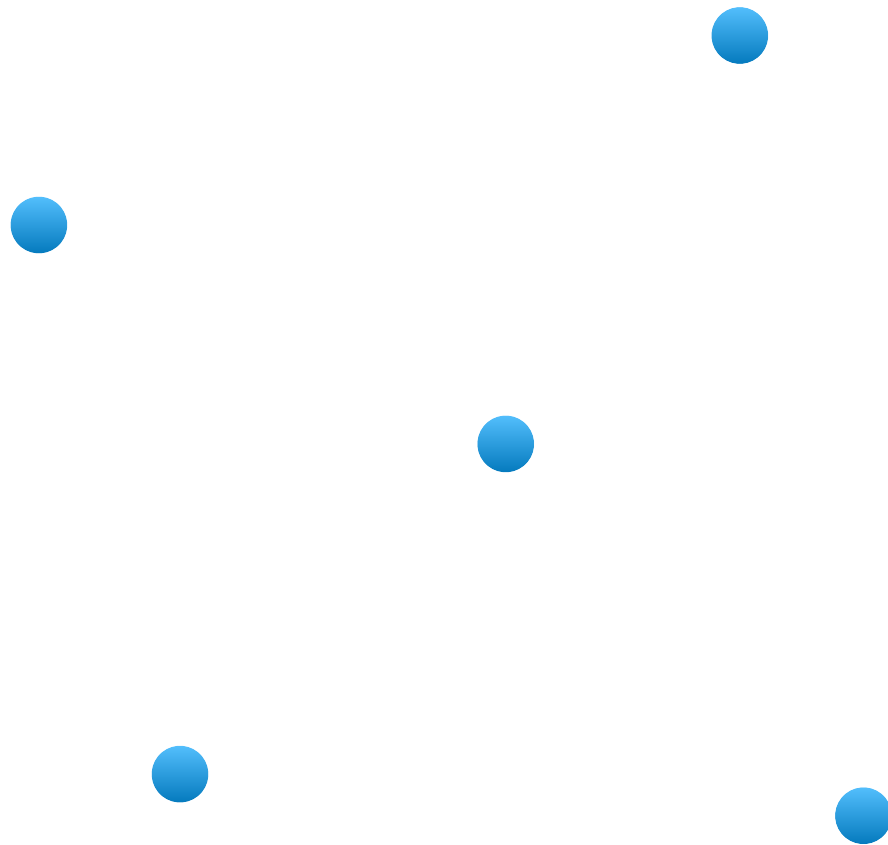
Error-correcting codes = Sphere packings

Code with minimum distance $d > 2t$



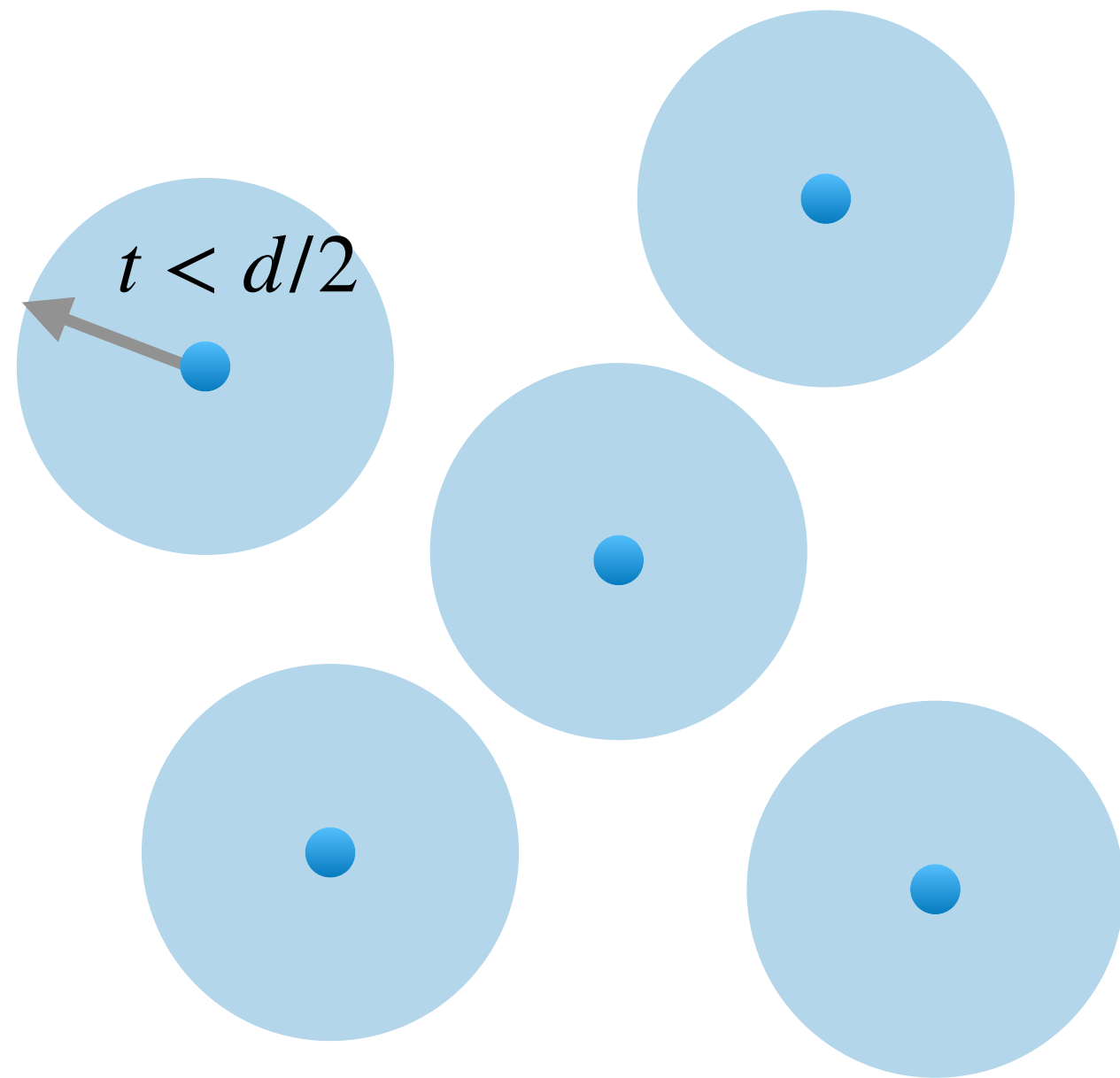
Error-correcting codes = Sphere packings

Code with minimum distance $d > 2t$



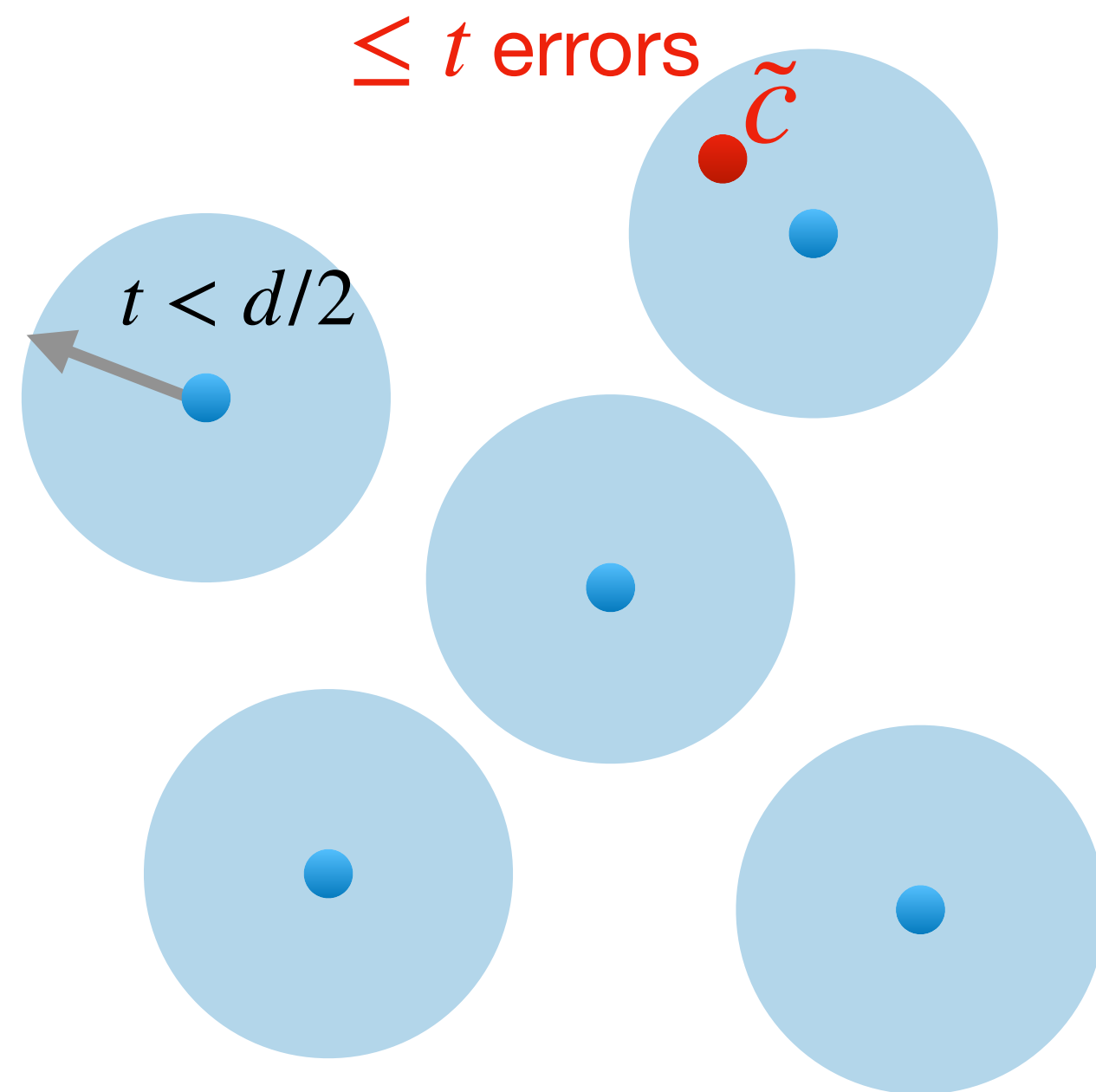
Error-correcting codes = Sphere packings

Code with minimum distance $d > 2t$



Error-correcting codes = Sphere packings

Code with minimum distance $d > 2t$



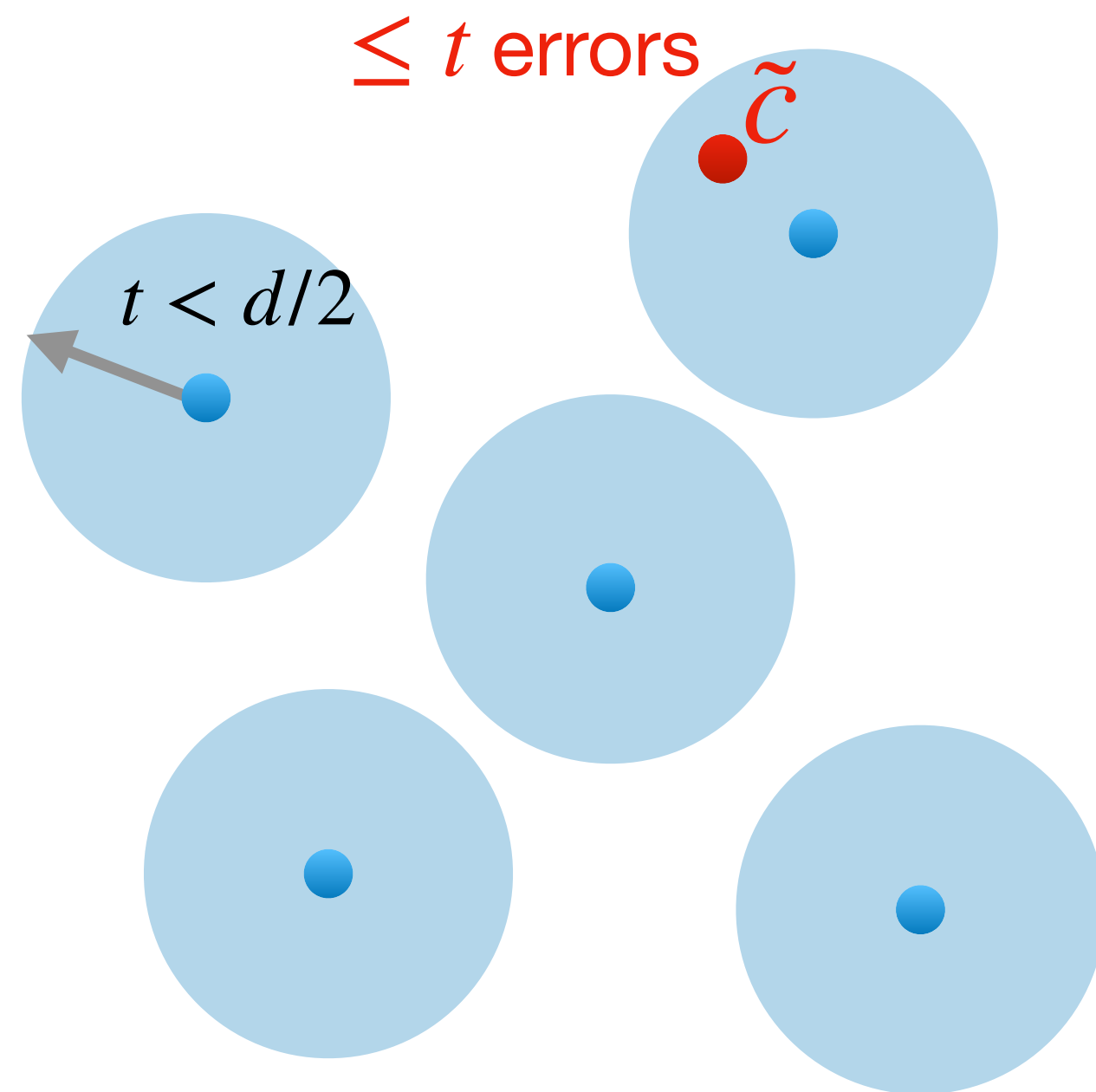
Error-correcting codes = Sphere packings

Code with minimum distance $d > 2t$

Pack more
spheres



Can transmit more
messages



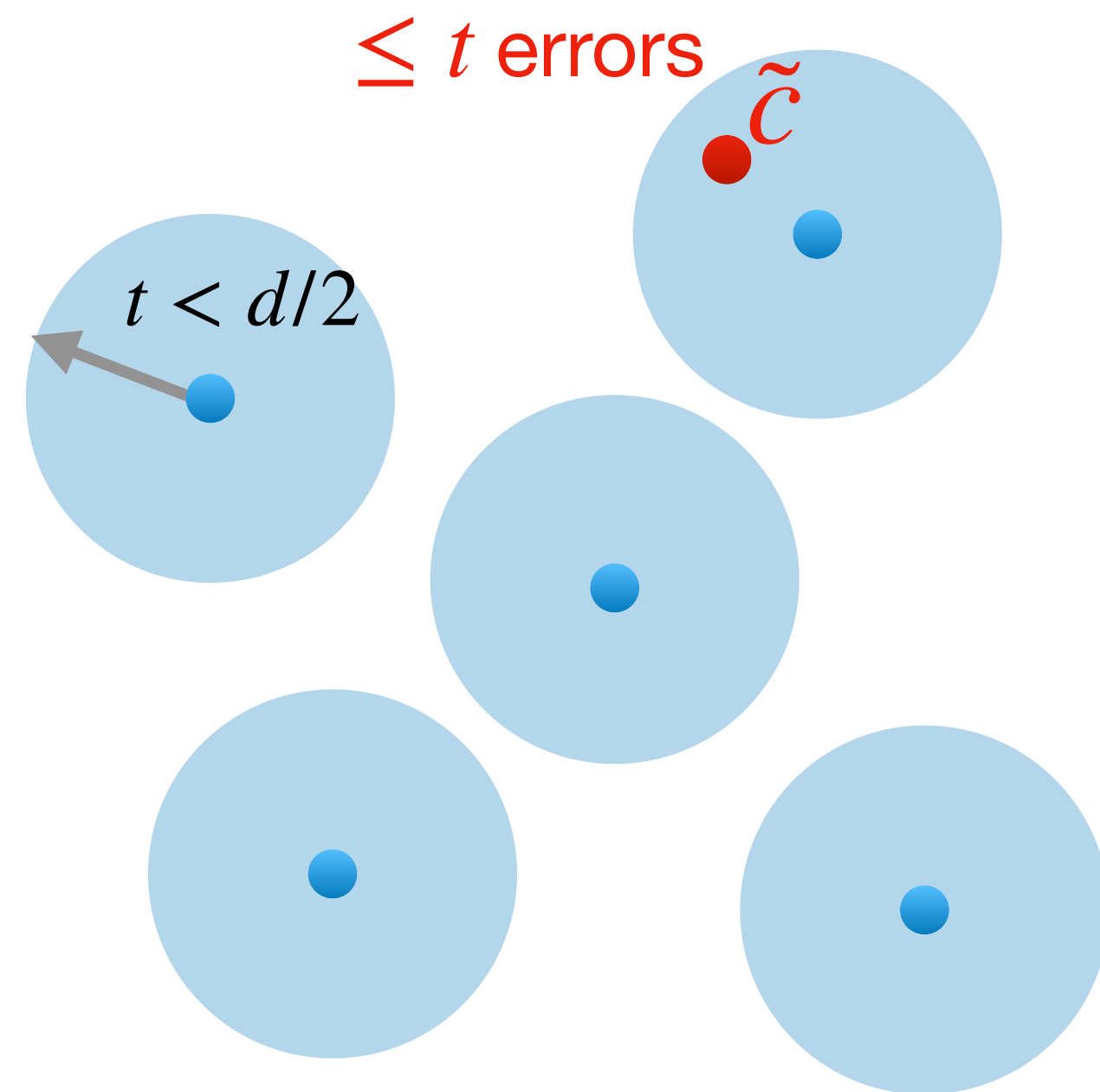
Error-correcting codes = Sphere packings

Code with minimum distance $d > 2t$

Pack more
spheres



Can transmit more
messages



The most basic questions:

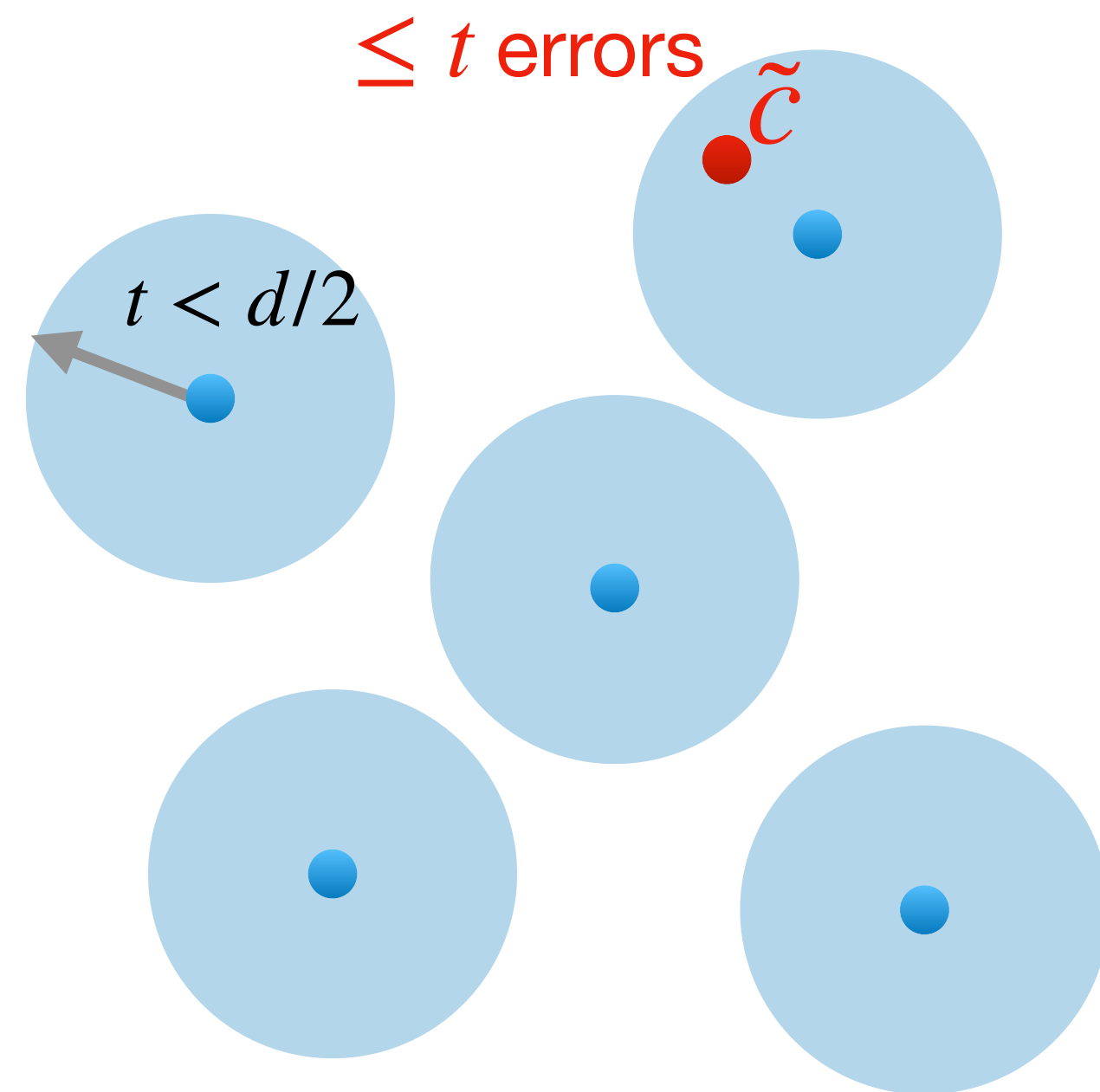
Error-correcting codes = Sphere packings

Code with minimum distance $d > 2t$

Pack more
spheres



Can transmit more
messages



The most basic questions:

- What's the size of the largest code with block length n that corrects t errors?

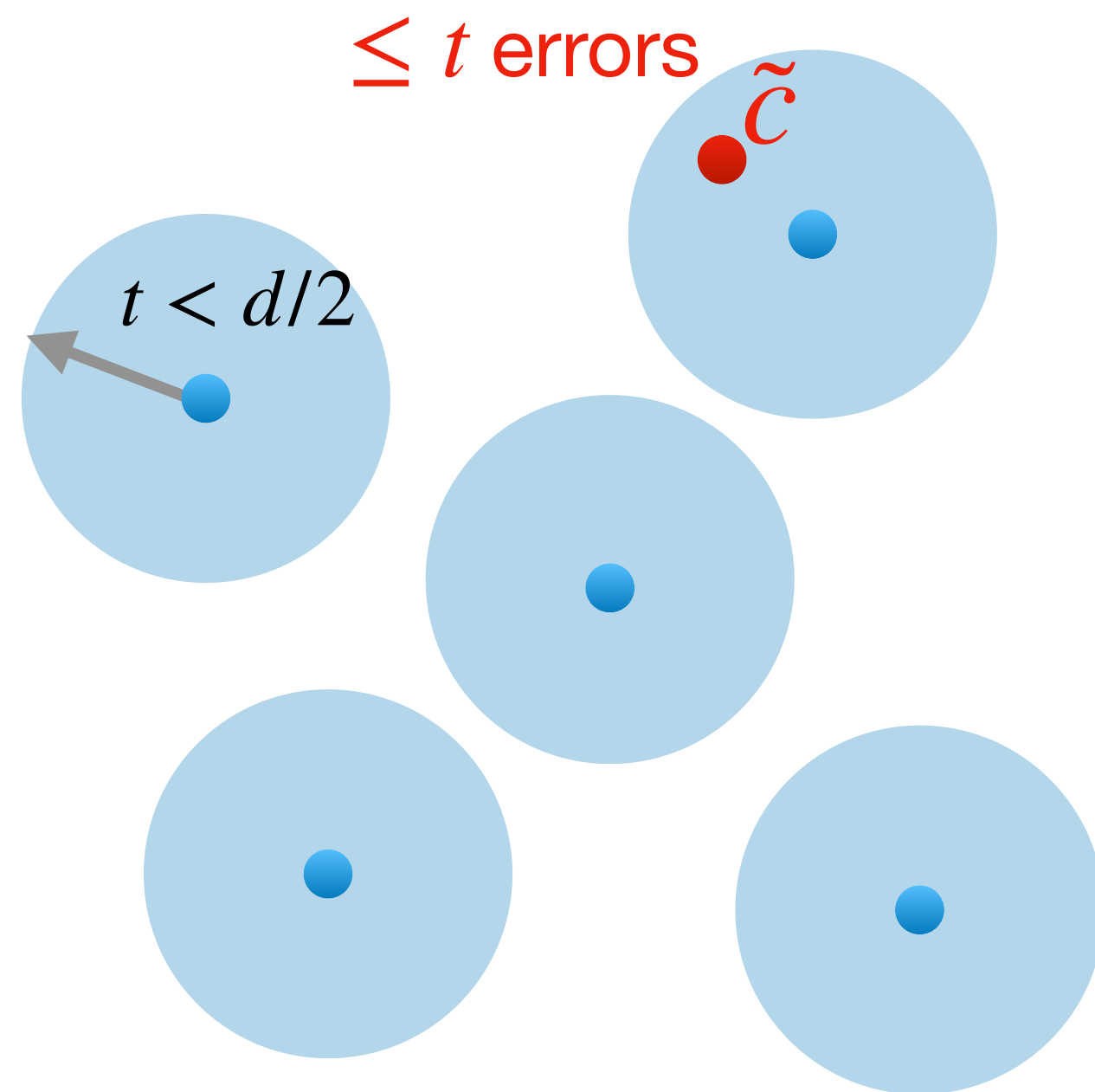
Error-correcting codes = Sphere packings

Code with minimum distance $d > 2t$

Pack more
spheres



Can transmit more
messages



The most basic questions:

- What's the size of the largest code with block length n that corrects t errors?
- How can we encode and decode efficiently?

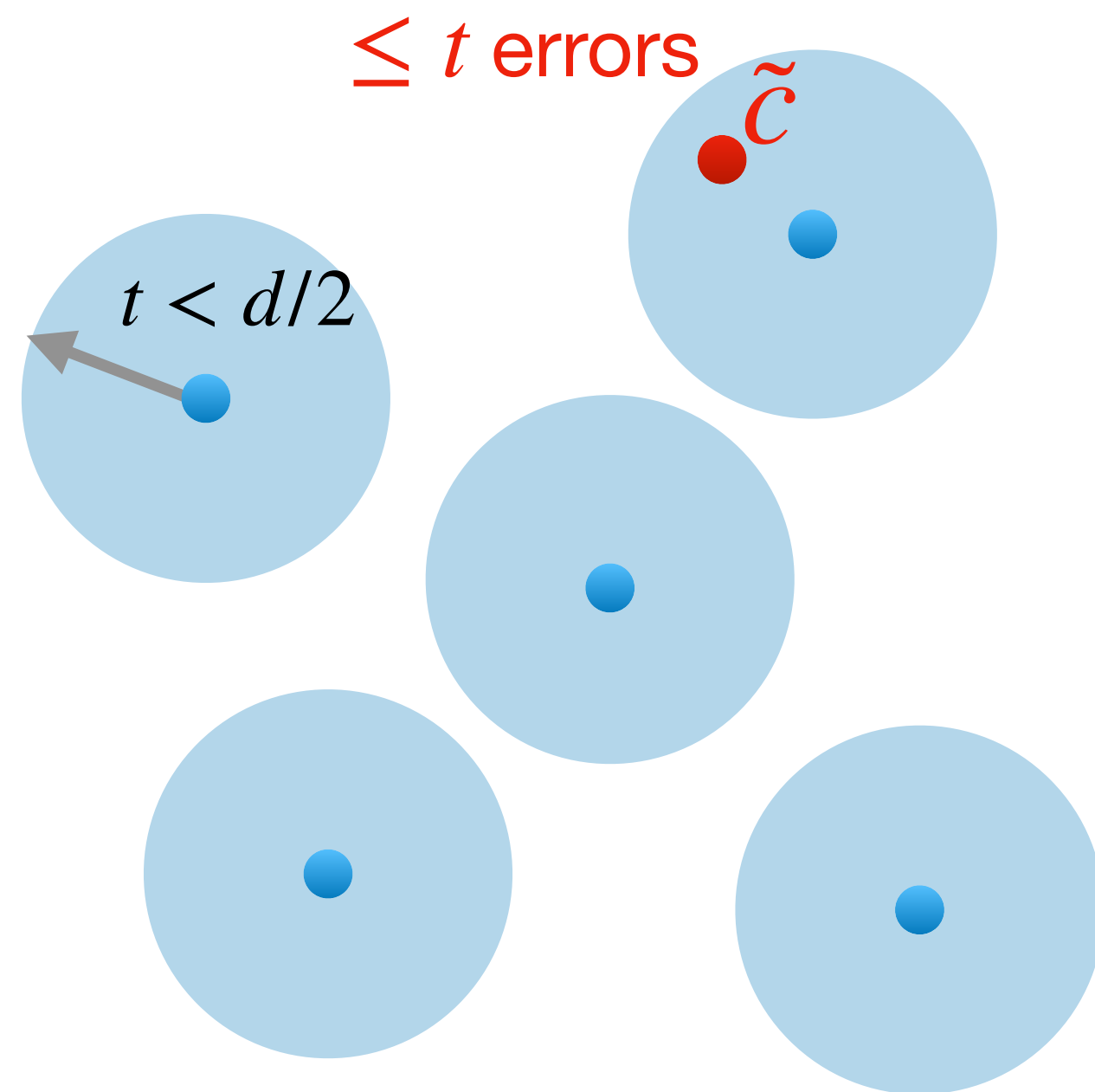
Error-correcting codes = Sphere packings

Code with minimum distance $d > 2t$

Pack more
spheres



Can transmit more
messages



The most basic questions:

- What's the size of the largest code with block length n that corrects t errors?
- How can we encode and decode efficiently?

Warmup: Correcting 1 error

Warmup: Correcting 1 error

Equivalent to constructing large $C \subseteq \{0,1\}^n$ with minimum distance ≥ 3 .

Warmup: Correcting 1 error

Equivalent to constructing large $C \subseteq \{0,1\}^n$ with minimum distance ≥ 3 .

A first approach: repetition!

Warmup: Correcting 1 error

Equivalent to constructing large $C \subseteq \{0,1\}^n$ with minimum distance ≥ 3 .

A first approach: repetition!

$0101 \mapsto 000111000111$

Warmup: Correcting 1 error

Equivalent to constructing large $C \subseteq \{0,1\}^n$ with minimum distance ≥ 3 .

A first approach: repetition!

$0101 \mapsto 000111000111$

Get code $C \subseteq \{0,1\}^n$ of size $2^{n/3}$ correcting 1 error.

Warmup: Correcting 1 error

Equivalent to constructing large $C \subseteq \{0,1\}^n$ with minimum distance ≥ 3 .

A first approach: repetition!

$0101 \mapsto 000111000111$

Get code $C \subseteq \{0,1\}^n$ of size $2^{n/3}$ correcting 1 error.

Can we do better?

Hamming codes (or, linear algebra is nice)

Hamming codes (or, linear algebra is nice)

Linear subspaces of $\mathbb{F}_2^n$ turn out to be a great tool for error-correction.

Hamming codes (or, linear algebra is nice)

Linear subspaces of $\mathbb{F}_2^n$ turn out to be a great tool for error-correction.

Write $C = \ker(H)$ for a carefully chosen matrix H .

Hamming codes (or, linear algebra is nice)

Linear subspaces of $\mathbb{F}_2^n$ turn out to be a great tool for error-correction.

Write $C = \ker(H)$ for a carefully chosen matrix H .

Some nice properties of C (easy consequences of linearity):

Hamming codes (or, linear algebra is nice)

Linear subspaces of $\mathbb{F}_2^n$ turn out to be a great tool for error-correction.

Write $C = \ker(H)$ for a carefully chosen matrix H .

Some nice properties of C (easy consequences of linearity):

Min. dist. of $C = \min_{c \in C \setminus \{0\}} |\{i : c_i \neq 0\}| = \text{min. nr. nonzero coordinates of nonzero } c \in C ;$

Hamming codes (or, linear algebra is nice)

Linear subspaces of $\mathbb{F}_2^n$ turn out to be a great tool for error-correction.

Write $C = \ker(H)$ for a carefully chosen matrix H .

Some nice properties of C (easy consequences of linearity):

Min. dist. of $C = \min_{c \in C \setminus \{0\}} |\{i : c_i \neq 0\}| = \text{min. nr. nonzero coordinates of nonzero } c \in C ;$

Min. dist. of C is $\geq d$ iff all subsets of $d - 1$ columns of H are linearly independent.

Hamming codes (or, linear algebra is nice)

Minimum distance of $C = \ker(H)$
is at least 3



any subset of 2 columns of H
is linearly independent

Hamming codes (or, linear algebra is nice)

Minimum distance of $C = \ker(H)$
is at least 3



any subset of 2 columns of H
is linearly independent

this just means that H cannot have
repeated columns!

Hamming codes (or, linear algebra is nice)

Minimum distance of $C = \ker(H)$
is at least 3



any subset of 2 columns of H
is linearly independent

this just means that H cannot have
repeated columns!

There is an $h \times (2^h - 1)$ matrix H whose columns are all nonzero and distinct.

Hamming codes (or, linear algebra is nice)

Minimum distance of $C = \ker(H)$
is at least 3



any subset of 2 columns of H
is linearly independent

this just means that H cannot have
repeated columns!

There is an $h \times (2^h - 1)$ matrix H whose columns are all nonzero and distinct.

$$h = 3 \quad \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Hamming codes (or, linear algebra is nice)

Minimum distance of $C = \ker(H)$
is at least 3



any subset of 2 columns of H
is linearly independent

this just means that H cannot have
repeated columns!

There is an $h \times (2^h - 1)$ matrix H whose columns are all nonzero and distinct.

- Block length $n = 2^h - 1$

$$h = 3 \quad \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Hamming codes (or, linear algebra is nice)

Minimum distance of $C = \ker(H)$
is at least 3



any subset of 2 columns of H
is linearly independent

this just means that H cannot have
repeated columns!

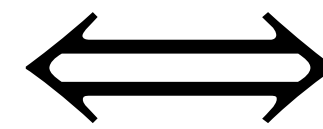
There is an $h \times (2^h - 1)$ matrix H whose columns are all nonzero and distinct.

$$h = 3 \quad \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

- Block length $n = 2^h - 1$
- $h = \log(n + 1)$

Hamming codes (or, linear algebra is nice)

Minimum distance of $C = \ker(H)$
is at least 3



any subset of 2 columns of H
is linearly independent

this just means that H cannot have
repeated columns!

There is an $h \times (2^h - 1)$ matrix H whose columns are all nonzero and distinct.

$$h = 3 \quad \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

- Block length $n = 2^h - 1$
- $h = \log(n + 1)$
- Size $2^{n-h} = \frac{2^n}{n + 1}$

Hamming codes (or, linear algebra is nice)

Minimum distance of $C = \ker(H)$
is at least 3



any subset of 2 columns of H
is linearly independent

this just means that H cannot have
repeated columns!

There is an $h \times (2^h - 1)$ matrix H whose columns are all nonzero and distinct.

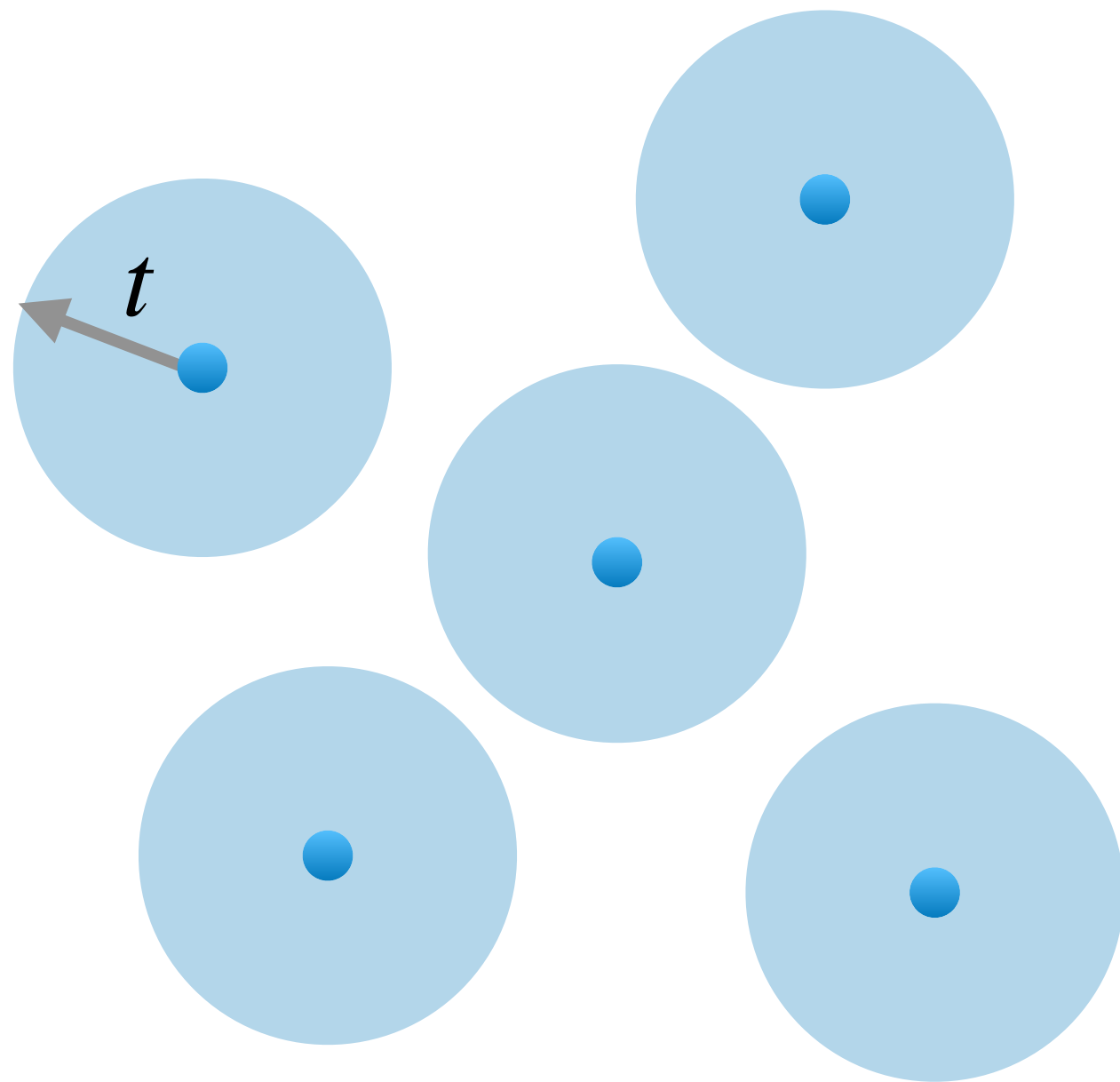
$$h = 3 \quad \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

- Block length $n = 2^h - 1$
- $h = \log(n + 1)$
- Size $2^{n-h} = \frac{2^n}{n + 1}$
- Corrects 1 error

Can we do better than Hamming codes?

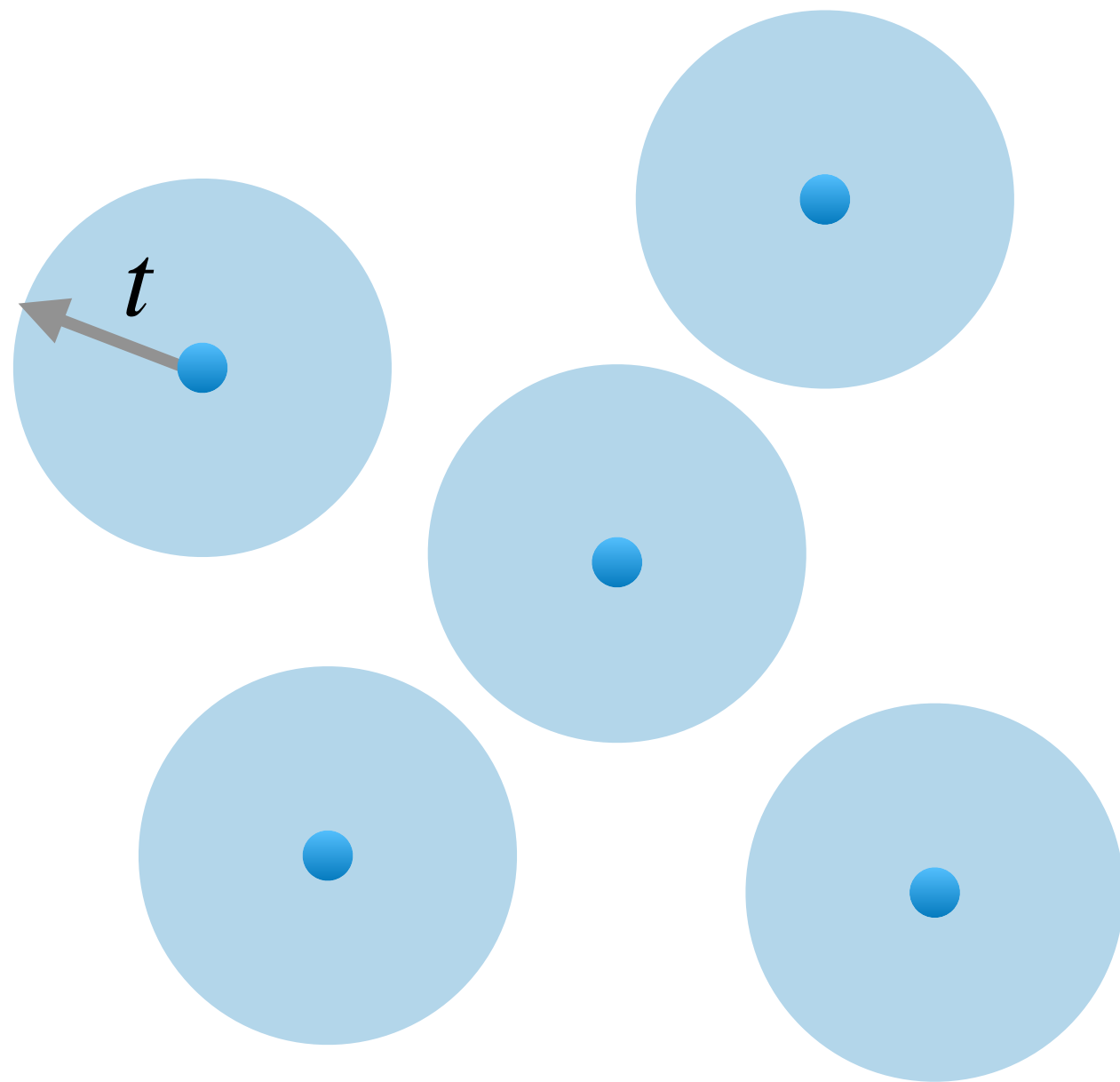
Can we do better than Hamming codes?

t -error-correcting code $C \subseteq \{0,1\}^n$



Can we do better than Hamming codes?

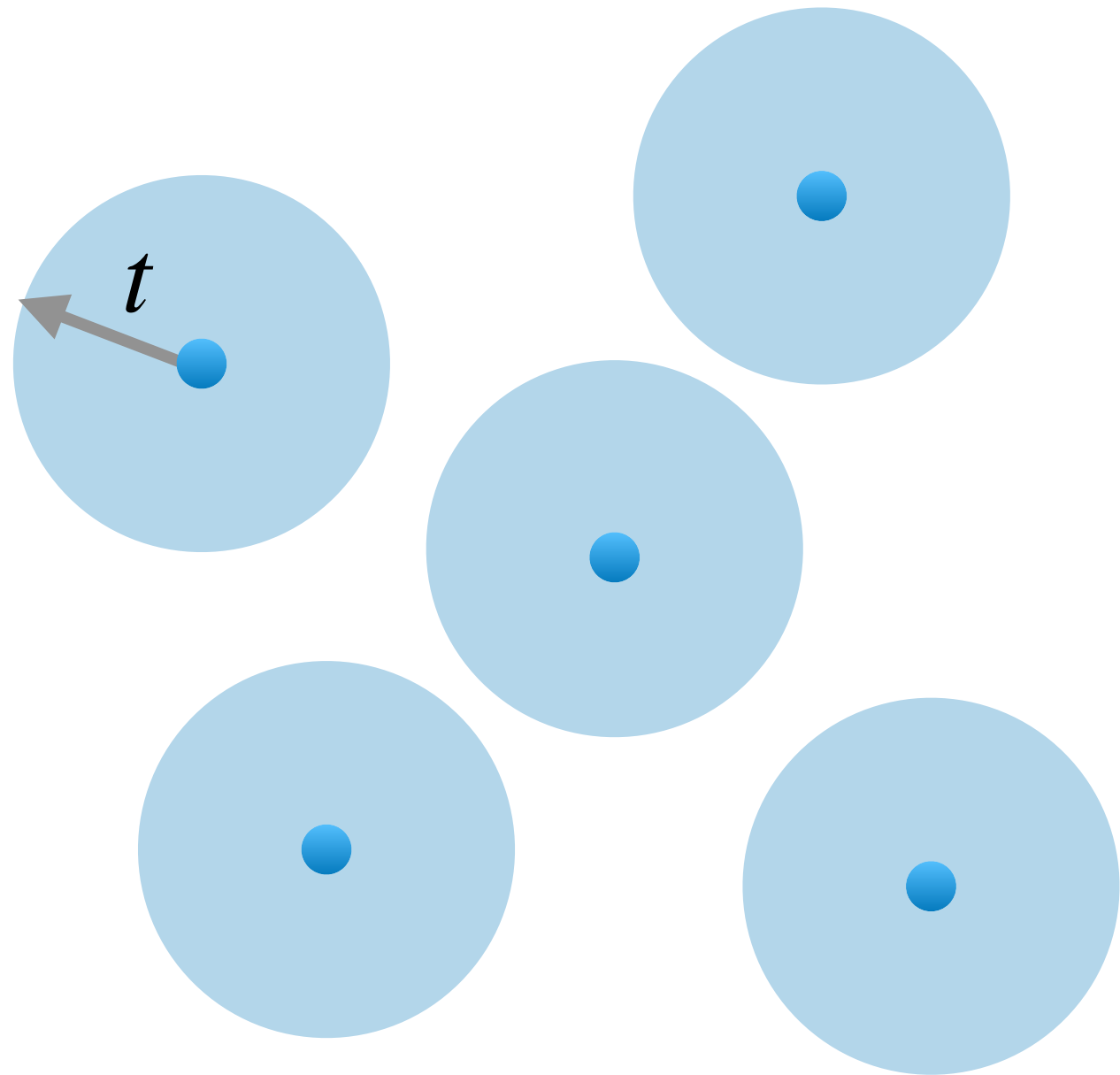
t -error-correcting code $C \subseteq \{0,1\}^n$



$$2^n \geq \left| \bigcup_{c \in C} B_t(c) \right|$$

Can we do better than Hamming codes?

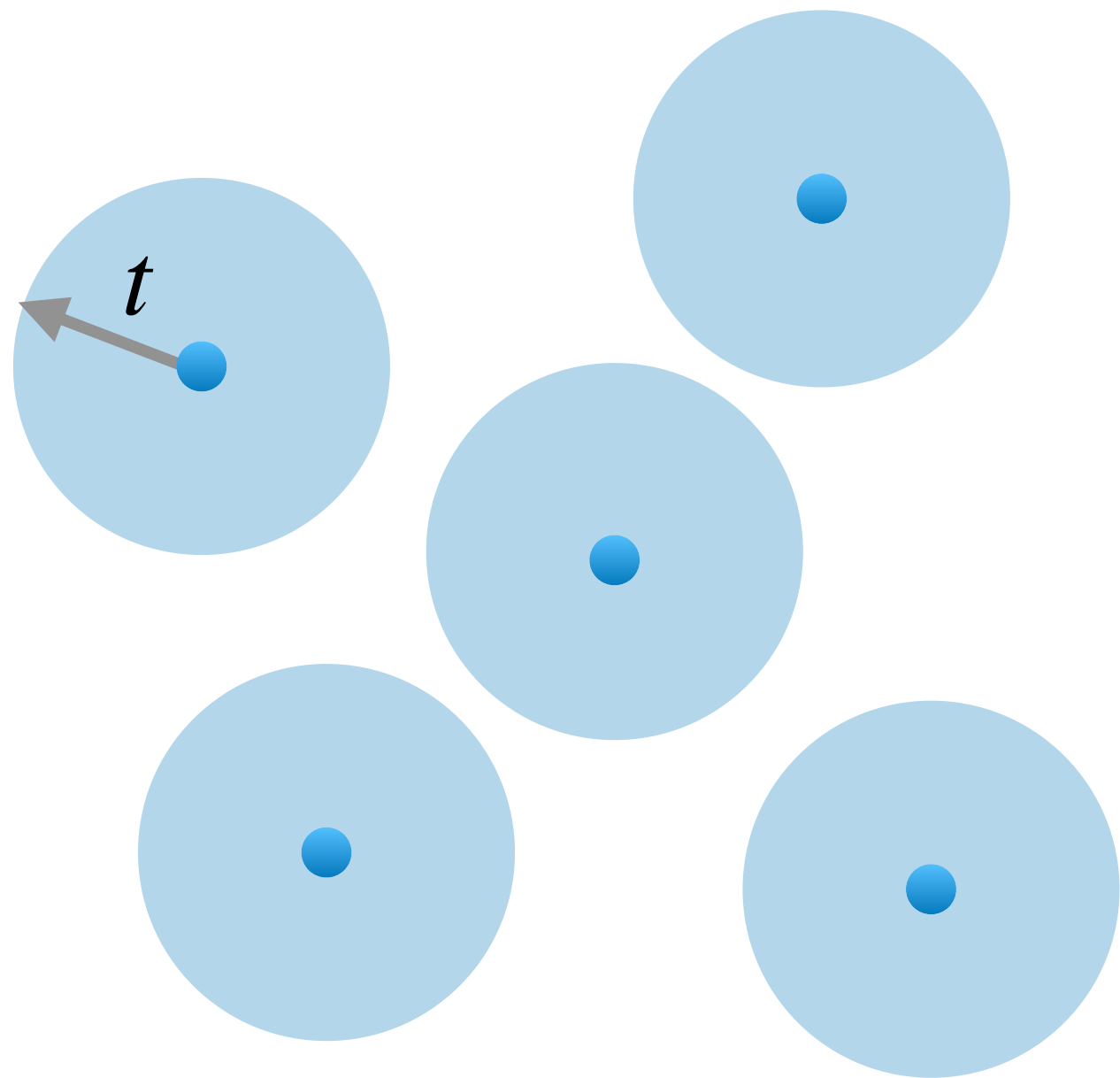
t -error-correcting code $C \subseteq \{0,1\}^n$



$$2^n \geq \left| \bigcup_{c \in C} B_t(c) \right|$$
$$= \sum_{c \in C} |B_t(c)|$$

Can we do better than Hamming codes?

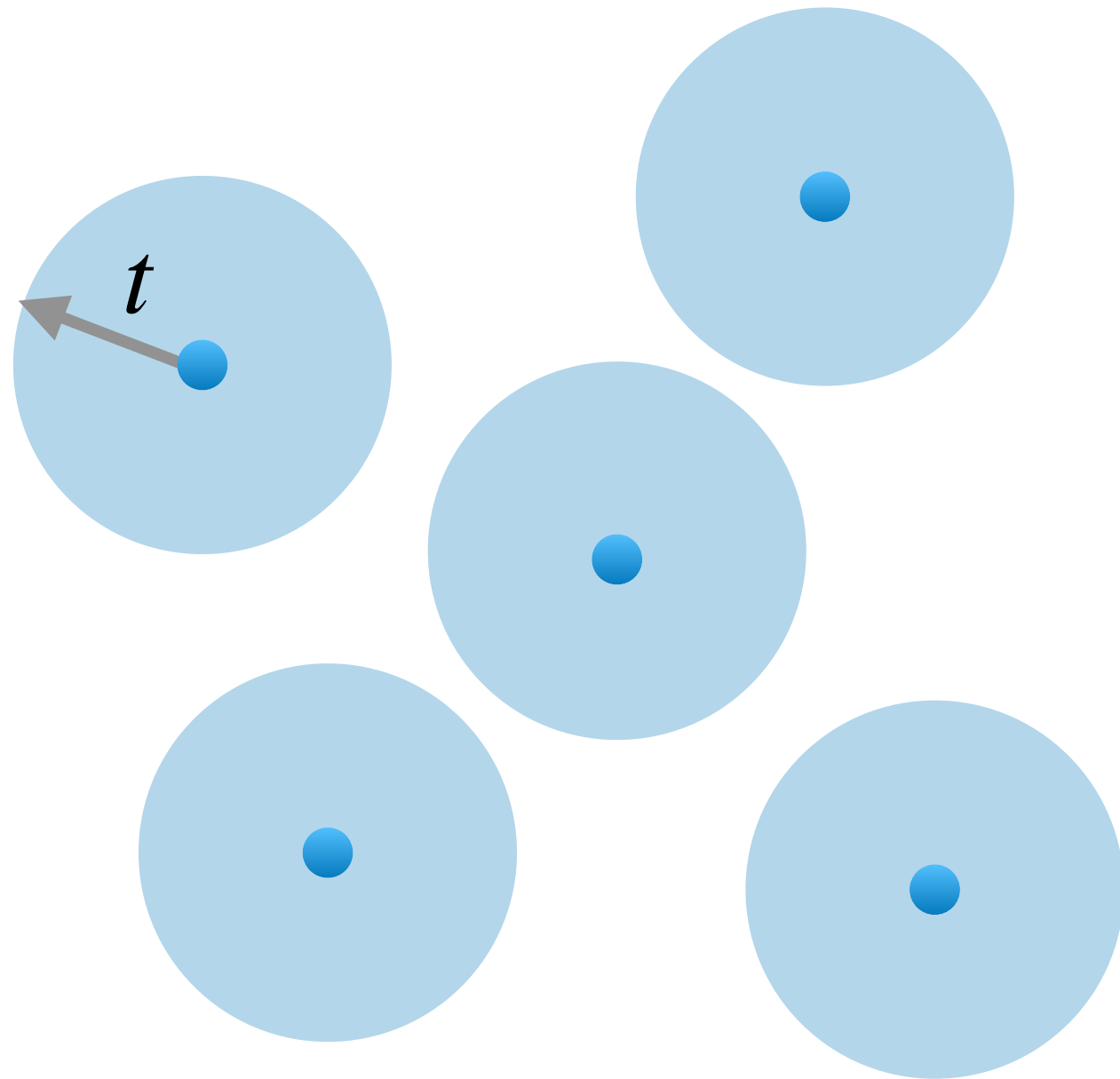
t -error-correcting code $C \subseteq \{0,1\}^n$



$$\begin{aligned} 2^n &\geq \left| \bigcup_{c \in C} B_t(c) \right| \\ &= \sum_{c \in C} |B_t(c)| \\ &= |C| \cdot |B_t(0)| \end{aligned}$$

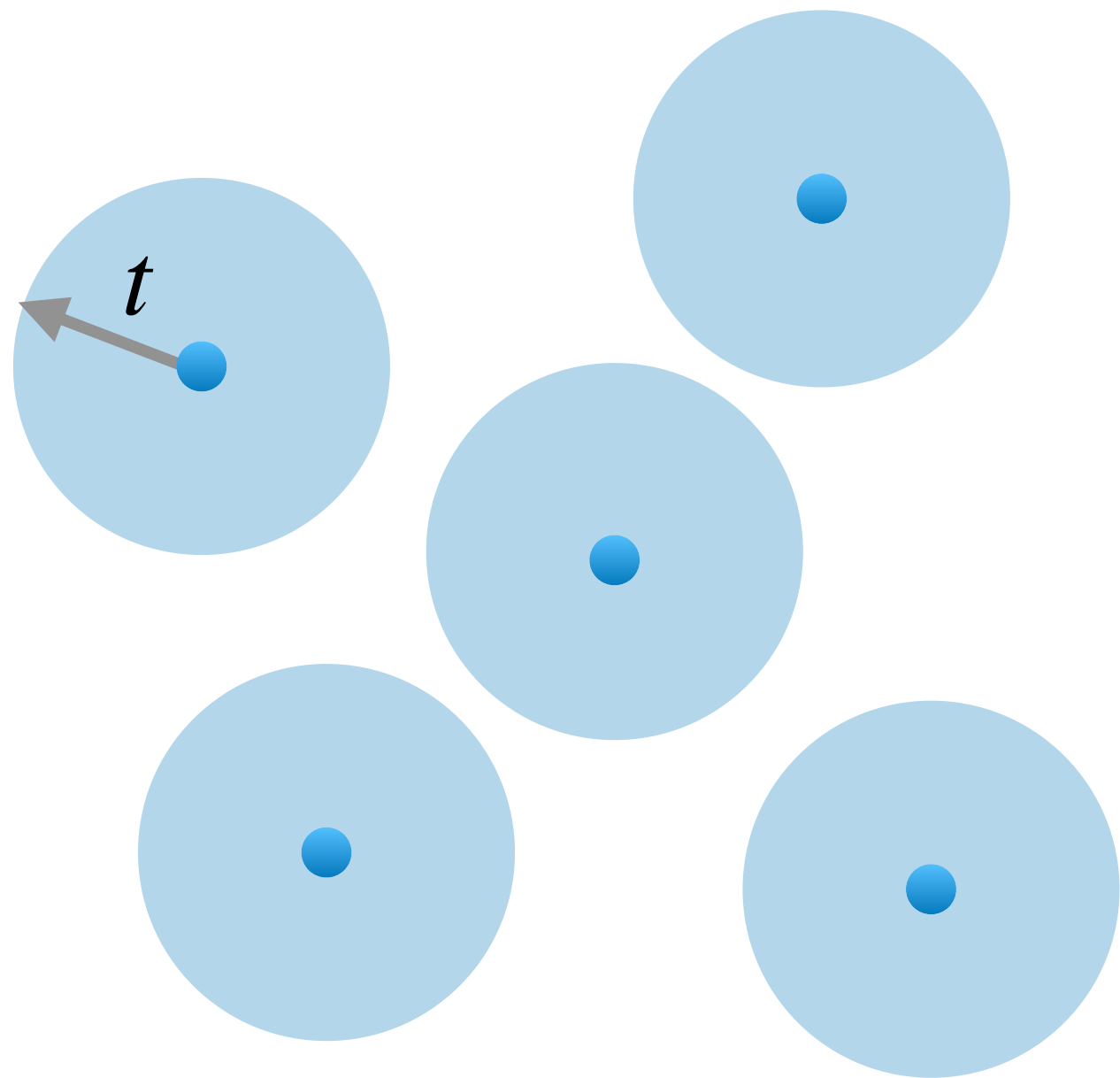
Can we do better than Hamming codes?

t -error-correcting code $C \subseteq \{0,1\}^n$



Can we do better than Hamming codes?

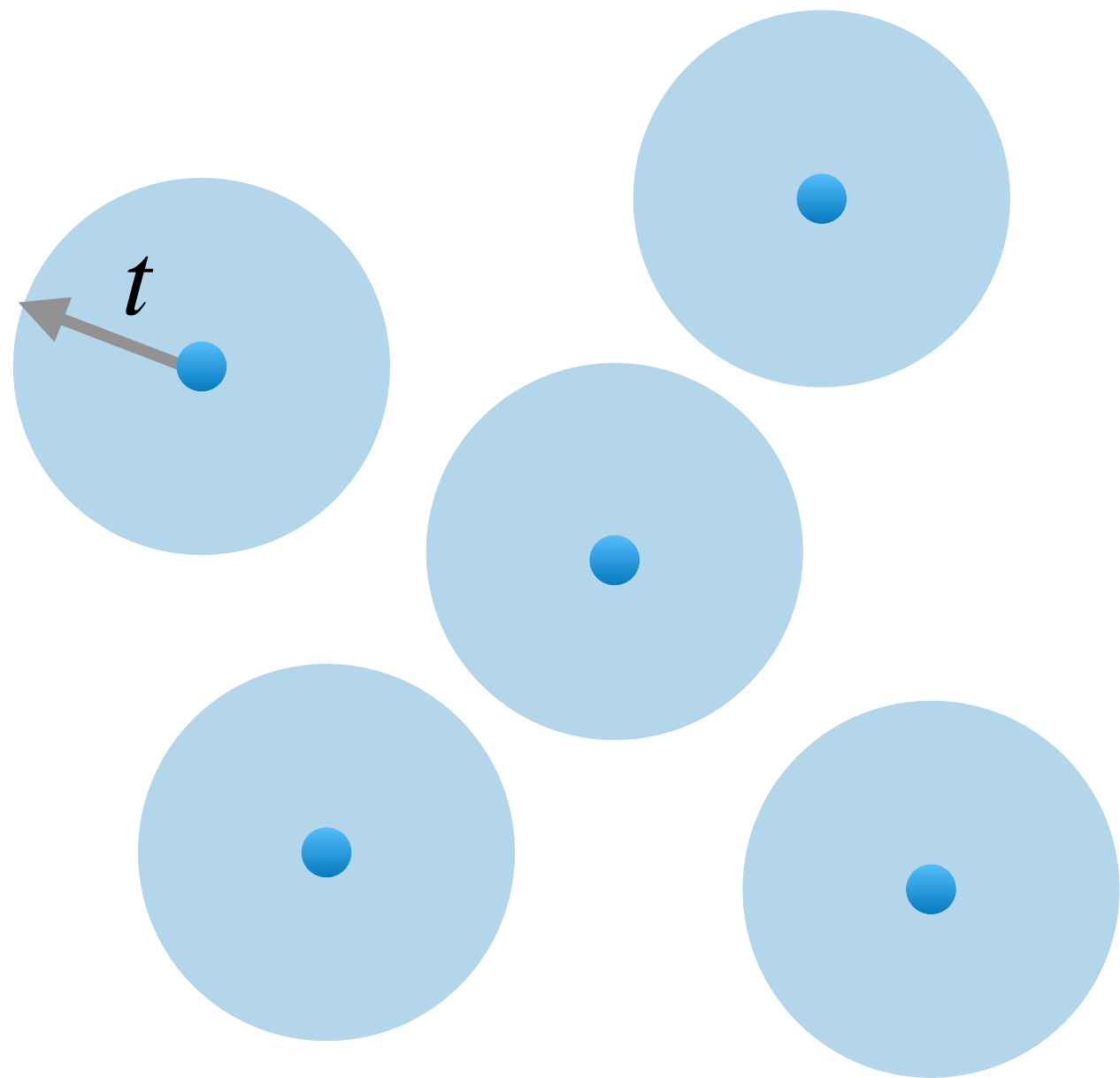
t -error-correcting code $C \subseteq \{0,1\}^n$



Hamming bound: $|C| \leq \frac{2^n}{|B_t(0)|}$

Can we do better than Hamming codes?

t -error-correcting code $C \subseteq \{0,1\}^n$

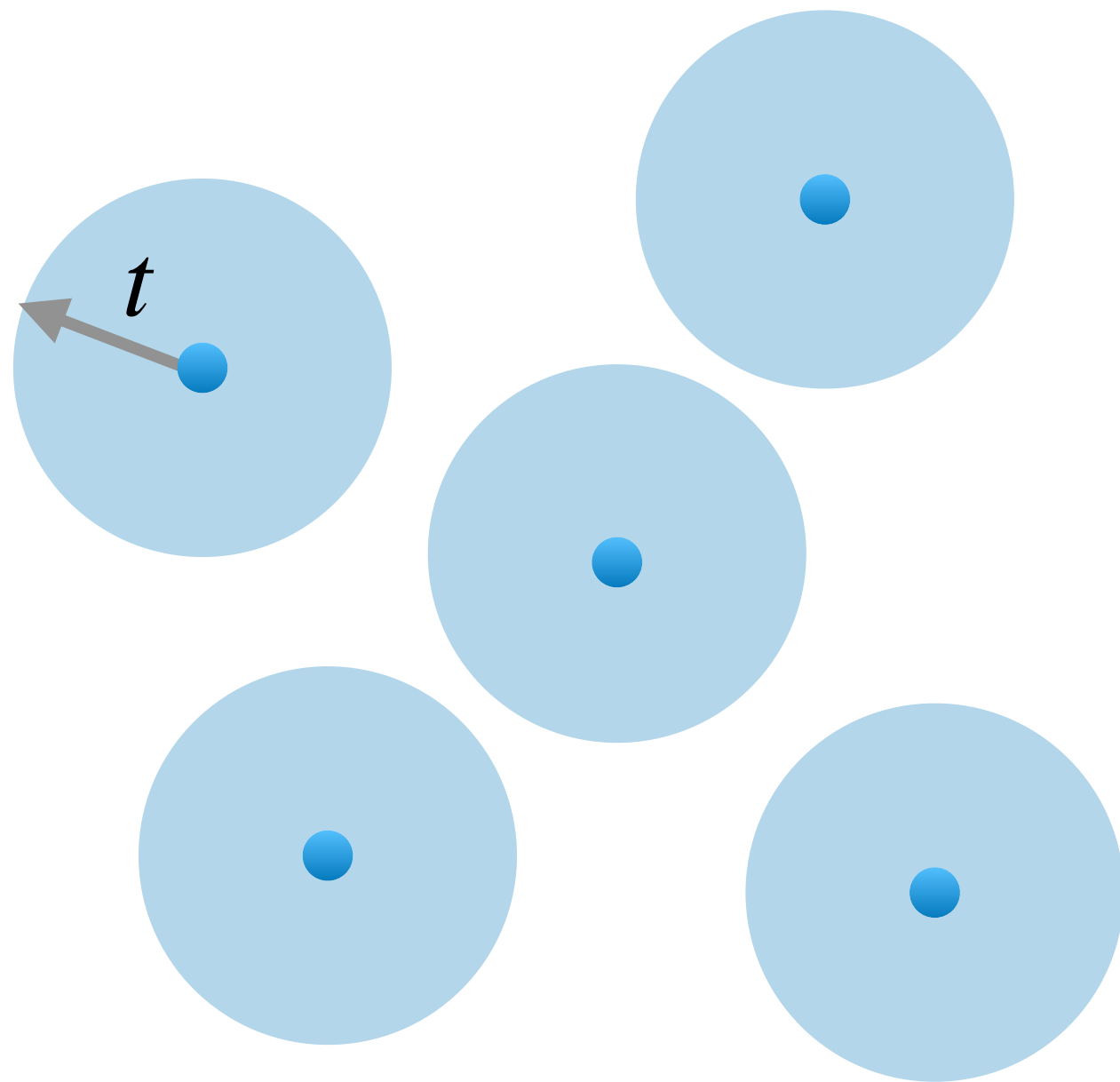


Hamming bound: $|C| \leq \frac{2^n}{|B_t(0)|}$

$$t = 1 \implies |C| \leq \frac{2^n}{n+1}$$

Can we do better than Hamming codes?

t -error-correcting code $C \subseteq \{0,1\}^n$



Hamming bound: $|C| \leq \frac{2^n}{|B_t(0)|}$

$$t = 1 \implies |C| \leq \frac{2^n}{n+1}$$

Hamming codes are optimal for
correcting 1 error! :)

What about correcting more errors?

What about correcting more errors?

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

What about correcting more errors?

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Focus on asymptotics for simplicity:

What about correcting more errors?

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Focus on asymptotics for simplicity:

- block length $n \rightarrow \infty$

What about correcting more errors?

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Focus on asymptotics for simplicity:

- block length $n \rightarrow \infty$
- Family of codes $(C_n)_{n \geq 1}$ with $C_n \subseteq \{0,1\}^n$ correcting ρn errors.

What about correcting more errors?

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Focus on asymptotics for simplicity:

- block length $n \rightarrow \infty$
- Family of codes $(C_n)_{n \geq 1}$ with $C_n \subseteq \{0,1\}^n$ correcting ρn errors.
- **Asymptotic rate:** $R = \lim_{n \rightarrow \infty} \frac{\log |C_n|}{n}$

What about correcting more errors?

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Focus on asymptotics for simplicity:

- block length $n \rightarrow \infty$
- Family of codes $(C_n)_{n \geq 1}$ with $C_n \subseteq \{0,1\}^n$ correcting ρn errors.
- **Asymptotic rate:** $R = \lim_{n \rightarrow \infty} \frac{\log |C_n|}{n}$

$$C_n \subseteq \{0,1\}^n \text{ has rate } R_n = \frac{\log |C_n|}{n} \iff |C_n| = 2^{R_n n}$$

What about correcting more errors?

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Focus on asymptotics for simplicity:

- block length $n \rightarrow \infty$
- Family of codes $(C_n)_{n \geq 1}$ with $C_n \subseteq \{0,1\}^n$ correcting ρn errors.
- **Asymptotic rate:** $R = \lim_{n \rightarrow \infty} \frac{\log |C_n|}{n}$

Fundamental question in coding theory:

How large can we make R as a function of ρ ?

Asymptotic Hamming bound

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Fundamental question in coding theory:

How large can we make R as a function of ρ ?

Asymptotic Hamming bound

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Fundamental question in coding theory:

How large can we make R as a function of ρ ?

Recall the Hamming bound: $|C| \leq 2^n / |B_t(0)|$

Asymptotic Hamming bound

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Fundamental question in coding theory:

How large can we make R as a function of ρ ?

Recall the Hamming bound: $|C| \leq 2^n / |B_t(0)|$

$|B_{\rho n}(0)| \approx 2^{nh(\rho)}$ for $\rho \leq 1/2$.

$h(x) = -x \log x - (1-x) \log(1-x)$

Asymptotic Hamming bound

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Fundamental question in coding theory:

How large can we make R as a function of ρ ?

Recall the Hamming bound: $|C| \leq 2^n / |B_t(0)|$

$|B_{\rho n}(0)| \approx 2^{nh(\rho)}$ for $\rho \leq 1/2$.

$h(x) = -x \log x - (1-x) \log(1-x)$

$|C| = 2^{Rn} \leq 2^n / 2^{h(\rho)n} \implies R \leq 1 - h(\rho)$ for $\rho \leq 1/2$.

Asymptotic Hamming bound

Regime of interest: Code $C \subseteq \{0,1\}^n$ correcting $t = \rho n$ errors, with block length n large.

Fundamental question in coding theory:

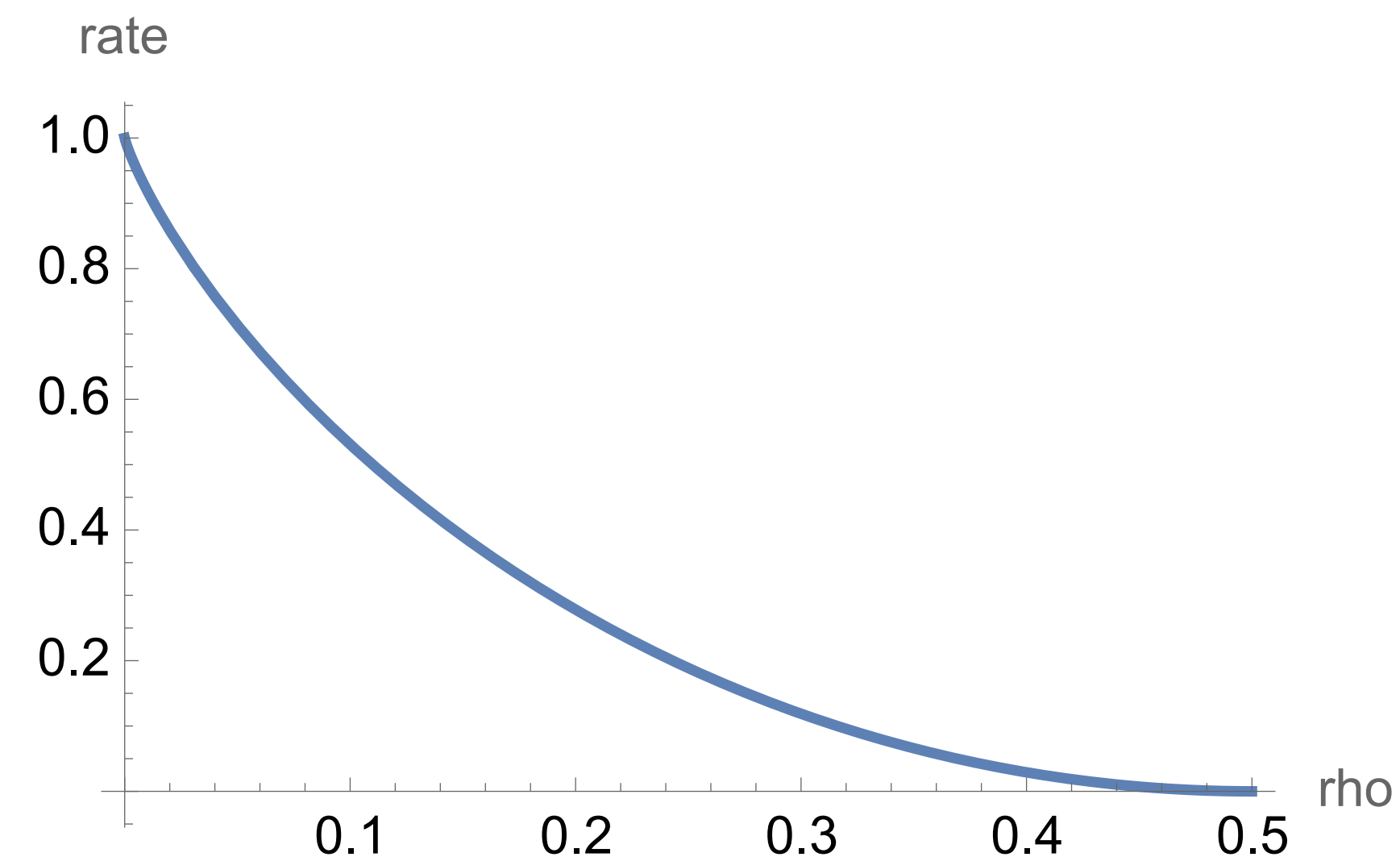
How large can we make R as a function of ρ ?

Recall the Hamming bound: $|C| \leq 2^n / |B_t(0)|$

$|B_{\rho n}(0)| \approx 2^{nh(\rho)}$ for $\rho \leq 1/2$.

$$h(x) = -x \log x - (1-x) \log(1-x)$$

$$|C| = 2^{Rn} \leq 2^n / 2^{h(\rho)n} \implies R \leq 1 - h(\rho) \text{ for } \rho \leq 1/2.$$



What *can* we do? The Gilbert-Varshamov bound

What *can* we do? The Gilbert-Varshamov bound

Goal:

Given a fraction of errors ρ , show existence of large code $C \subseteq \{0,1\}^n$ correcting ρn errors.

What *can* we do? The Gilbert-Varshamov bound

Goal:

Given a fraction of errors ρ , show existence of large code $C \subseteq \{0,1\}^n$ correcting ρn errors.

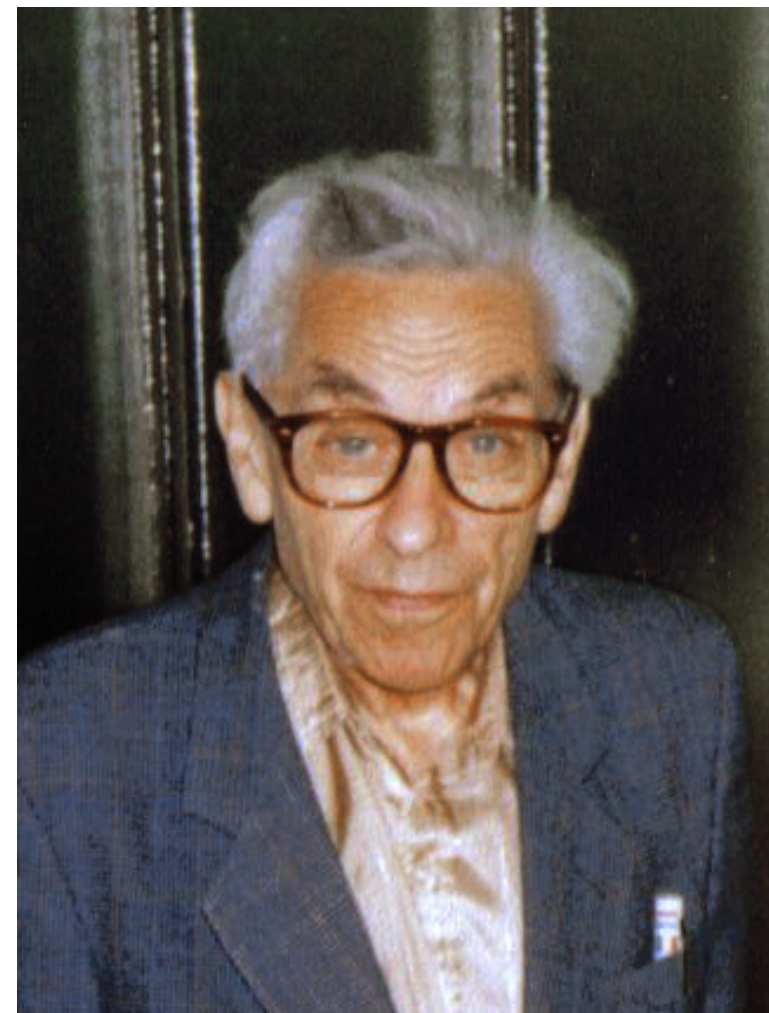
Linear codes are optimal for correcting 1 error. Can we use them to correct more errors?

What *can* we do? The Gilbert-Varshamov bound

Goal:

Given a fraction of errors ρ , show existence of large code $C \subseteq \{0,1\}^n$ correcting ρn errors.

Linear codes are optimal for correcting 1 error. Can we use them to correct more errors?



When stuck, try something random!

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

- Sample $n \times Rn$ matrix G with independent entries uniform over $\{0,1\}$.

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

- Sample $n \times Rn$ matrix G with independent entries uniform over $\{0,1\}$.
- Our code is $C = \{Gx : x \in \{0,1\}^{Rn}\}$, and $|C| = 2^{Rn}$ with high probability.

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

- Sample $n \times Rn$ matrix G with independent entries uniform over $\{0,1\}$.
- Our code is $C = \{Gx : x \in \{0,1\}^{Rn}\}$, and $|C| = 2^{Rn}$ with high probability.
- For each nonzero $x \in \{0,1\}^k$ the random variable Gx is uniform over $\{0,1\}^n$.

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

- Sample $n \times Rn$ matrix G with independent entries uniform over $\{0,1\}$.
- Our code is $C = \{Gx : x \in \{0,1\}^{Rn}\}$, and $|C| = 2^{Rn}$ with high probability.
- For each nonzero $x \in \{0,1\}^k$ the random variable Gx is uniform over $\{0,1\}^n$.
- $\Pr[Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq \frac{|B_{2\rho n}(0)|}{2^n} \approx 2^{-(1-h(2\rho))n}$

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

- Sample $n \times Rn$ matrix G with independent entries uniform over $\{0,1\}$.
- Our code is $C = \{Gx : x \in \{0,1\}^{Rn}\}$, and $|C| = 2^{Rn}$ with high probability.
- For each nonzero $x \in \{0,1\}^k$ the random variable Gx is uniform over $\{0,1\}^n$.
- $\Pr[Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq \frac{|B_{2\rho n}(0)|}{2^n} \approx 2^{-(1-h(2\rho))n}$
- Since $\Pr[A_1 \vee A_2 \vee \dots \vee A_k] \leq \sum_{i=1}^k \Pr[A_i]$, we get

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

- Sample $n \times Rn$ matrix G with independent entries uniform over $\{0,1\}$.
- Our code is $C = \{Gx : x \in \{0,1\}^{Rn}\}$, and $|C| = 2^{Rn}$ with high probability.
- For each nonzero $x \in \{0,1\}^k$ the random variable Gx is uniform over $\{0,1\}^n$.

- $\Pr[Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq \frac{|B_{2\rho n}(0)|}{2^n} \approx 2^{-(1-h(2\rho))n}$

- Since $\Pr[A_1 \vee A_2 \vee \dots \vee A_k] \leq \sum_{i=1}^k \Pr[A_i]$, we get

$$\Pr[\exists x \neq 0 : Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq |C| \cdot 2^{-(1-h(2\rho))n}$$

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

- Sample $n \times Rn$ matrix G with independent entries uniform over $\{0,1\}$.
- Our code is $C = \{Gx : x \in \{0,1\}^{Rn}\}$, and $|C| = 2^{Rn}$ with high probability.
- For each nonzero $x \in \{0,1\}^k$ the random variable Gx is uniform over $\{0,1\}^n$.

- $\Pr[Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq \frac{|B_{2\rho n}(0)|}{2^n} \approx 2^{-(1-h(2\rho))n}$

- Since $\Pr[A_1 \vee A_2 \vee \dots \vee A_k] \leq \sum_{i=1}^k \Pr[A_i]$, we get

$$\Pr[\exists x \neq 0 : Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq |C| \cdot 2^{-(1-h(2\rho))n}$$

This is very small if rate $R < 1 - h(2\rho)$

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

- Sample $n \times Rn$ matrix G with independent entries uniform over $\{0,1\}$.
- Our code is $C = \{Gx : x \in \{0,1\}^{Rn}\}$, and $|C| = 2^{Rn}$ with high probability.
- For each nonzero $x \in \{0,1\}^k$ the random variable Gx is uniform over $\{0,1\}^n$.

- $\Pr[Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq \frac{|B_{2\rho n}(0)|}{2^n} \approx 2^{-(1-h(2\rho))n}$

- Since $\Pr[A_1 \vee A_2 \vee \dots \vee A_k] \leq \sum_{i=1}^k \Pr[A_i]$, we get

$$\Pr[\exists x \neq 0 : Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq |C| \cdot 2^{-(1-h(2\rho))n}$$

This is very small if rate $R < 1 - h(2\rho)$

What *can* we do? The Gilbert-Varshamov bound

We analyze the behavior of a **random** linear code!

- Sample $n \times Rn$ matrix G with independent entries uniform over $\{0,1\}$.
- Our code is $C = \{Gx : x \in \{0,1\}^{Rn}\}$, and $|C| = 2^{Rn}$ with high probability.
- For each nonzero $x \in \{0,1\}^k$ the random variable Gx is uniform over $\{0,1\}^n$.

- $\Pr[Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq \frac{|B_{2\rho n}(0)|}{2^n} \approx 2^{-(1-h(2\rho))n}$

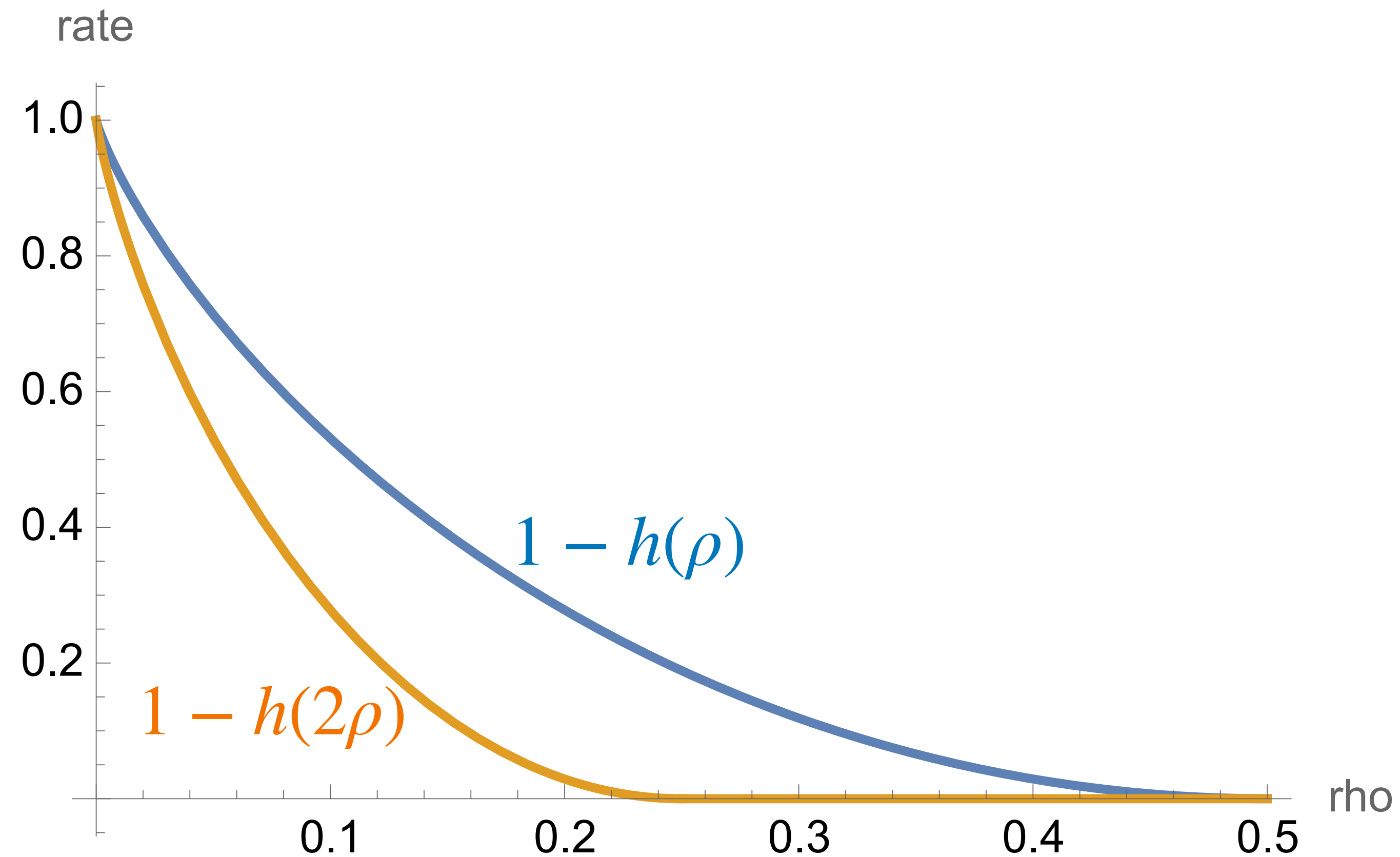
- Since $\Pr[A_1 \vee A_2 \vee \dots \vee A_k] \leq \sum_{i=1}^k \Pr[A_i]$, we get

$$\Pr[\exists x \neq 0 : Gx \text{ has fewer than } 2\rho n \text{ nonzero coordinates}] \leq |C| \cdot 2^{-(1-h(2\rho))n}$$

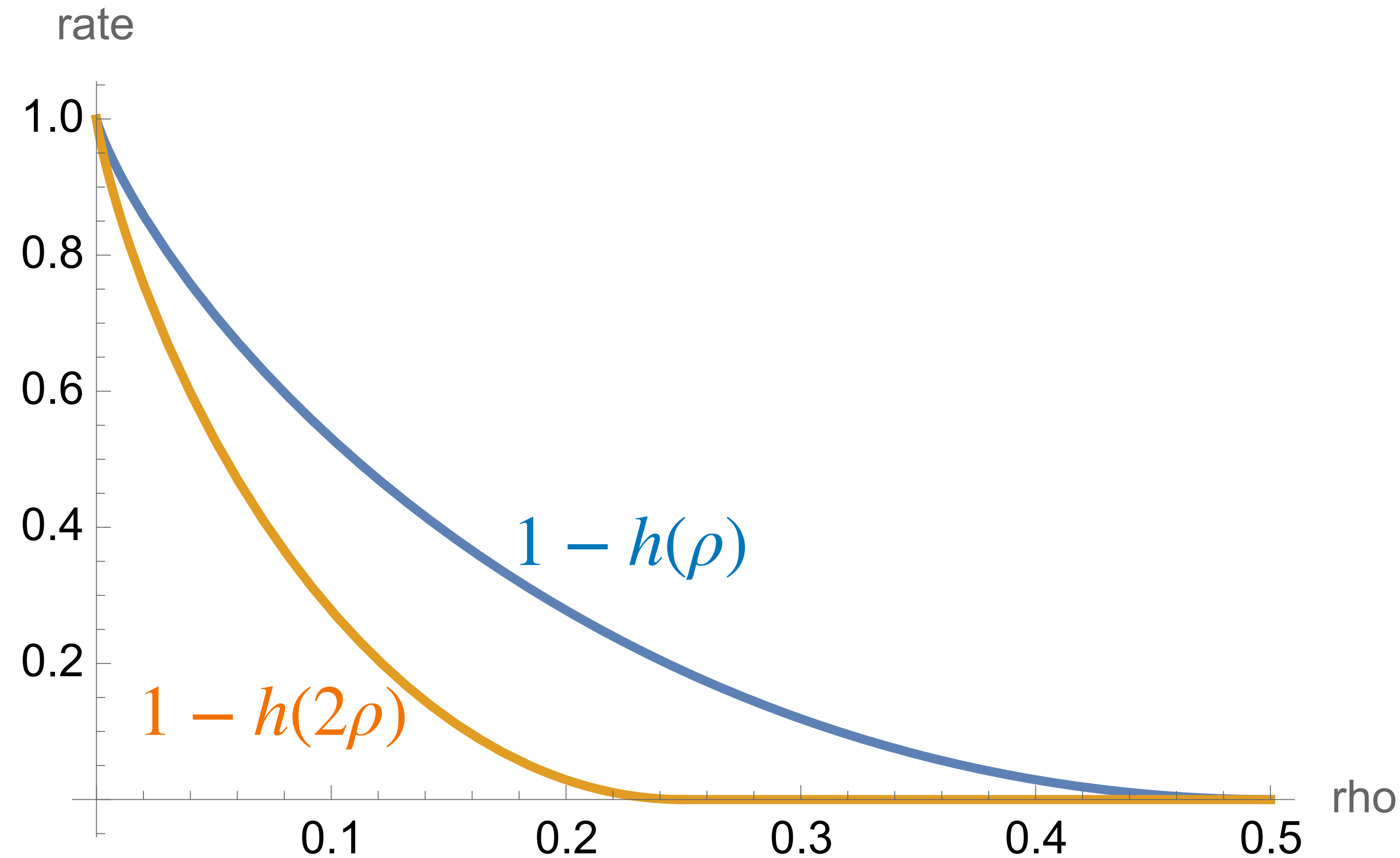
This is very small if rate $R < 1 - h(2\rho)$

- So there exist (many) linear codes of asymptotic rate $1 - h(2\rho)$ correcting ρn errors!

Comparing the Hamming and Gilbert-Varshamov bounds

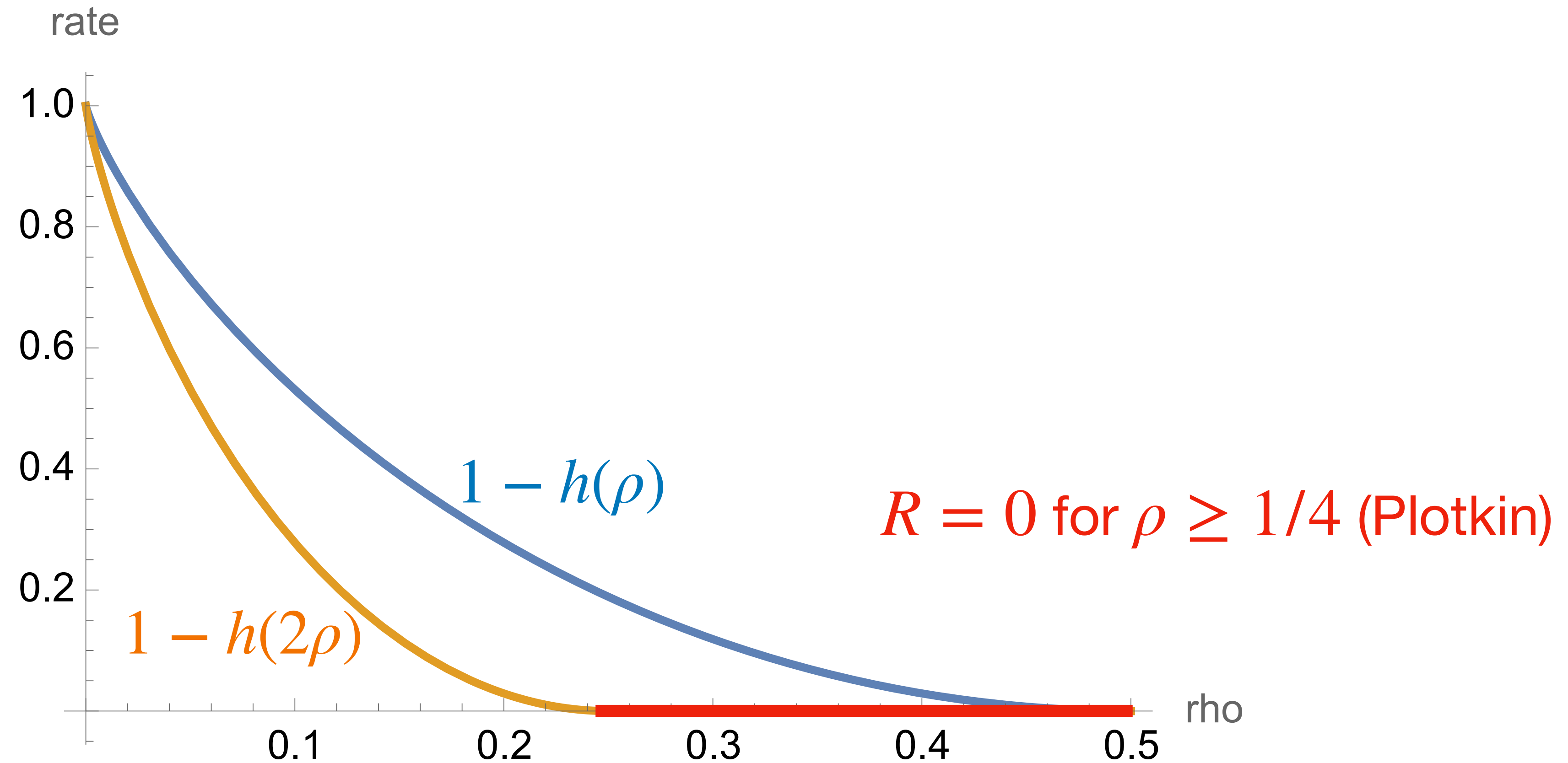


Comparing the Hamming and Gilbert-Varshamov bounds

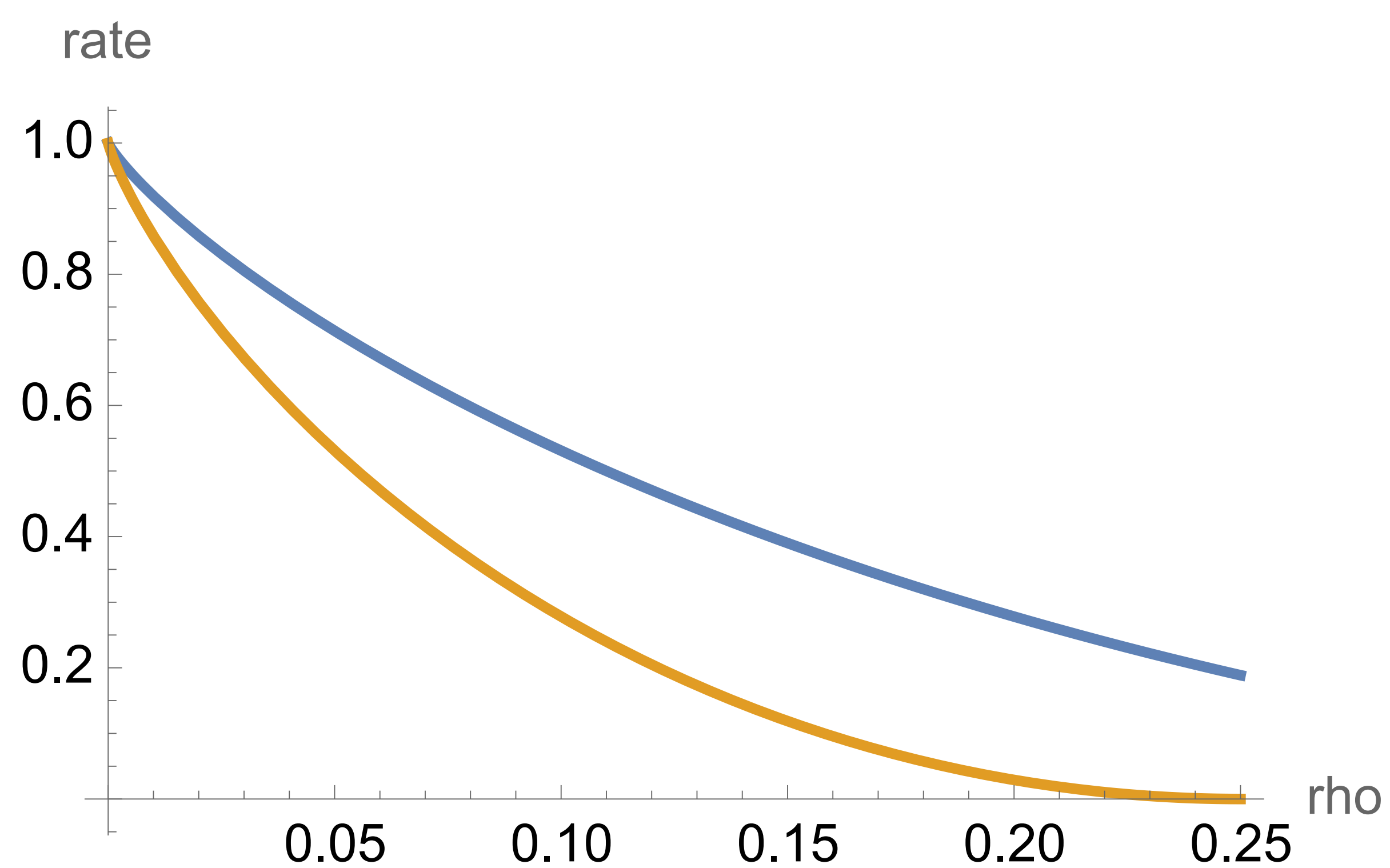


Can we do better?

Comparing the Hamming and Gilbert-Varshamov bounds



The story so far



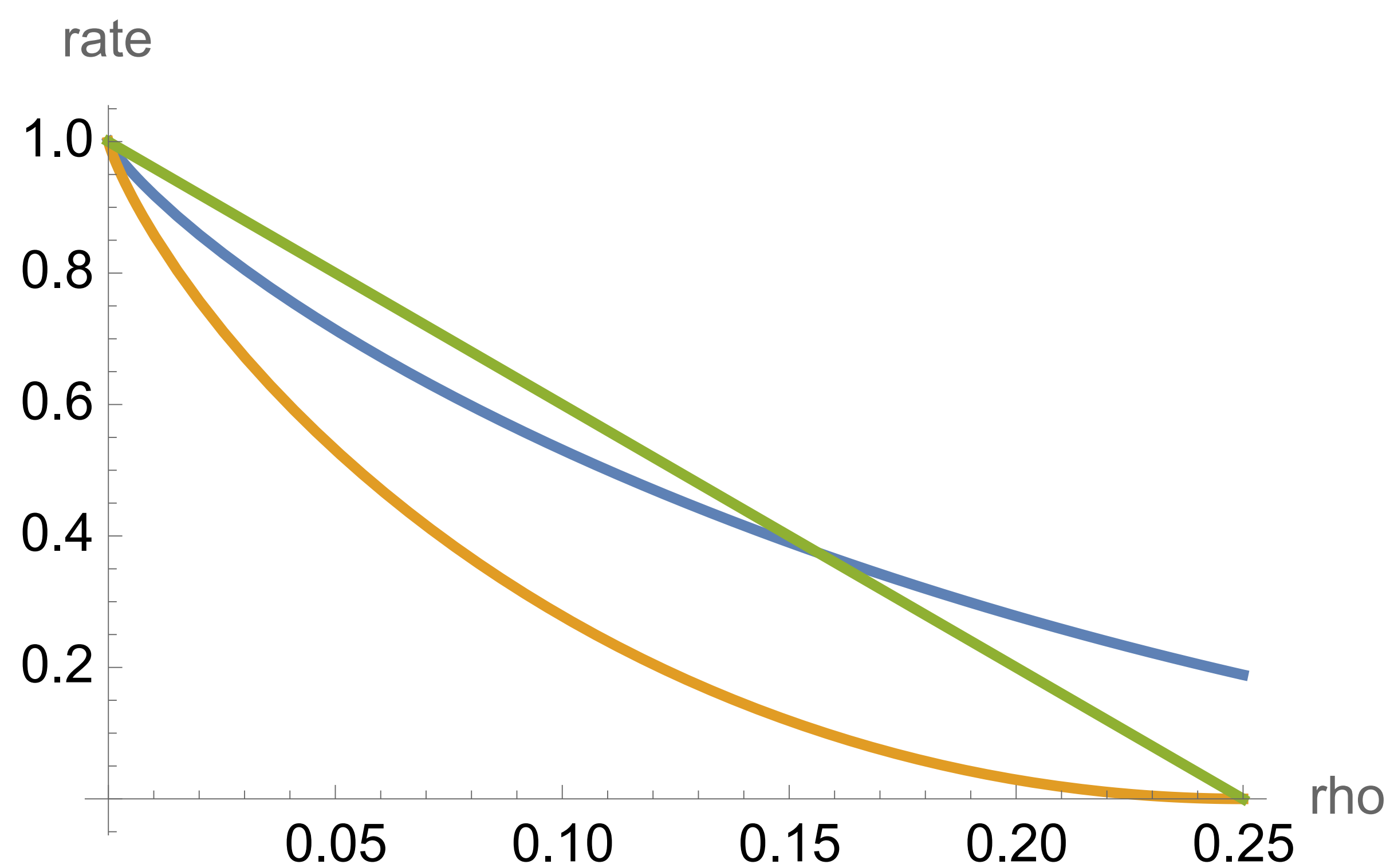
Lower bounds

- GV (1950s)

Upper bounds

- Hamming 1950s)

The story so far



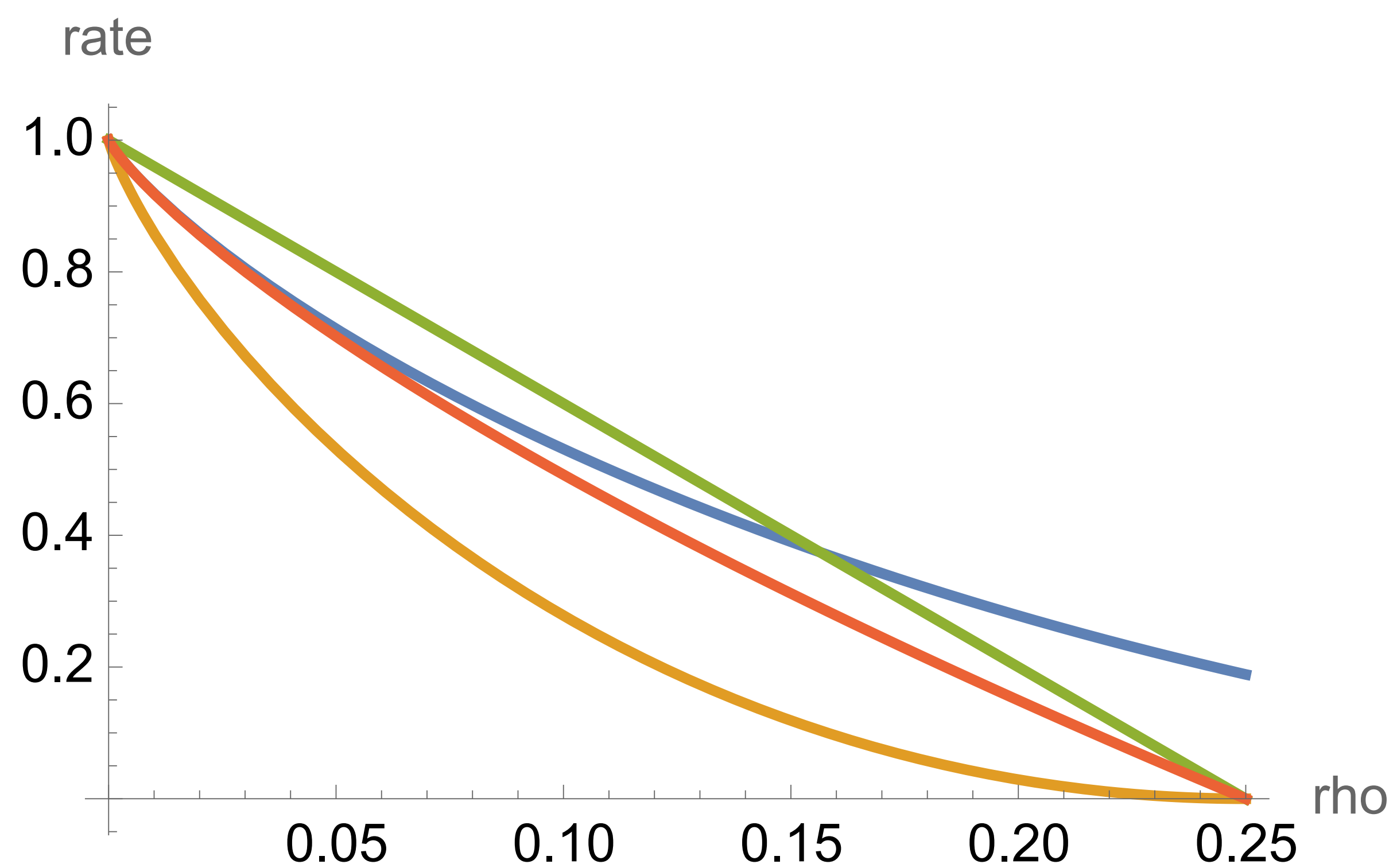
Lower bounds

- GV (1950s)

Upper bounds

- Hamming (1950s)
- Plotkin (1950s)

The story so far



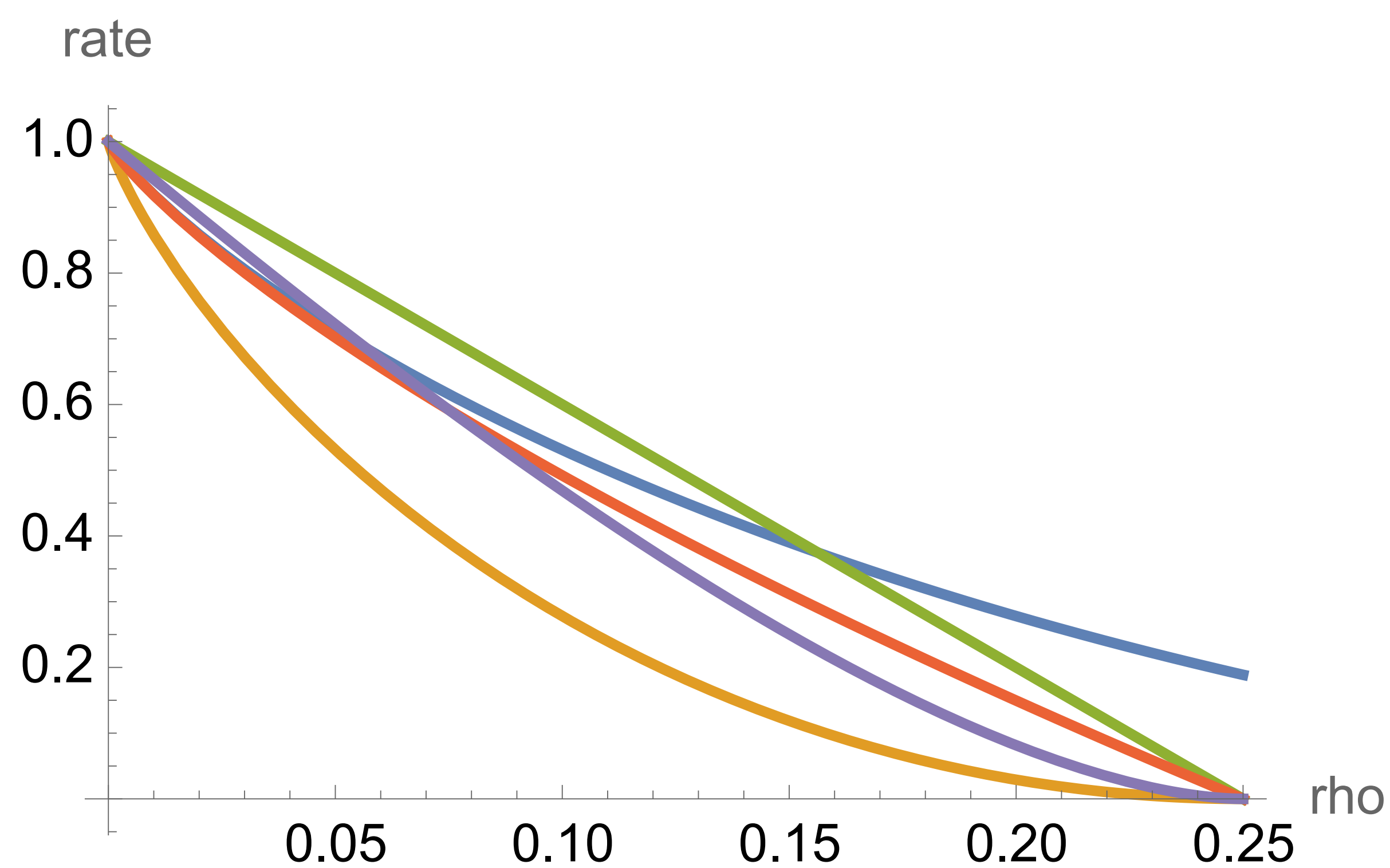
Lower bounds

- GV (1950s)

Upper bounds

- Hamming (1950s)
- Plotkin (1950s)
- Elias-Bassalygo (1950s)

The story so far



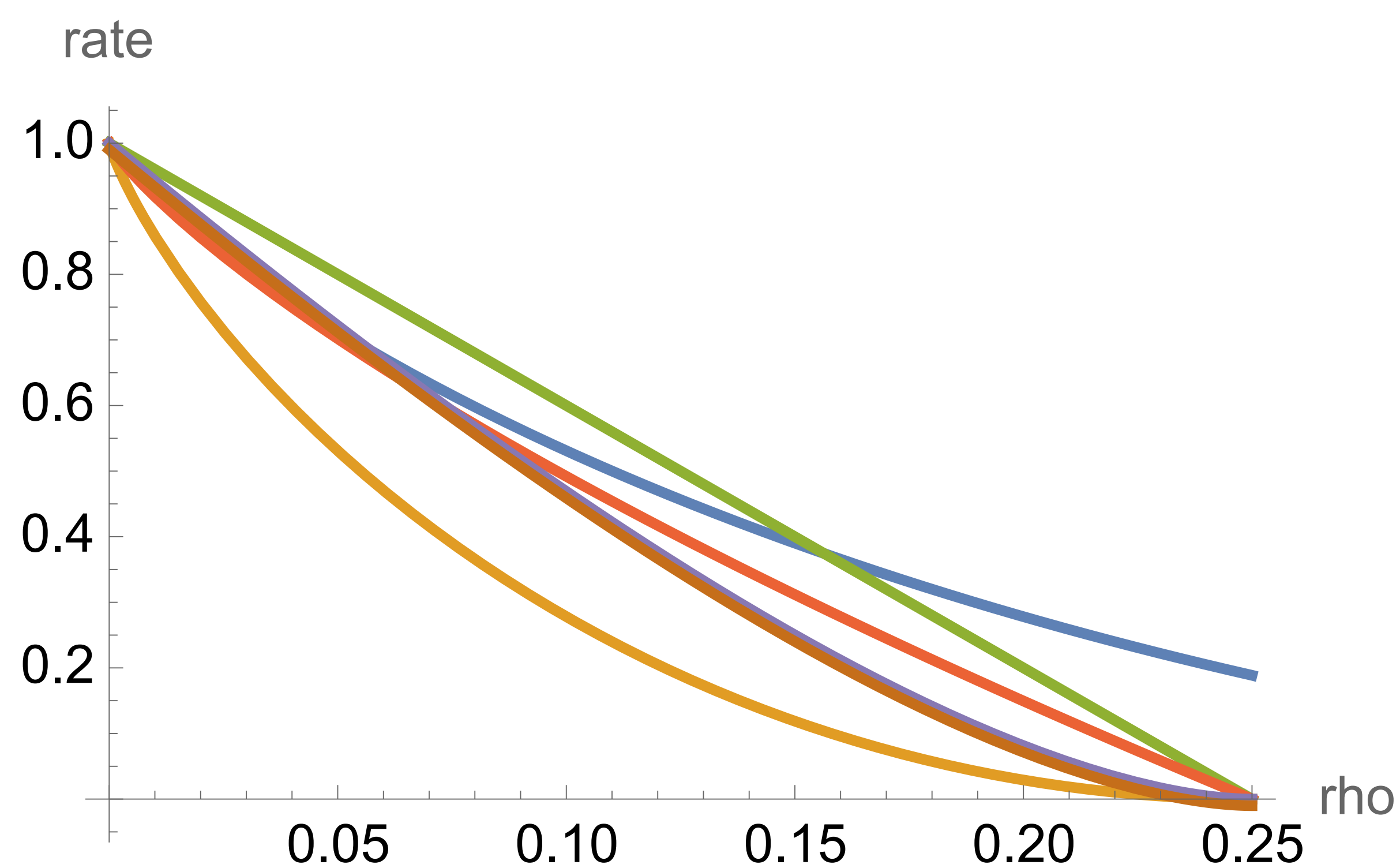
Lower bounds

- GV (1950s)

Upper bounds

- Hamming (1950s)
- Plotkin (1950s)
- Elias-Bassalygo (1950s)
- MRRW (1970s)

The story so far



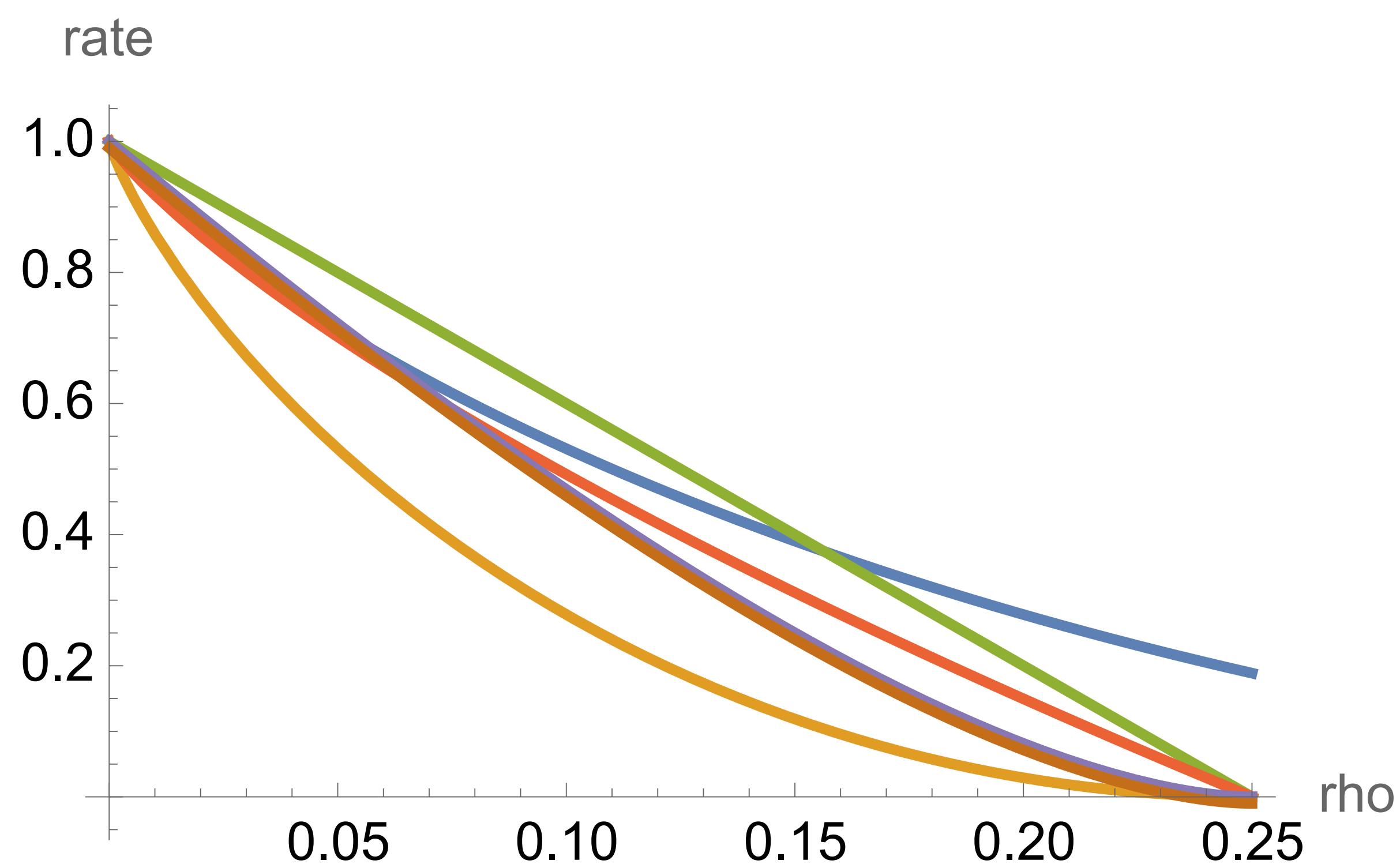
Lower bounds

- GV (1950s)

Upper bounds

- Hamming (1950s)
- Plotkin (1950s)
- Elias-Bassalygo (1950s)
- MRRW (1970s)
- Alrabiah-Guruswami/Open AI (2026)

The story so far



Lower bounds

- GV (1950s)

Upper bounds

- Hamming (1950s)
- Plotkin (1950s)
- Elias-Bassalygo (1950s)
- MRRW (1970s)
- Alrabiah-Guruswami/Open AI (2026)

Major open problem in coding theory: Improve on the GV bound for binary codes

What about random errors?

What about random errors?

- We don't have a clear picture of the optimal rate we can achieve while correcting a ρ -fraction of adversarial errors.

What about random errors?

- We don't have a clear picture of the optimal rate we can achieve while correcting a ρ -fraction of adversarial errors.
- It turns out that the picture becomes much nicer when we move to random errors!

What about random errors?

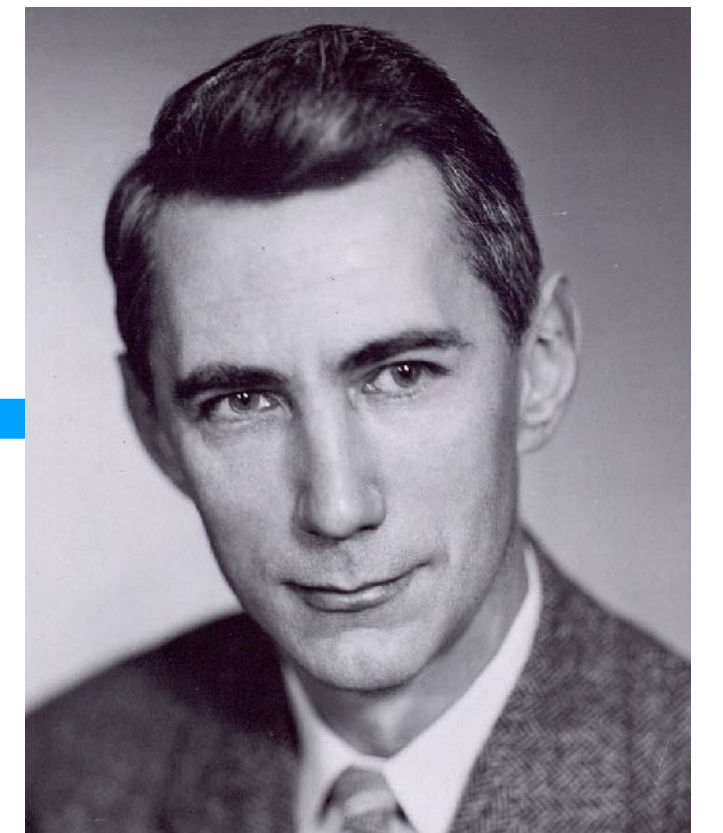
- We don't have a clear picture of the optimal rate we can achieve while correcting a ρ -fraction of adversarial errors.
- It turns out that the picture becomes much nicer when we move to random errors!
- This is the crown jewel of **information theory**.

What about random errors?

- We don't have a clear picture of the optimal rate we can achieve while correcting a ρ -fraction of adversarial errors.
- It turns out that the picture becomes much nicer when we move to random errors!
- This is the crown jewel of **information theory**.

The probabilistic perspective (Shannon 1948)

- Noisy channel introduces random errors
 - Each symbol independently replaced by a different one with probability p .



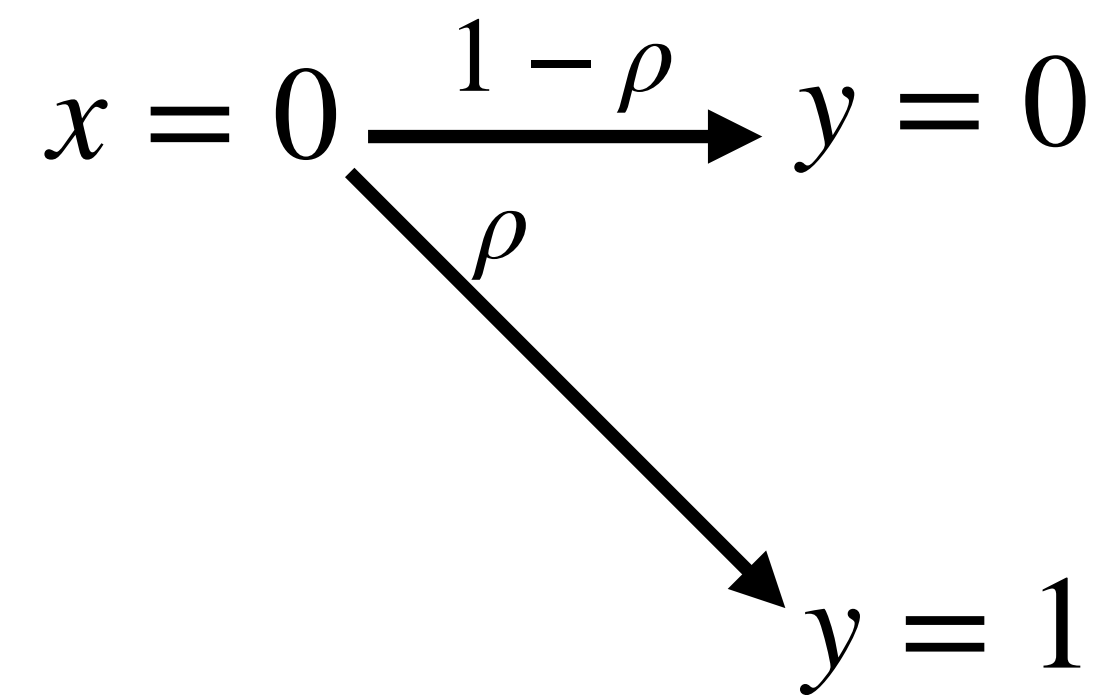
The binary symmetric channel

The binary symmetric channel

Given an input $c \in \{0,1\}^n$, the binary symmetric channel independently flips each coordinate of c with probability ρ .

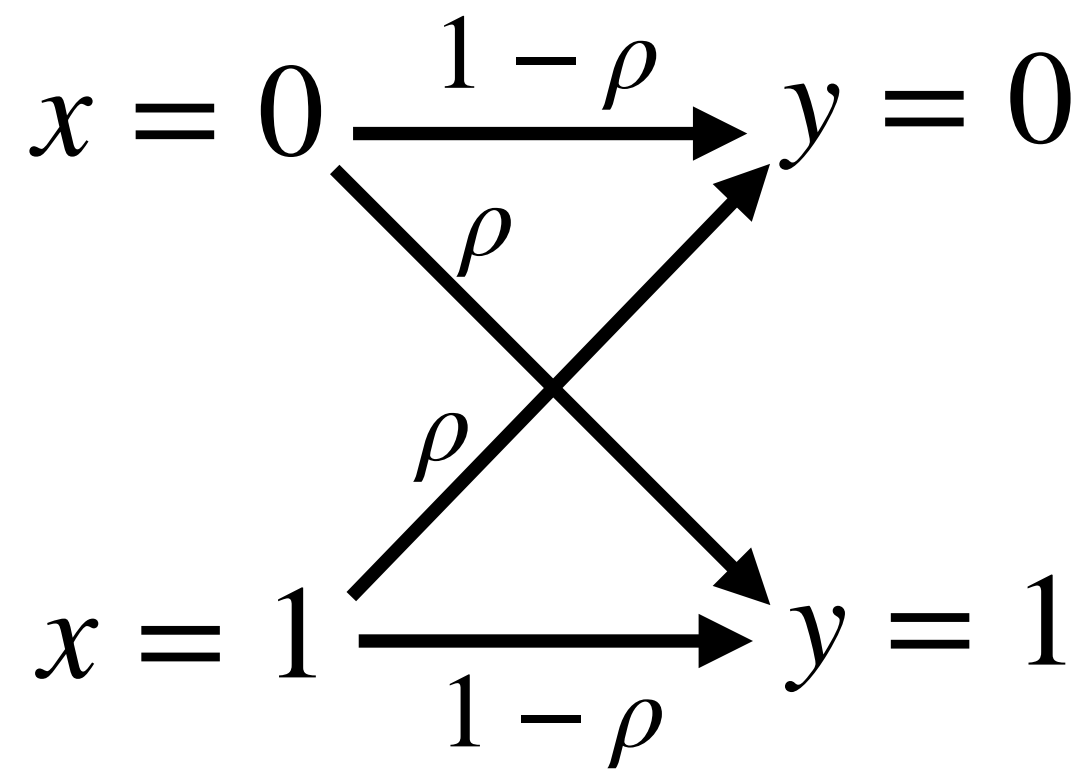
The binary symmetric channel

Given an input $c \in \{0,1\}^n$, the binary symmetric channel independently flips each coordinate of c with probability ρ .



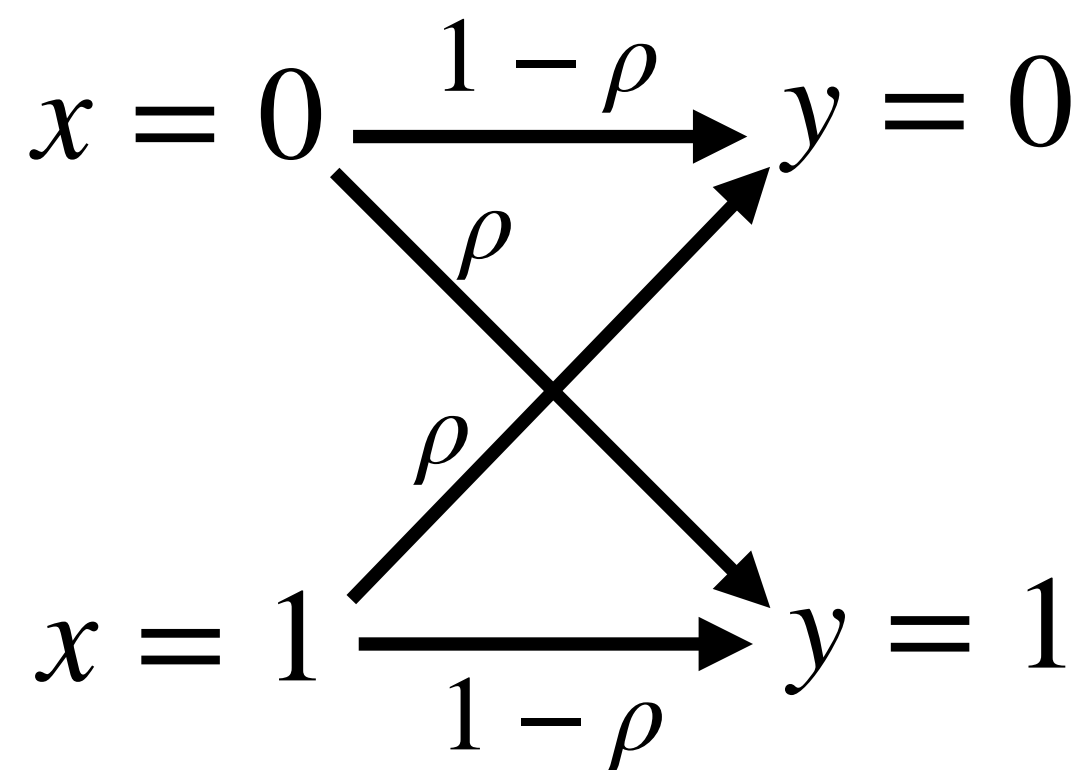
The binary symmetric channel

Given an input $c \in \{0,1\}^n$, the binary symmetric channel independently flips each coordinate of c with probability ρ .



The binary symmetric channel

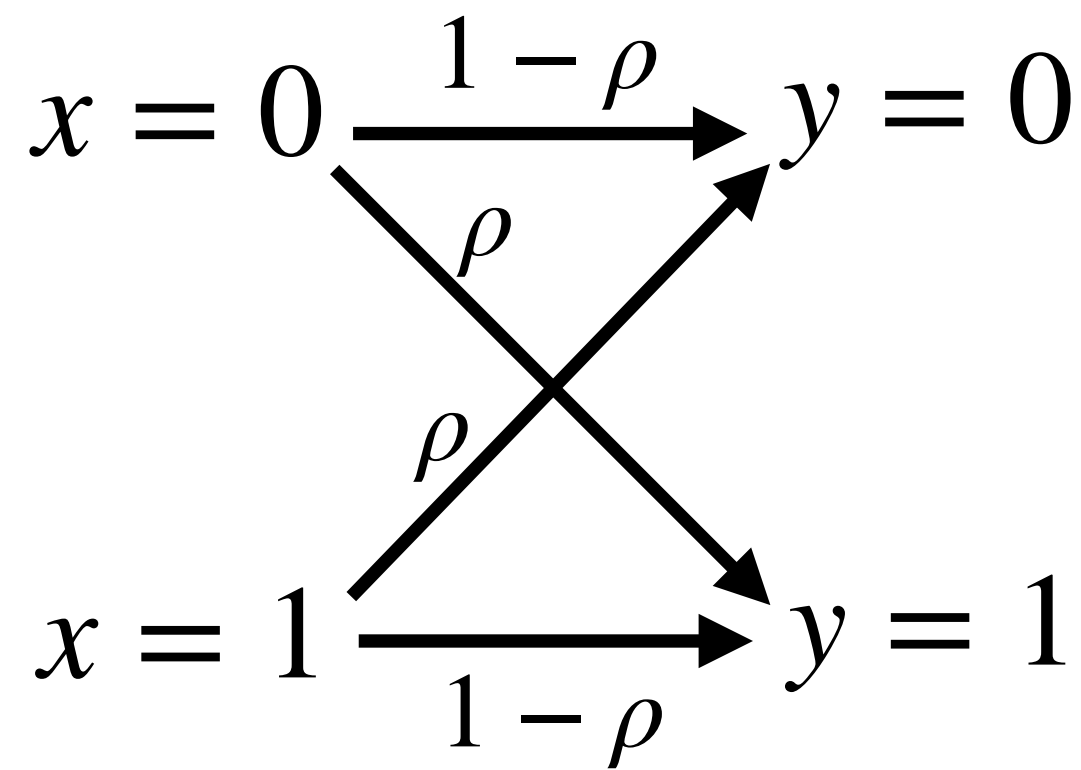
Given an input $c \in \{0,1\}^n$, the binary symmetric channel independently flips each coordinate of c with probability ρ .



- **Hamming's model:** introduces ρn errors in arbitrary coordinates of its choice.

The binary symmetric channel

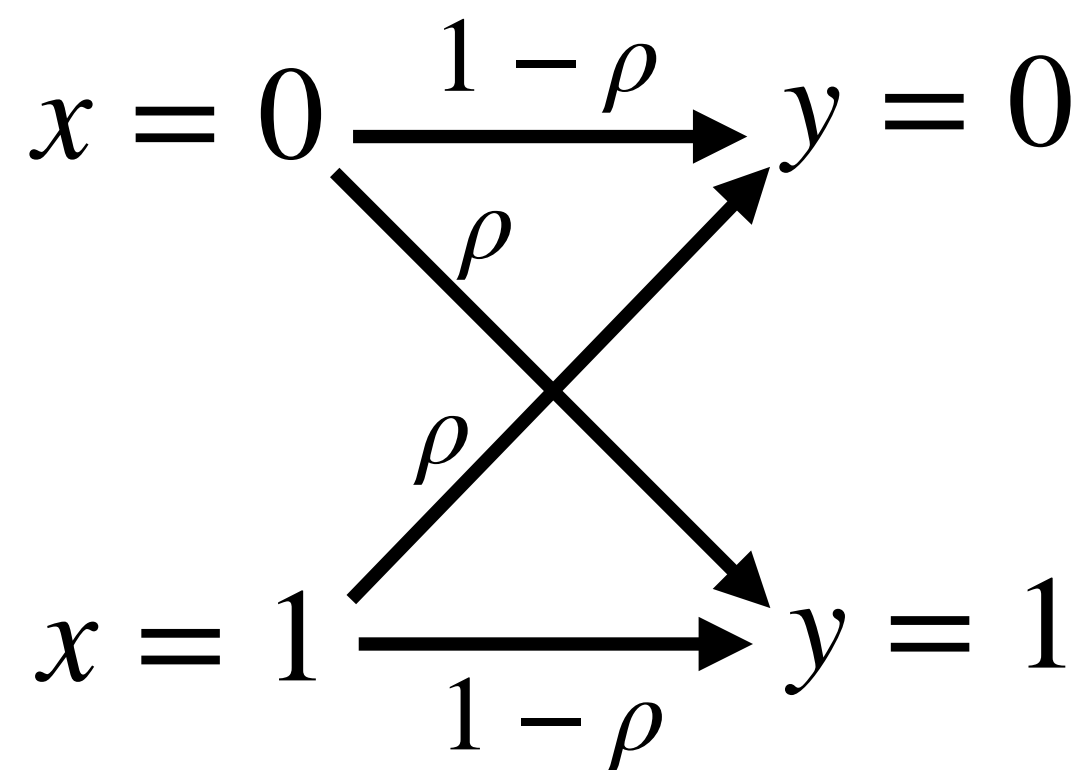
Given an input $c \in \{0,1\}^n$, the binary symmetric channel independently flips each coordinate of c with probability ρ .



- **Hamming's model:** introduces ρn errors in arbitrary coordinates of its choice.
- **Binary symmetric channel:** introduces ρn errors in random coordinates.

The binary symmetric channel

Given an input $c \in \{0,1\}^n$, the binary symmetric channel independently flips each coordinate of c with probability ρ .



- **Hamming's model:** introduces ρn errors in arbitrary coordinates of its choice.
- **Binary symmetric channel:** introduces ρn errors in random coordinates.

The basic question: What's the optimal rate of **reliable communication** over the binary symmetric channel with error probability ρ ?

A special case of Shannon's noisy channel coding theorem

A special case of Shannon's noisy channel coding theorem

We say that a rate R is *achievable* if there are:

- A family of codes $(C_n)_{n \in \mathbb{N}}$ with $C_n \subseteq \{0,1\}^n$ of size at least $2^{R_n n}$ with $R_n \rightarrow R$;
- A decoder Dec such that $\Pr[\text{Dec}(\tilde{c}) \neq c] \leq \varepsilon_n$ for all $c \in C_n$ with $\varepsilon_n \rightarrow 0$;

The **coding capacity** $C(\rho)$ is the supremum of all achievable rates.

A special case of Shannon's noisy channel coding theorem

We say that a rate R is *achievable* if there are:

- A family of codes $(C_n)_{n \in \mathbb{N}}$ with $C_n \subseteq \{0,1\}^n$ of size at least $2^{R_n n}$ with $R_n \rightarrow R$;
- A decoder Dec such that $\Pr[\text{Dec}(\tilde{c}) \neq c] \leq \varepsilon_n$ for all $c \in C_n$ with $\varepsilon_n \rightarrow 0$;

The **coding capacity** $C(\rho)$ is the supremum of all achievable rates.

Shannon's noisy channel coding theorem (for the BSC):

We have $C(\rho) = 1 - h(\rho)$ for all $\rho \in [0,1]$.

A special case of Shannon's noisy channel coding theorem

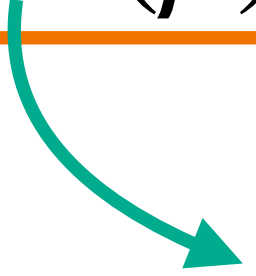
We say that a rate R is *achievable* if there are:

- A family of codes $(C_n)_{n \in \mathbb{N}}$ with $C_n \subseteq \{0,1\}^n$ of size at least $2^{R_n n}$ with $R_n \rightarrow R$;
- A decoder Dec such that $\Pr[\text{Dec}(\tilde{c}) \neq c] \leq \varepsilon_n$ for all $c \in C_n$ with $\varepsilon_n \rightarrow 0$;

The **coding capacity** $C(\rho)$ is the supremum of all achievable rates.

Shannon's noisy channel coding theorem (for the BSC):

We have $C(\rho) = 1 - h(\rho)$ for all $\rho \in [0,1]$.



matches the asymptotic Hamming bound!

A special case of Shannon's noisy channel coding theorem

Shannon's noisy channel coding theorem (for the BSC):

We have $C(\rho) = 1 - h(\rho)$ for all $\rho \in [0,1]$.

A special case of Shannon's noisy channel coding theorem

Shannon's noisy channel coding theorem (for the BSC):

We have $C(\rho) = 1 - h(\rho)$ for all $\rho \in [0,1]$.

Shannon proved this result by analyzing a **uniformly random code**.

A special case of Shannon's noisy channel coding theorem

Shannon's noisy channel coding theorem (for the BSC):

We have $C(\rho) = 1 - h(\rho)$ for all $\rho \in [0,1]$.

Shannon proved this result by analyzing a **uniformly random code**.

Uniformly random code $C \subseteq \{0,1\}^n$ of rate R :

Independently sample 2^{Rn} times uniformly at random from $\{0,1\}^n$.

A special case of Shannon's noisy channel coding theorem

Shannon's noisy channel coding theorem (for the BSC):

We have $C(\rho) = 1 - h(\rho)$ for all $\rho \in [0,1]$.

Shannon proved this result by analyzing a **uniformly random code**.

Uniformly random code $C \subseteq \{0,1\}^n$ of rate R :

Independently sample 2^{Rn} times uniformly at random from $\{0,1\}^n$.

Elias later showed that **random linear codes** also achieve capacity over the BSC.

A special case of Shannon's noisy channel coding theorem

Shannon's noisy channel coding theorem (for the BSC):

We have $C(\rho) = 1 - h(\rho)$ for all $\rho \in [0,1]$.

Shannon proved this result by analyzing a **uniformly random code**.

Uniformly random code $C \subseteq \{0,1\}^n$ of rate R :

Independently sample 2^{Rn} times uniformly at random from $\{0,1\}^n$.

Elias later showed that **random linear codes** also achieve capacity over the BSC.

A concrete realization of a fascinating high-level question:

When does a random **linear** code behave like a uniformly random code?

A special case of Shannon's noisy channel coding theorem

Shannon's noisy channel coding theorem (for the BSC):

We have $C(\rho) = 1 - h(\rho)$ for all $\rho \in [0,1]$.

Shannon proved this result by analyzing a **uniformly random code**.

Uniformly random code $C \subseteq \{0,1\}^n$ of rate R :

Independently sample 2^{Rn} times uniformly at random from $\{0,1\}^n$.

Elias later showed that **random linear codes** also achieve capacity over the BSC.

A concrete realization of a fascinating high-level question:

When does a random subspace of $\mathbb{F}_2^n$ behave like a random subset?

Some thoughts from Richard Hamming

Richard Hamming: You and Your Research

Some thoughts from Richard Hamming

Richard Hamming: You and Your Research

“Courage is one of the things that Shannon had supremely. You have only to think of his major theorem. He wants to create a method of coding, but he doesn't know what to do so he makes a random code.

Some thoughts from Richard Hamming

Richard Hamming: You and Your Research

“Courage is one of the things that Shannon had supremely. You have only to think of his major theorem. He wants to create a method of coding, but he doesn't know what to do so he makes a random code.

Then he is stuck. And then he asks the impossible question, ‘*What would the average random code do?*’ He then proves that the average code is arbitrarily good, and that therefore there must be at least one good code.

Some thoughts from Richard Hamming

Richard Hamming: You and Your Research

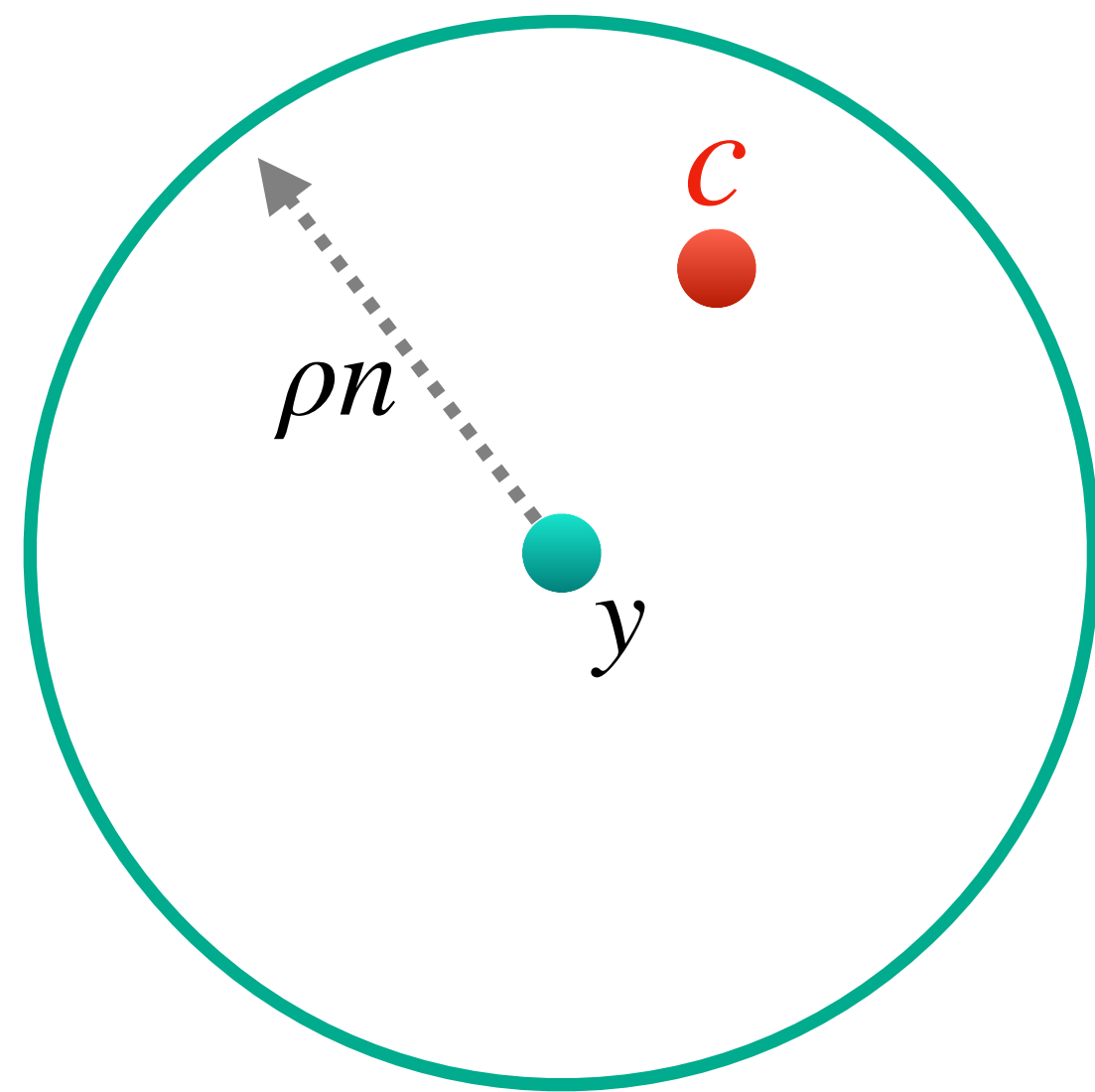
“Courage is one of the things that Shannon had supremely. You have only to think of his major theorem. He wants to create a method of coding, but he doesn't know what to do so he makes a random code.

Then he is stuck. And then he asks the impossible question, ‘*What would the average random code do?*’ He then proves that the average code is arbitrarily good, and that therefore there must be at least one good code.

Who but a man of infinite courage could have dared to think those thoughts?...”

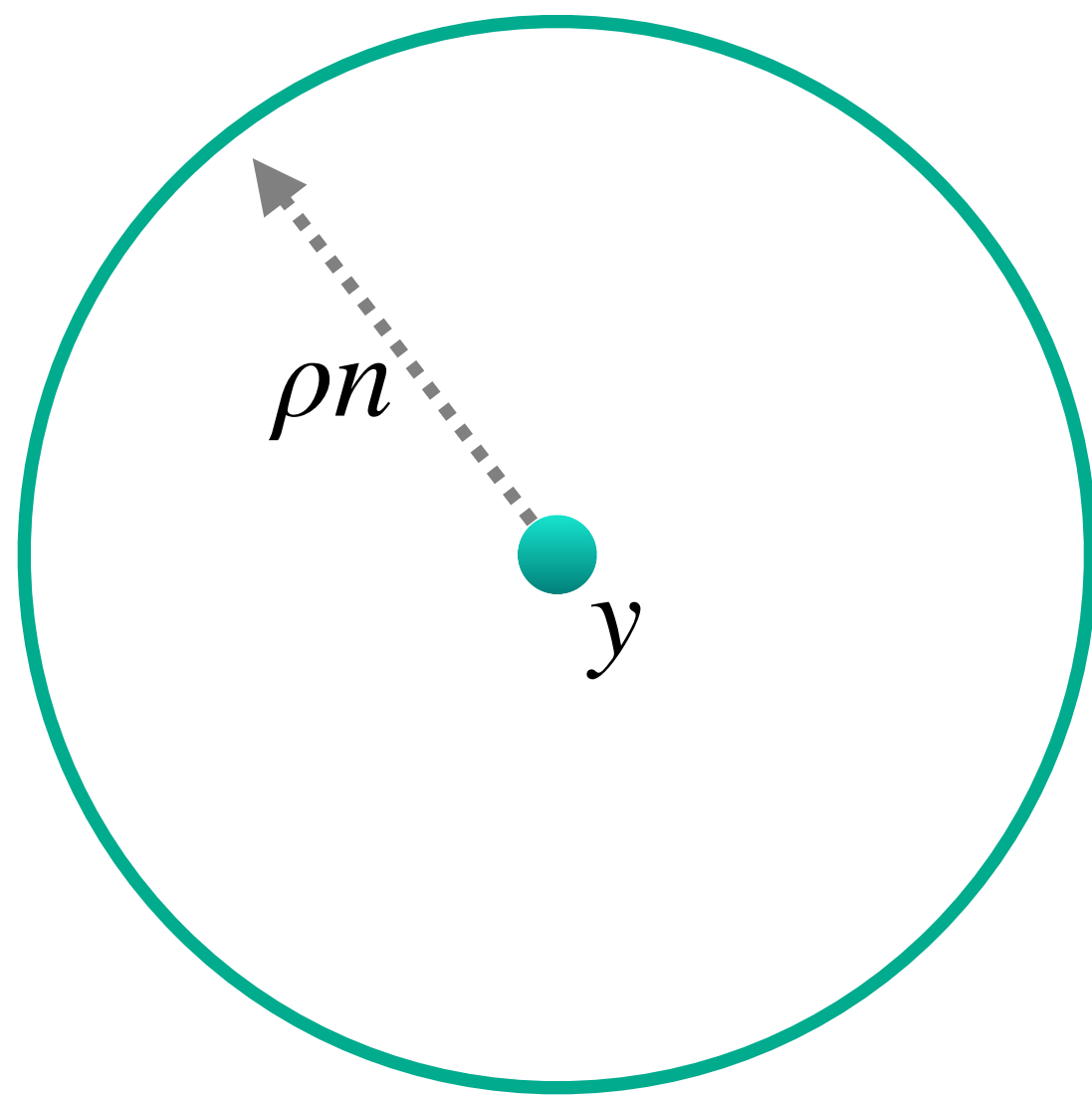
List-decoding: Bridging Hamming and Shannon

Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.



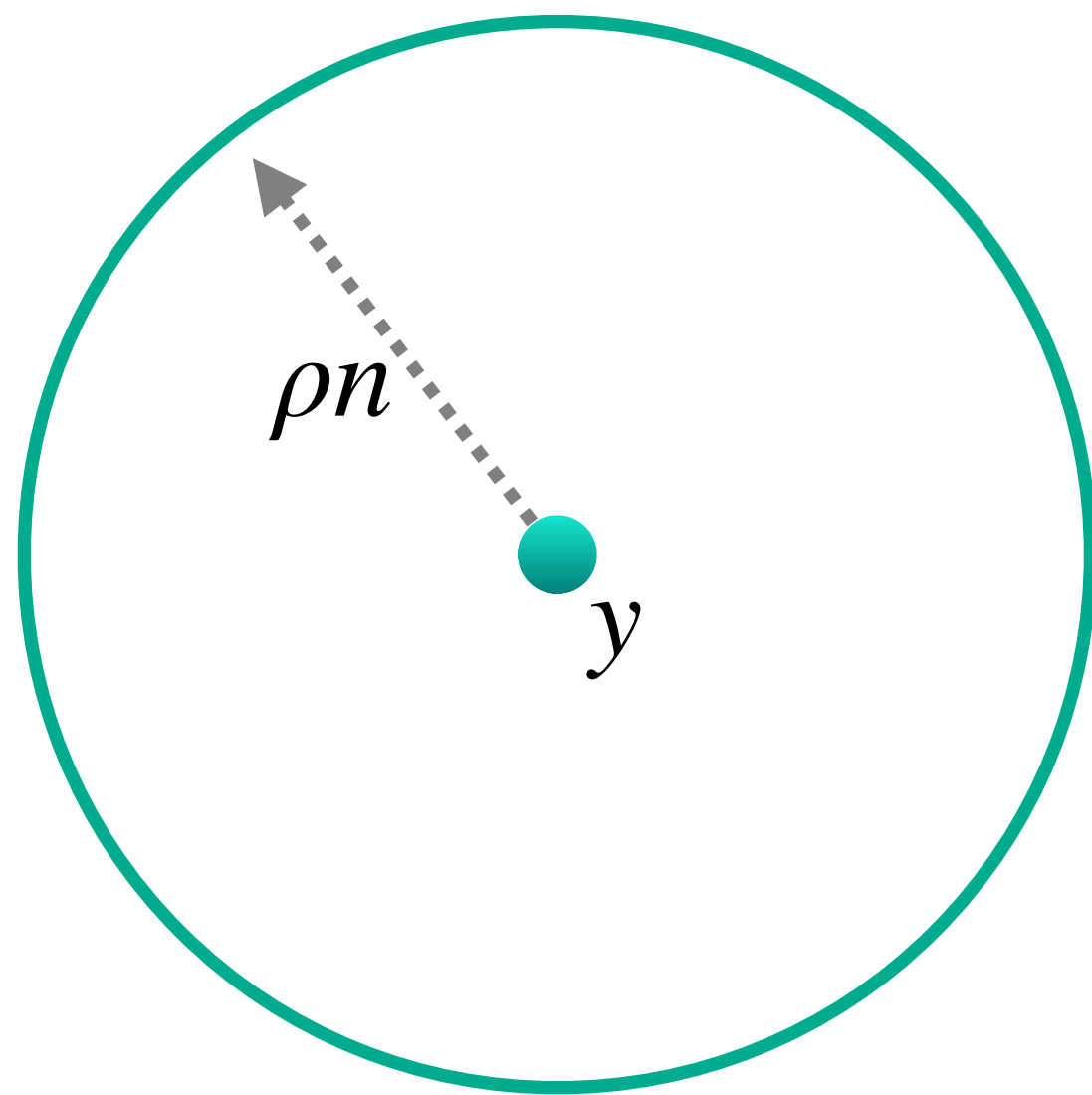
List-decoding: Bridging Hamming and Shannon

Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.



List-decoding: Bridging Hamming and Shannon

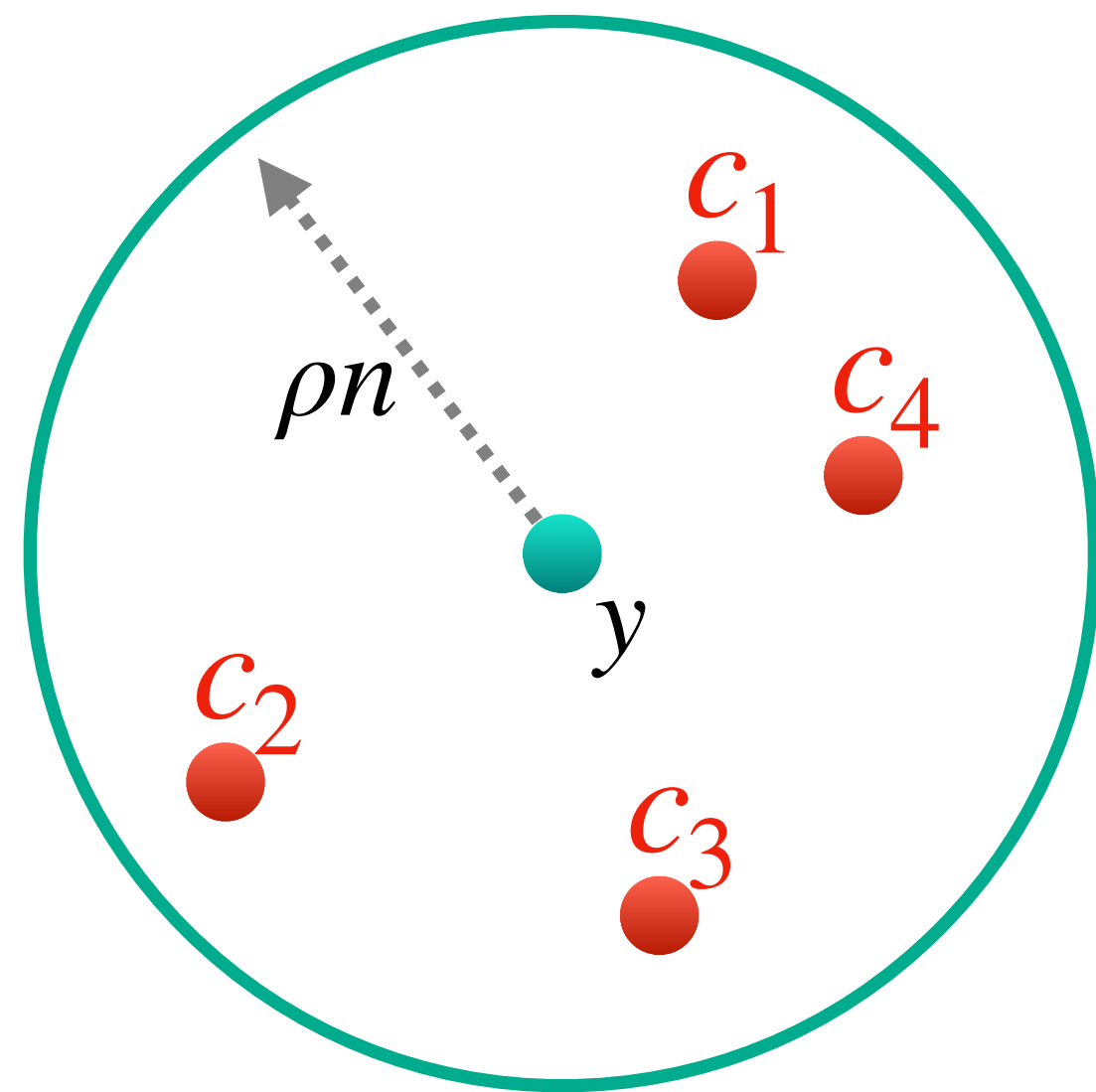
Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.



List-decoding: Bridging Hamming and Shannon

Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.

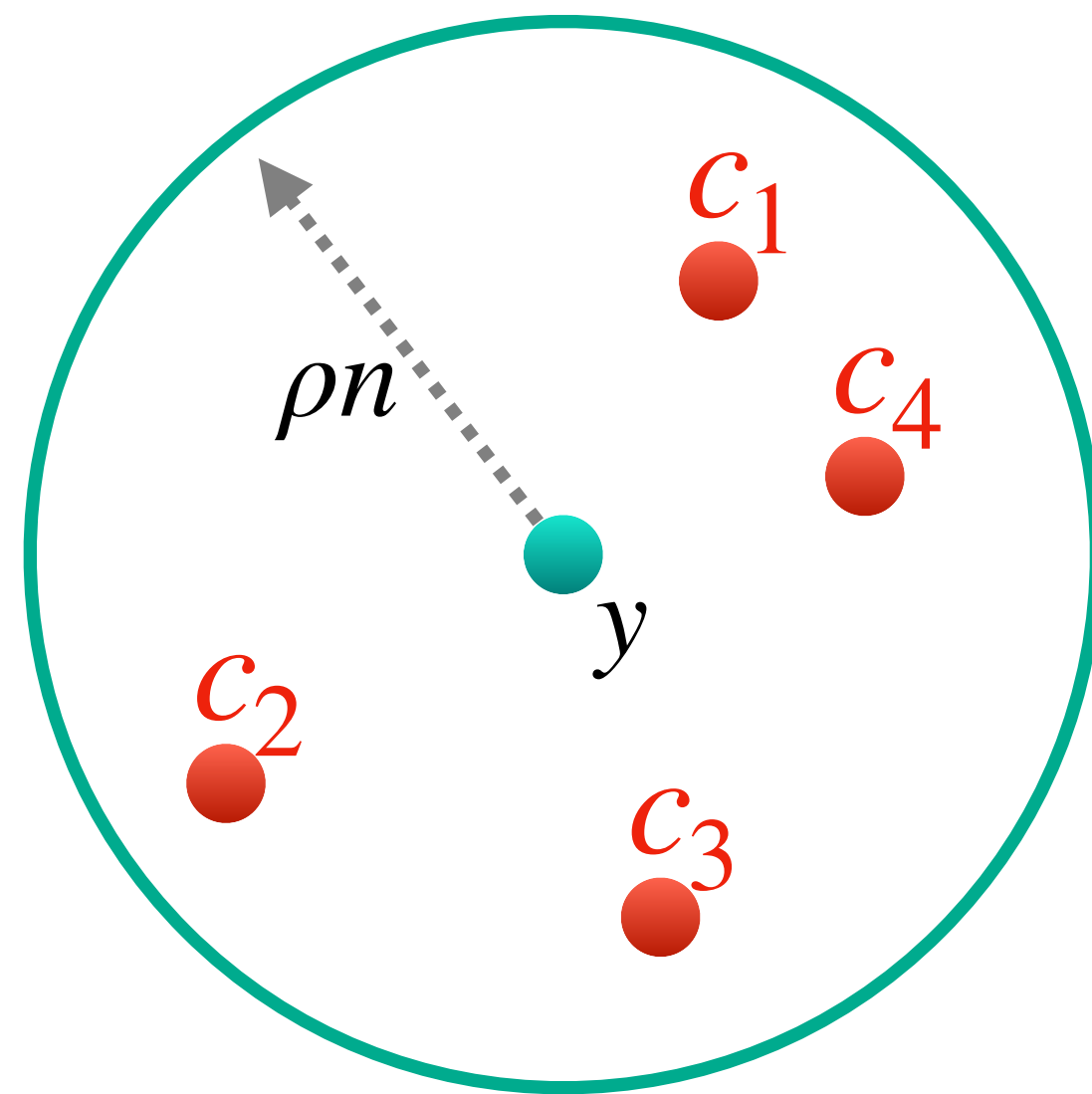
List-decoding: Every ball of radius ρn contains at most L codewords.



List-decoding: Bridging Hamming and Shannon

Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.

List-decoding: Every ball of radius ρn contains at most L codewords.

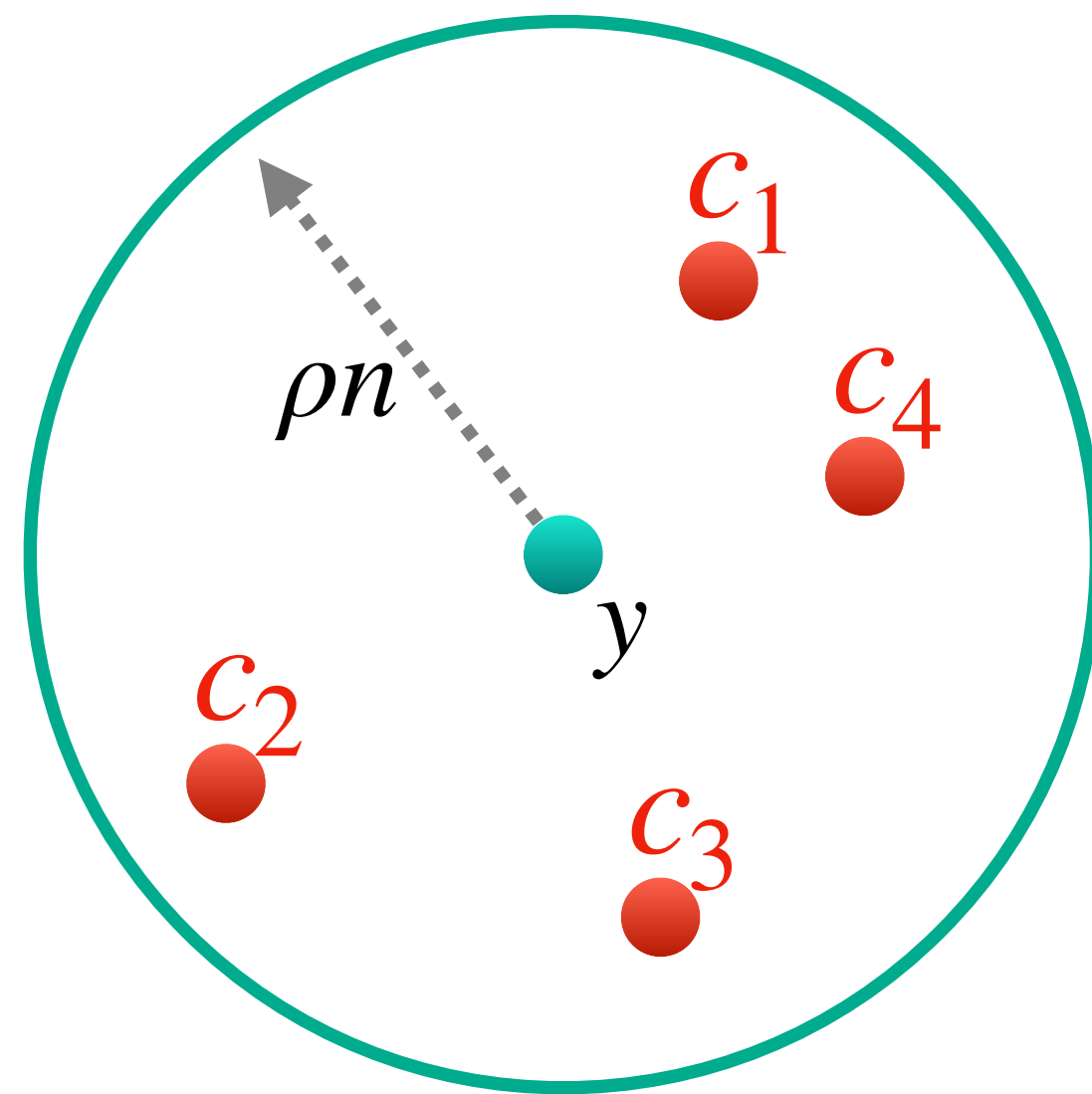


Why care about list-decoding?

List-decoding: Bridging Hamming and Shannon

Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.

List-decoding: Every ball of radius ρn contains at most L codewords.



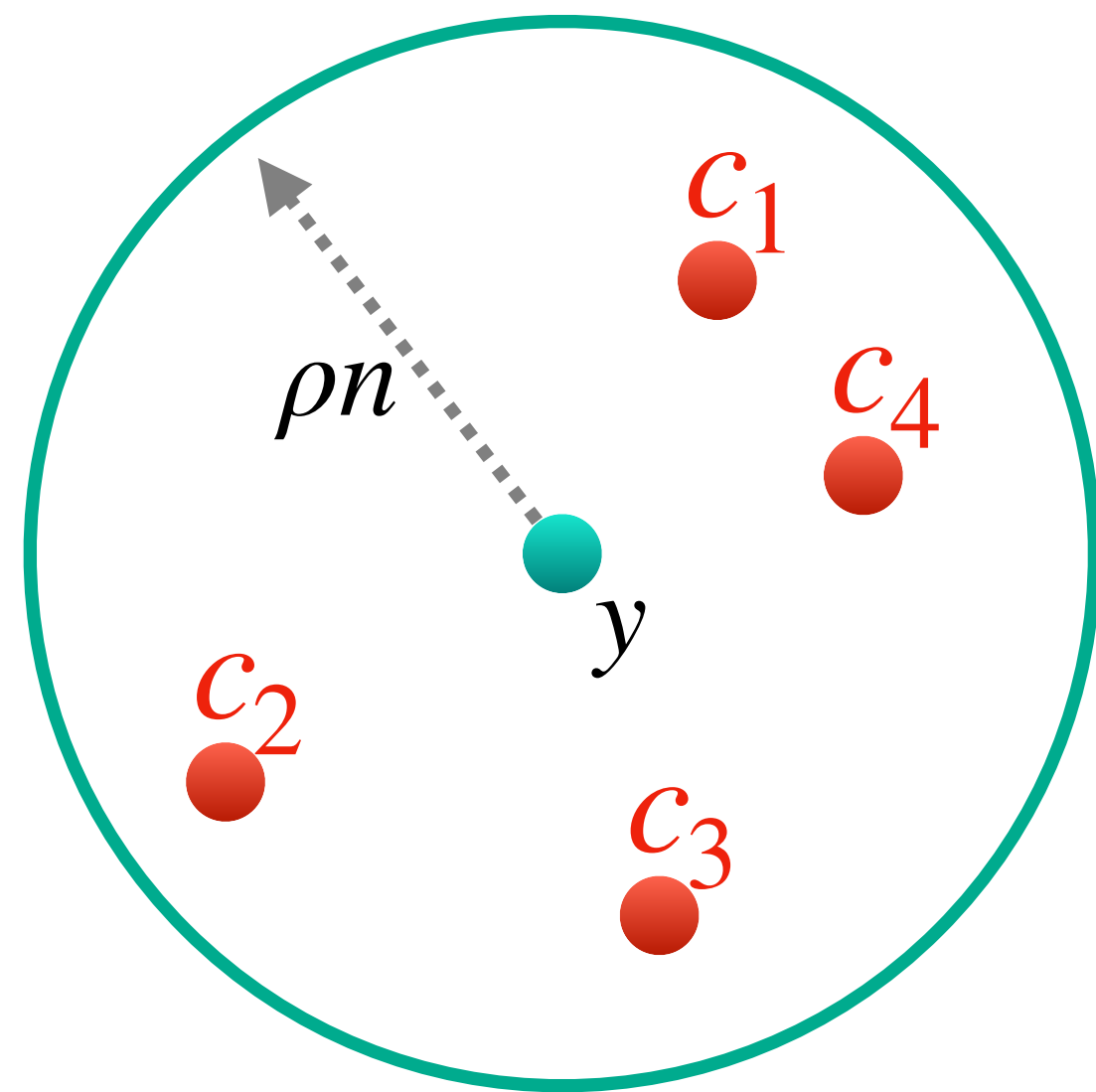
Why care about list-decoding?

- Natural geometric notion;

List-decoding: Bridging Hamming and Shannon

Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.

List-decoding: Every ball of radius ρn contains at most L codewords.



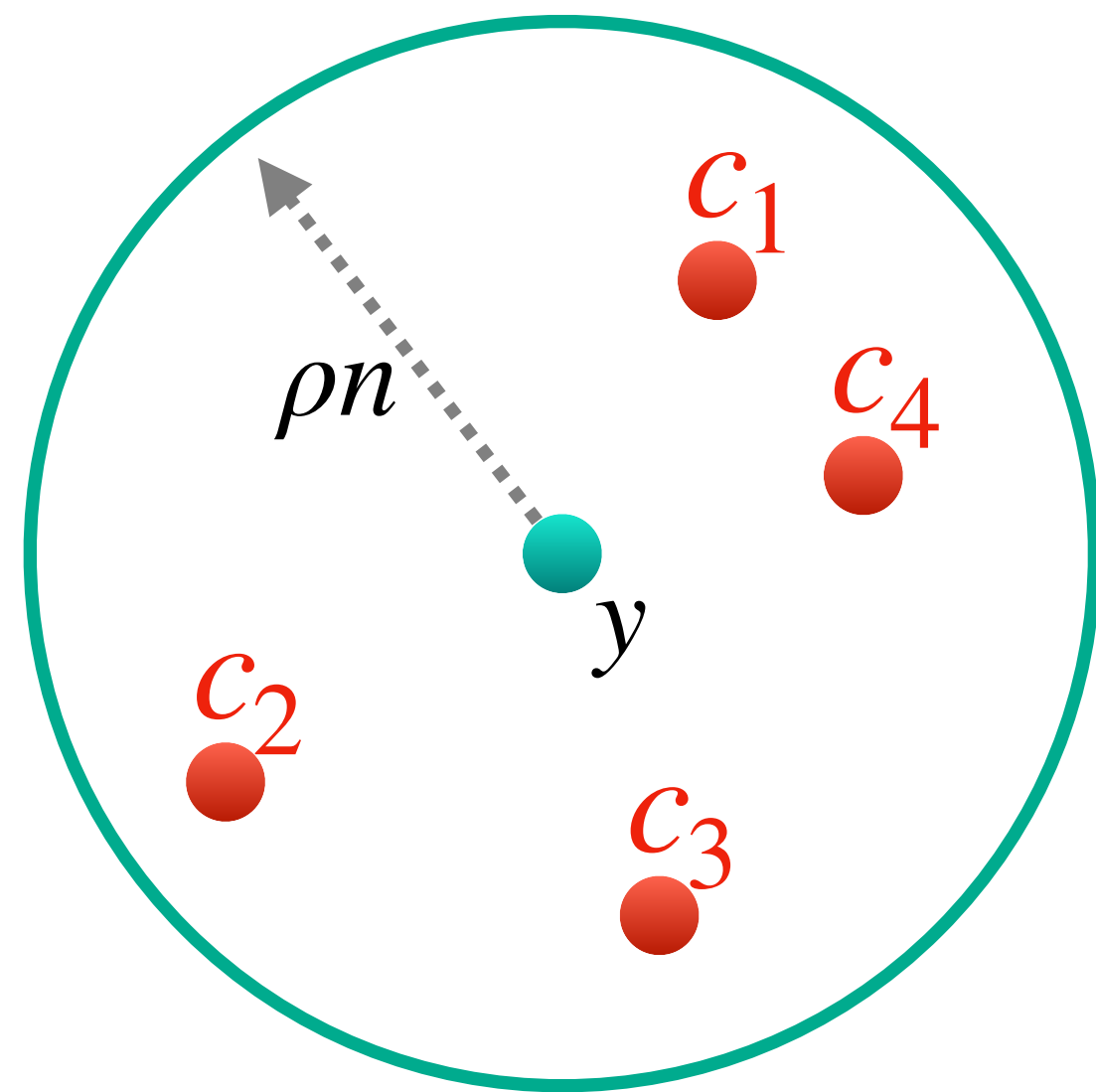
Why care about list-decoding?

- Natural geometric notion;
- Small lists may be OK in practice (pruned with contextual information);

List-decoding: Bridging Hamming and Shannon

Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.

List-decoding: Every ball of radius ρn contains at most L codewords.



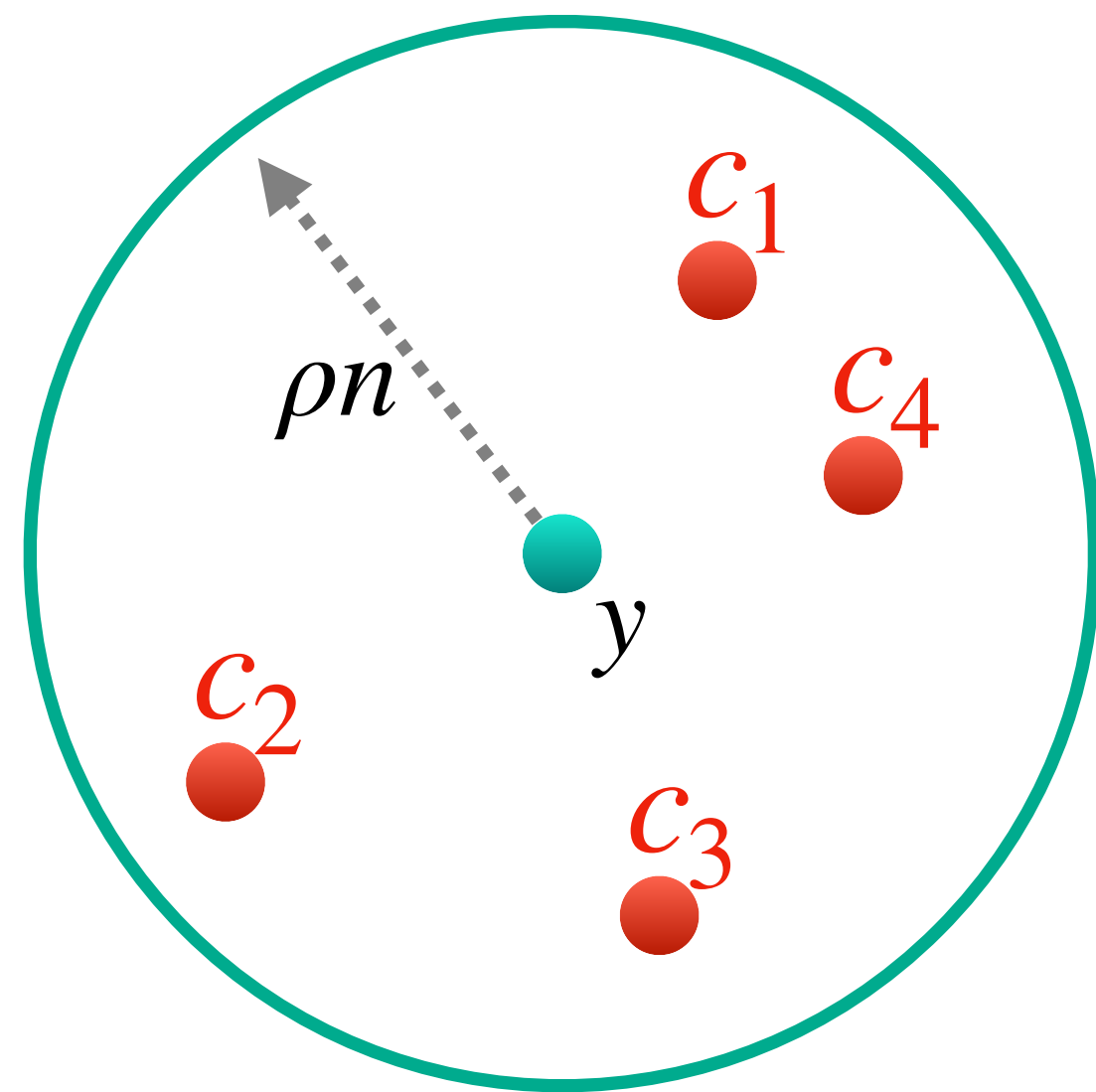
Why care about list-decoding?

- Natural geometric notion;
- Small lists may be OK in practice (pruned with contextual information);
- Stepping stone towards unique decoding;

List-decoding: Bridging Hamming and Shannon

Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.

List-decoding: Every ball of radius ρn contains at most L codewords.



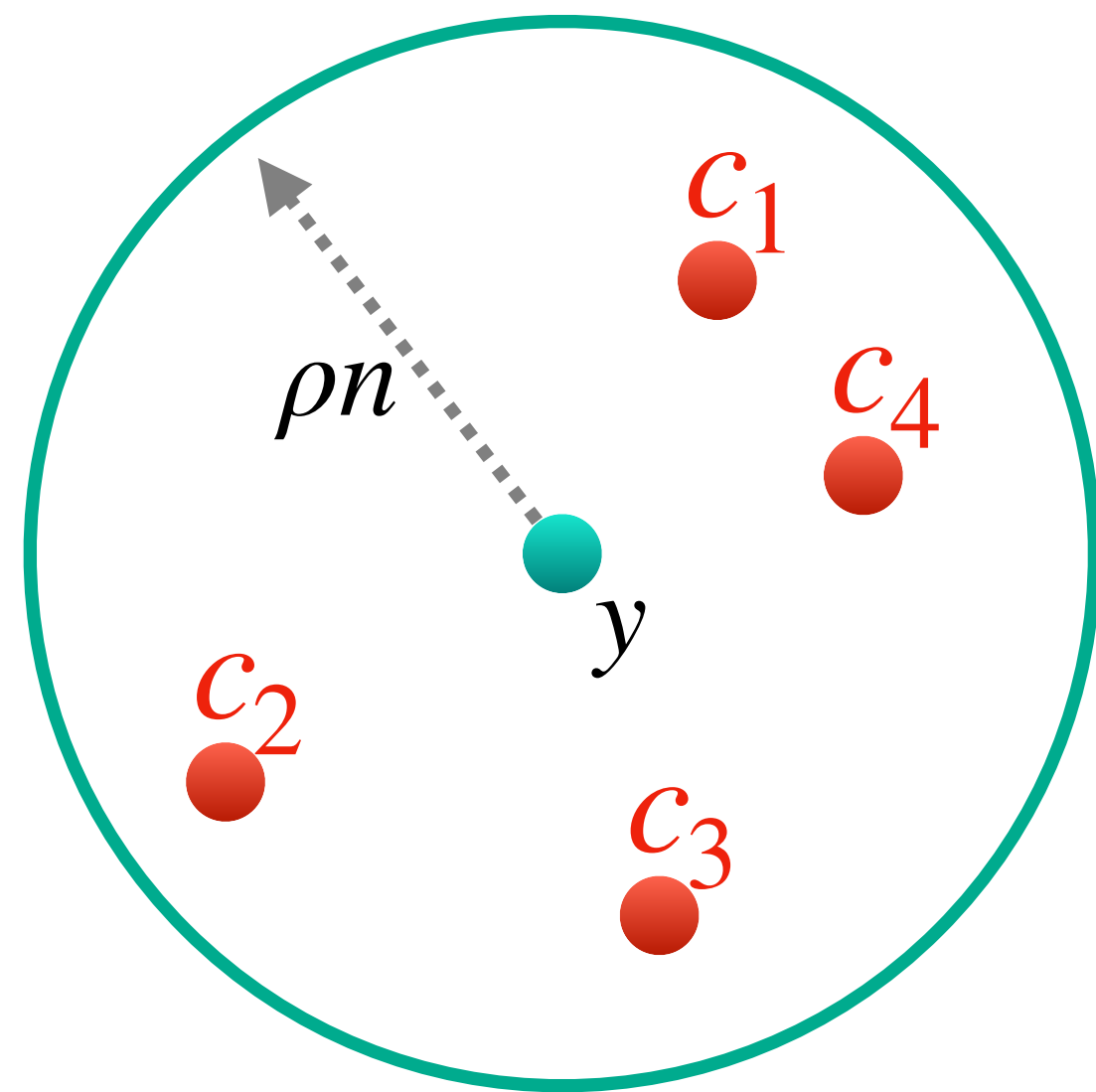
Why care about list-decoding?

- Natural geometric notion;
- Small lists may be OK in practice (pruned with contextual information);
- Stepping stone towards unique decoding;
- Curious connections to random errors!

List-decoding: Bridging Hamming and Shannon

Previously, unique decoding: Every ball of radius ρn contains at most 1 codeword.

List-decoding: Every ball of radius ρn contains at most L codewords.



$$C \subseteq \{0,1\}^n \text{ is } (\rho, L)\text{-list-decodable if for all } y \in \{0,1\}^n, \\ |\{c \in C : d(c, y) \leq \rho n\}| \leq L$$

List-decoding capacity

Like before, we wonder what rates can be achieved for list-decoding.

List-decoding capacity

Like before, we wonder what rates can be achieved for list-decoding.

More concretely:

List-decoding capacity

Like before, we wonder what rates can be achieved for list-decoding.

More concretely:

How large can the (asymptotic) rate of a (ρ, L) -list-decodable code be for fixed radius ρ and “**small**” list size L ?

List-decoding capacity

Like before, we wonder what rates can be achieved for list-decoding.

More concretely:

How large can the (asymptotic) rate of a (ρ, L) -list-decodable code be for fixed radius ρ and “small” list size L ?

List-decoding capacity:

Supremum of all rates R for which there exists a family of (ρ, L_n) -list-decodable codes $C_n \subseteq \{0,1\}^n$ with rate $R_n \rightarrow R$ and list size $L_n \leq n^\alpha$ for all sufficiently large n .

List-decoding capacity

List-decoding capacity:

Supremum of all rates R for which there exists a family of (ρ, L_n) -list-decodable codes $C_n \subseteq \{0,1\}^n$ with rate $R_n \rightarrow R$ and list size $L_n \leq n^\alpha$ for all sufficiently large n .

List-decoding capacity

List-decoding capacity:

Supremum of all rates R for which there exists a family of (ρ, L_n) -list-decodable codes $C_n \subseteq \{0,1\}^n$ with rate $R_n \rightarrow R$ and list size $L_n \leq n^\alpha$ for all sufficiently large n .

This is easy to work out, and we can establish a strong threshold!

List-decoding capacity

List-decoding capacity:

Supremum of all rates R for which there exists a family of (ρ, L_n) -list-decodable codes $C_n \subseteq \{0,1\}^n$ with rate $R_n \rightarrow R$ and list size $L_n \leq n^\alpha$ for all sufficiently large n .

This is easy to work out, and we can establish a strong threshold!

The list-decoding capacity at radius ρ is $1 - h(\rho)$. Furthermore, for every $\varepsilon > 0$:

List-decoding capacity

List-decoding capacity:

Supremum of all rates R for which there exists a family of (ρ, L_n) -list-decodable codes $C_n \subseteq \{0,1\}^n$ with rate $R_n \rightarrow R$ and list size $L_n \leq n^\alpha$ for all sufficiently large n .

This is easy to work out, and we can establish a strong threshold!

The list-decoding capacity at radius ρ is $1 - h(\rho)$. Furthermore, for every $\varepsilon > 0$:

- Every code of rate $R \geq 1 - h(\rho) + \varepsilon$ has **exponential** list size $L \geq 2^{\varepsilon n}$.

List-decoding capacity

List-decoding capacity:

Supremum of all rates R for which there exists a family of (ρ, L_n) -list-decodable codes $C_n \subseteq \{0,1\}^n$ with rate $R_n \rightarrow R$ and list size $L_n \leq n^\alpha$ for all sufficiently large n .

This is easy to work out, and we can establish a strong threshold!

The list-decoding capacity at radius ρ is $1 - h(\rho)$. Furthermore, for every $\varepsilon > 0$:

- Every code of rate $R \geq 1 - h(\rho) + \varepsilon$ has **exponential** list size $L \geq 2^{\varepsilon n}$.
- A **uniformly random code** of rate $R = 1 - h(\rho) - \varepsilon$ has **constant** list size $L \leq 1/\varepsilon$ with high probability.

List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;

List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

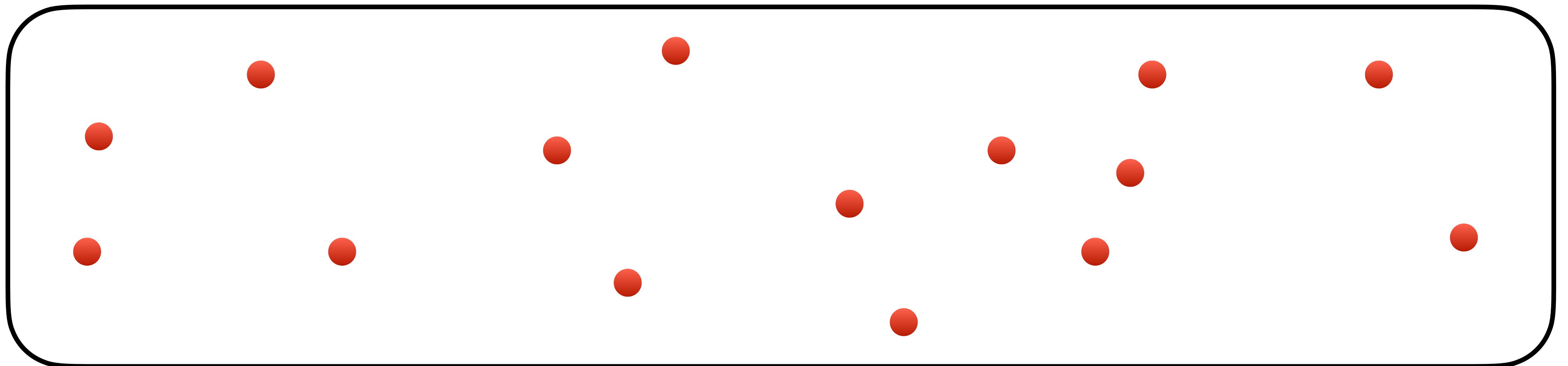
- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:

List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);

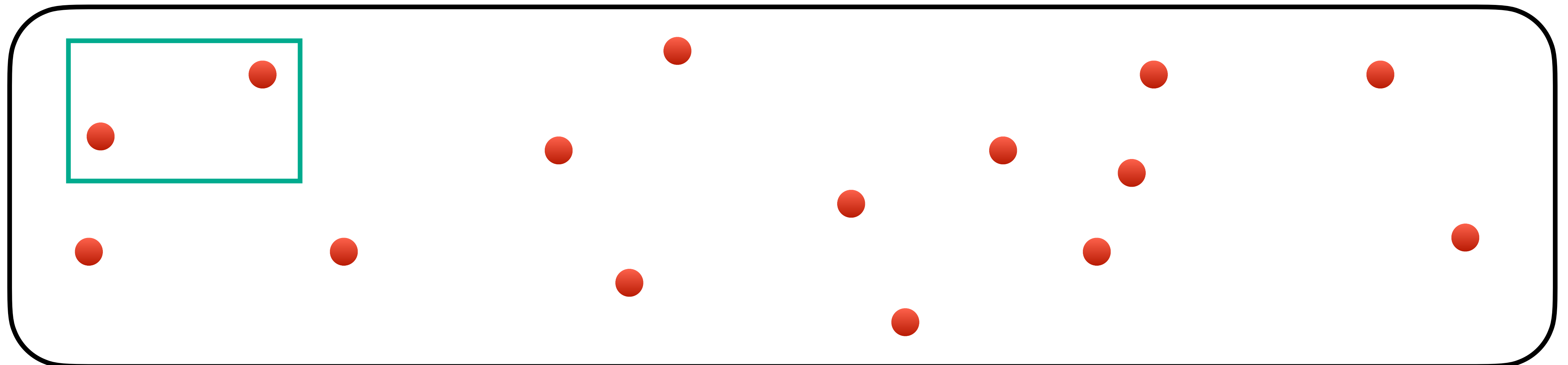
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



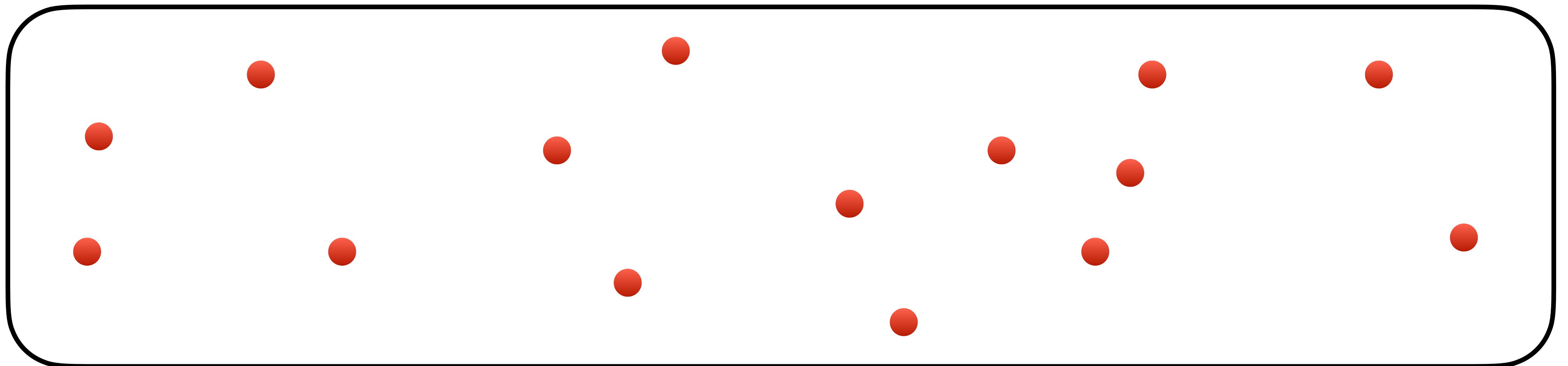
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



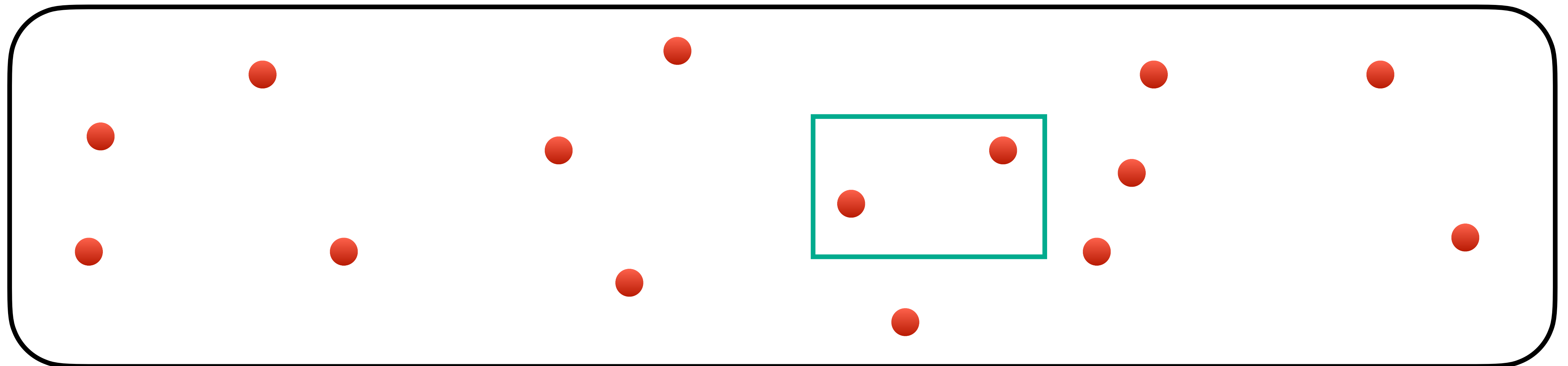
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



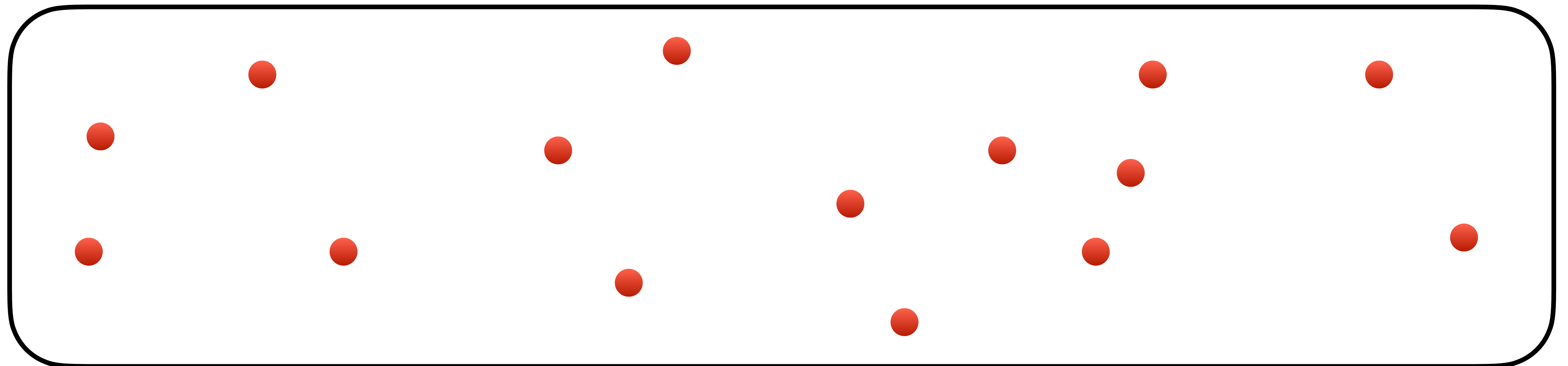
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



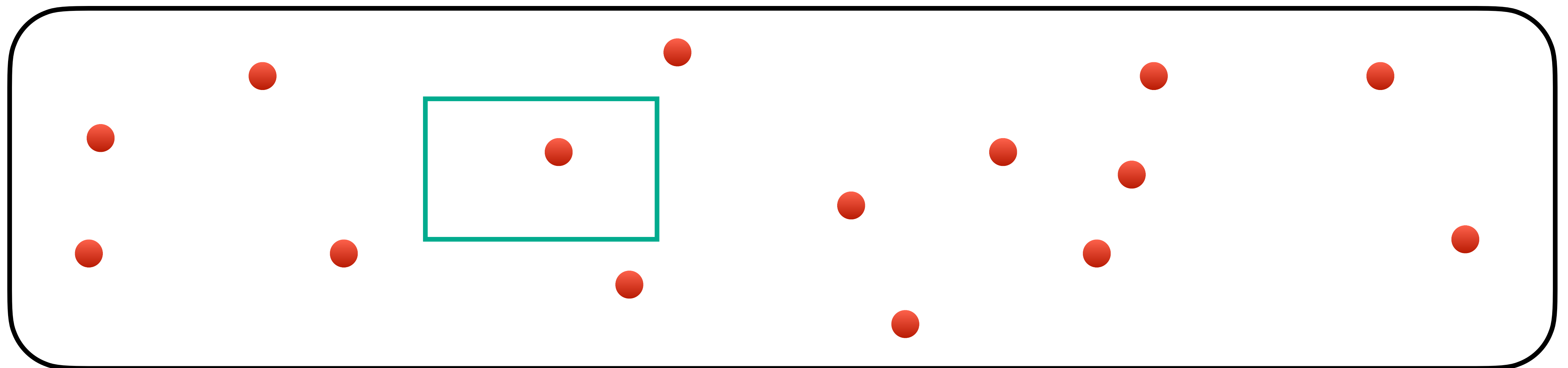
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



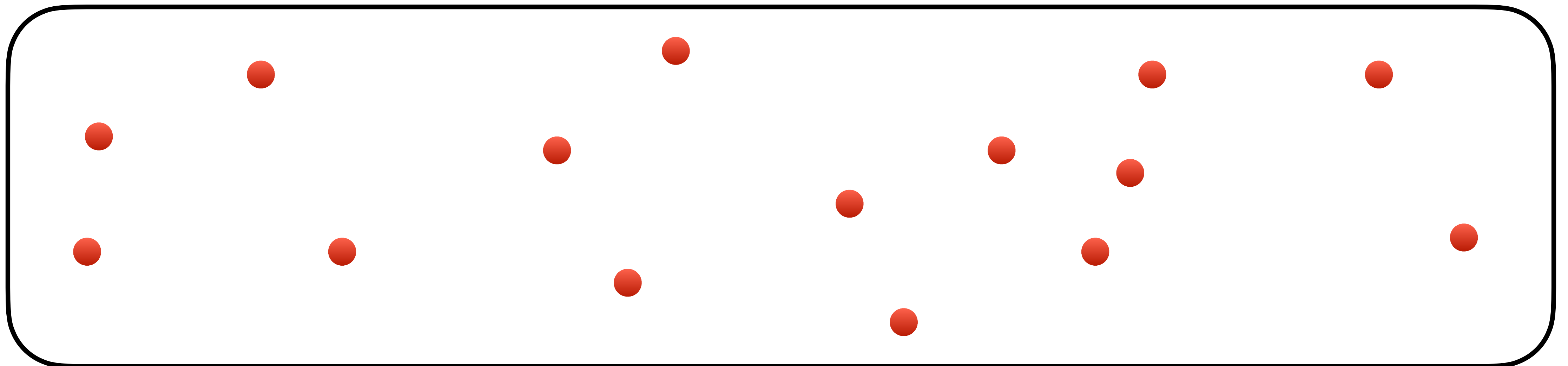
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



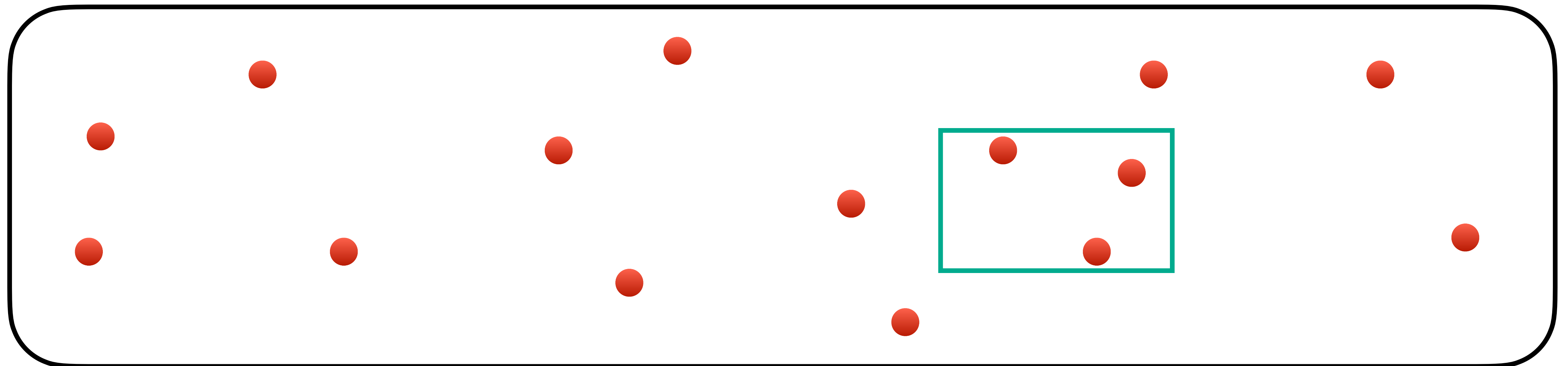
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



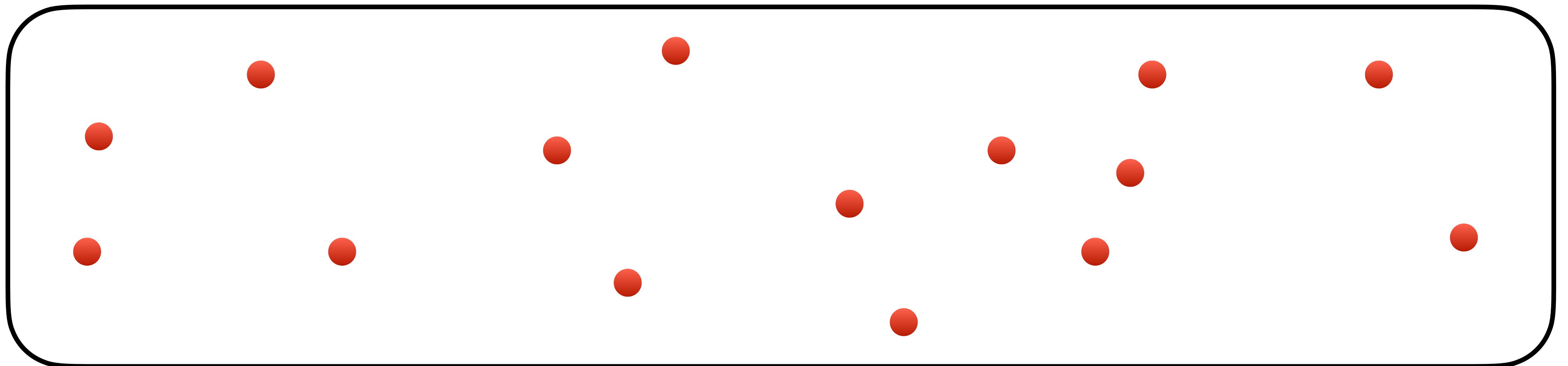
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



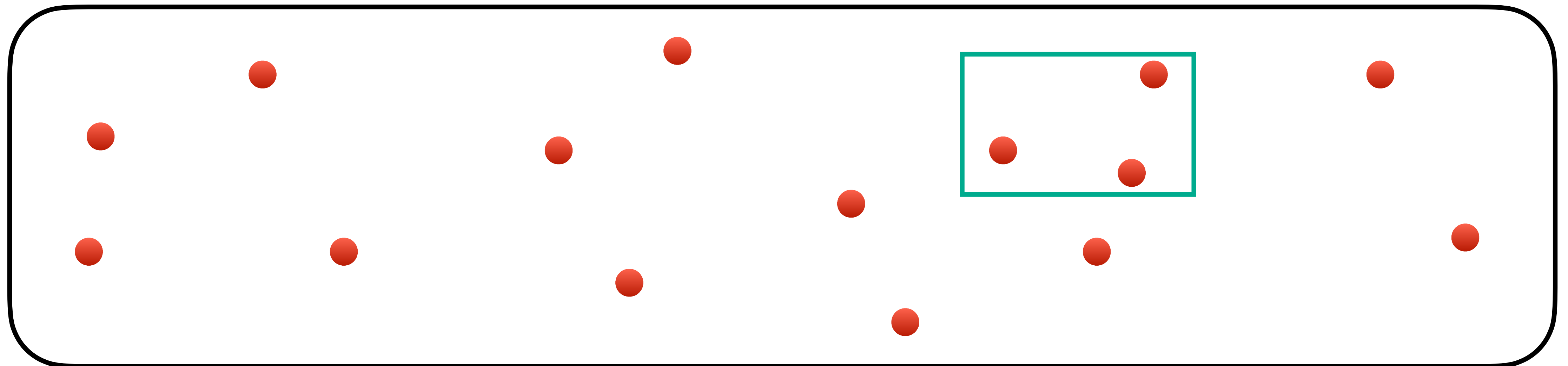
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



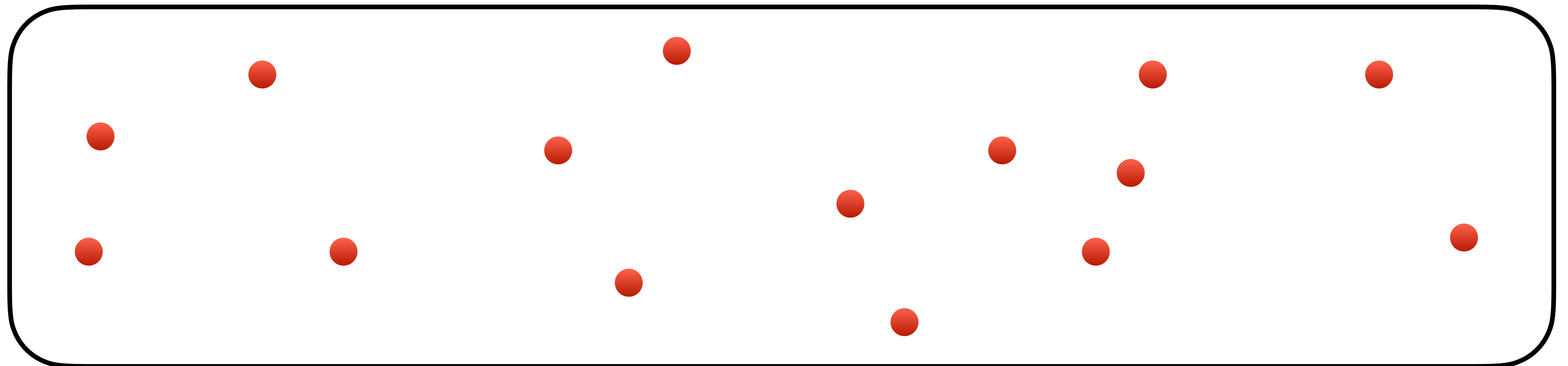
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);



List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);

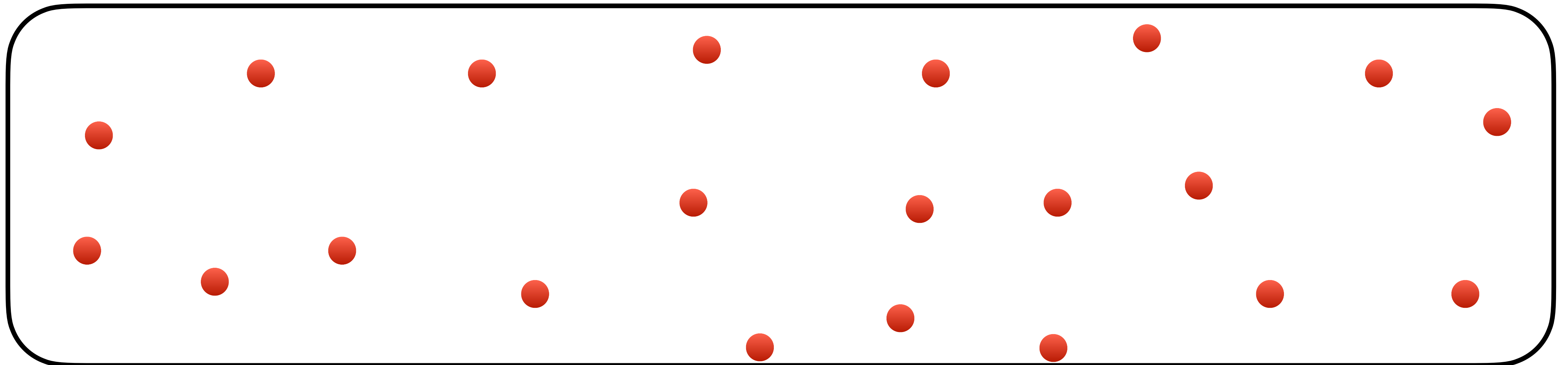


List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);
 - Covering problems dual to sphere packing.

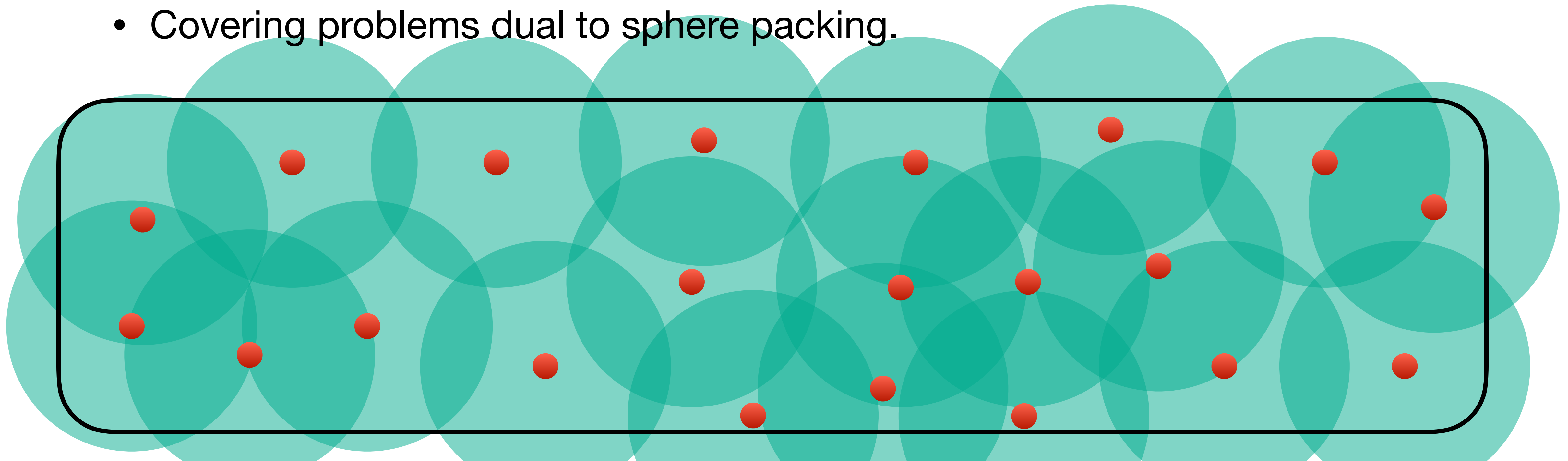
List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
- This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);
 - Covering problems dual to sphere packing.



List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
- This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);
 - Covering problems dual to sphere packing.



List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);
 - Covering problems dual to sphere packing.

List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);
 - Covering problems dual to sphere packing.
- For unique decoding, the best we know is $1 - h(2\rho)$, achieved by random linear codes;

List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);
 - Covering problems dual to sphere packing.
- For unique decoding, the best we know is $1 - h(2\rho)$, achieved by random linear codes;
 - Curiously, uniformly random codes are worse!

List-decoding adversarial bit-flips vs. Unique-decoding adversarial bit-flips

- For list-decoding, the optimal rate is $1 - h(\rho)$, achieved both by uniformly random codes and by random linear codes;
 - This match between uniformly random codes and random linear codes holds more generally for:
 - Many shapes beyond Hamming balls (e.g., combinatorial rectangles);
 - Covering problems dual to sphere packing.
- For unique decoding, the best we know is $1 - h(2\rho)$, achieved by random linear codes;
 - Curiously, uniformly random codes are worse!
 - A uniformly random code only achieves rate $\frac{1}{2}(1 - h(2\rho))$ for unique decoding.

List-decoding adversarial bit-flips vs. Unique-decoding random bit-flips

List-decoding adversarial bit-flips vs. Unique-decoding random bit-flips

- In both cases, the optimal rate is $1 - h(\rho)$;

List-decoding adversarial bit-flips vs. Unique-decoding random bit-flips

- In both cases, the optimal rate is $1 - h(\rho)$;
- In both cases, the optimal rate is achieved with high probability by:

List-decoding adversarial bit-flips vs. Unique-decoding random bit-flips

- In both cases, the optimal rate is $1 - h(\rho)$;
- In both cases, the optimal rate is achieved with high probability by:
 - A uniformly random code;

List-decoding adversarial bit-flips vs. Unique-decoding random bit-flips

- In both cases, the optimal rate is $1 - h(\rho)$;
- In both cases, the optimal rate is achieved with high probability by:
 - A uniformly random code;
 - A random linear code [Guruswami-Håstad-Sudan-Zuckerman 2002, Li-Wootters 2018].

List-decoding adversarial bit-flips vs. Unique-decoding random bit-flips

- In both cases, the optimal rate is $1 - h(\rho)$;
- In both cases, the optimal rate is achieved with high probability by:
 - A uniformly random code;
 - A random linear code [Guruswami-Håstad-Sudan-Zuckerman 2002, Li-Wootters 2018].
- Is this a coincidence?

List-decoding adversarial bit-flips vs. Unique-decoding random bit-flips

- In both cases, the optimal rate is $1 - h(\rho)$;
- In both cases, the optimal rate is achieved with high probability by:
 - A uniformly random code;
 - A random linear code [Guruswami-Håstad-Sudan-Zuckerman 2002, Li-Wootters 2018].
- Is this a coincidence?

Pernice-Sprumont-Wootters 2025:

Ignoring technicalities... Any* linear code that achieves list-decoding capacity also achieves capacity on the BSC.

Philosophizing beyond bit-flips

Philosophizing beyond bit-flips

- Based on what we've seen, we may hope that insights about channel capacity tell us new things about list-decoding capacity, and vice-versa;

Philosophizing beyond bit-flips

- Based on what we've seen, we may hope that insights about channel capacity tell us new things about list-decoding capacity, and vice-versa;
- For bit-flips this is less interesting, because the capacity of the BSC and the capacity of list-decoding from bit-flips are both easy to determine.

Philosophizing beyond bit-flips

- Based on what we've seen, we may hope that insights about channel capacity tell us new things about list-decoding capacity, and vice-versa;
- For bit-flips this is less interesting, because the capacity of the BSC and the capacity of list-decoding from bit-flips are both easy to determine.

But what about other types of errors?

Insertions and deletions

Simple models of errors that cause loss of synchronization.

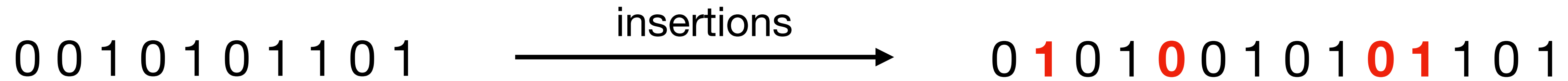
Insertions and deletions

Simple models of errors that cause loss of synchronization.

0 0 1 0 1 0 1 1 0 1 $\xrightarrow{\text{insertions}}$ 0 1 0 1 0 0 1 0 1 0 1 1 0 1

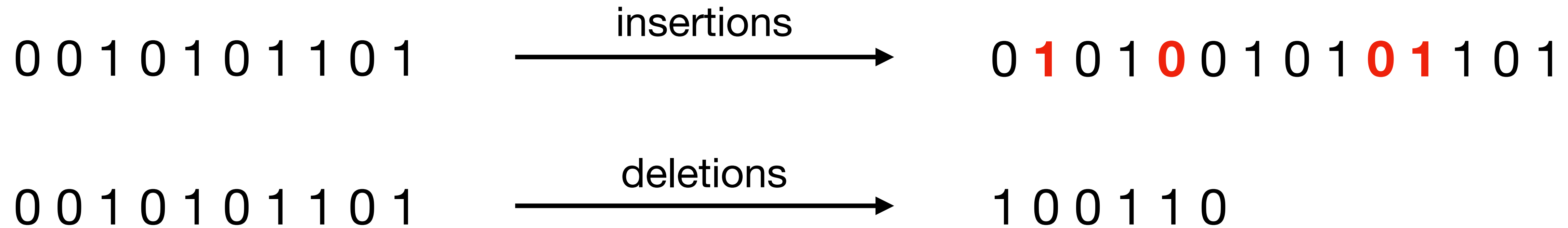
Insertions and deletions

Simple models of errors that cause loss of synchronization.



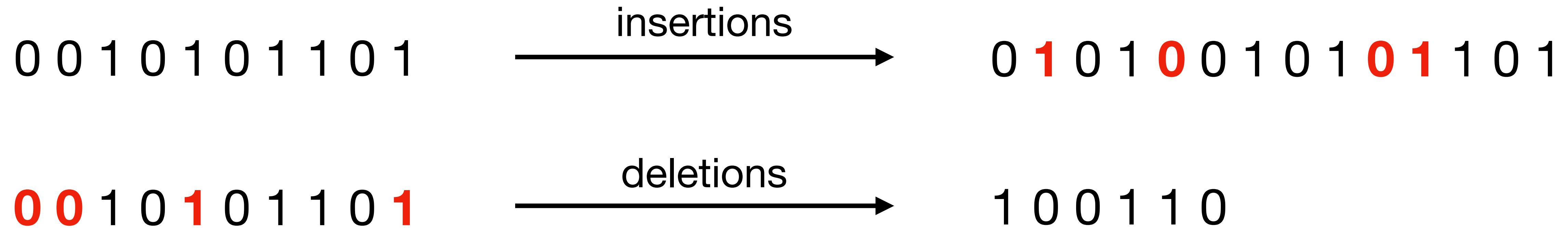
Insertions and deletions

Simple models of errors that cause loss of synchronization.



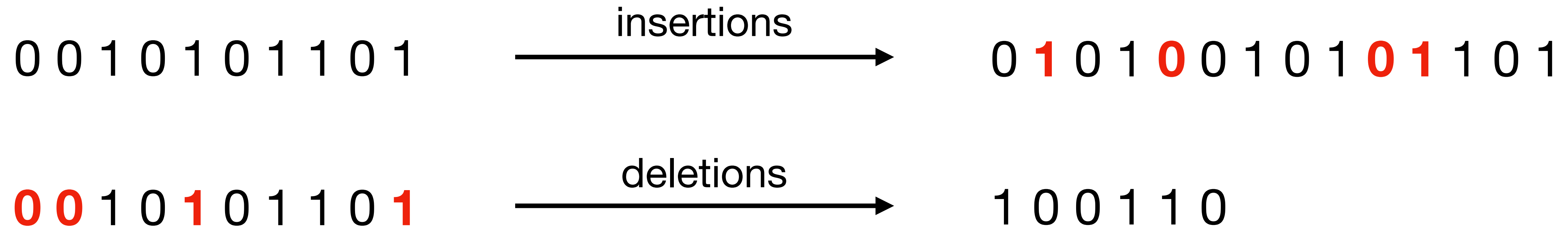
Insertions and deletions

Simple models of errors that cause loss of synchronization.



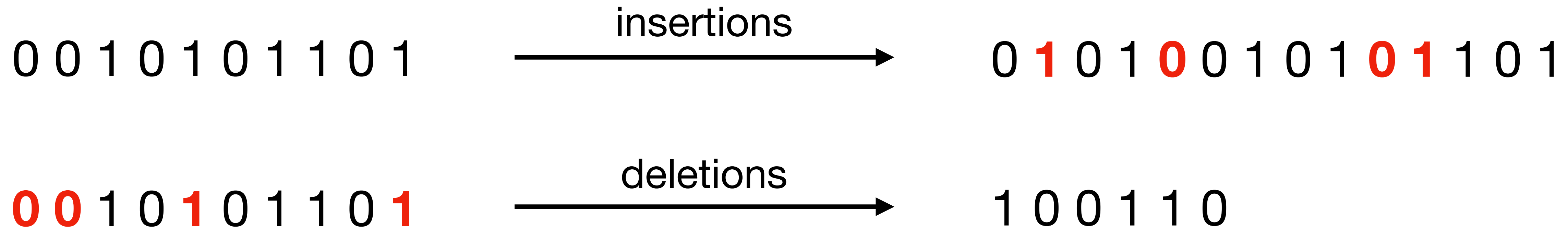
Insertions and deletions

Simple models of errors that cause loss of synchronization.



Insertions and deletions

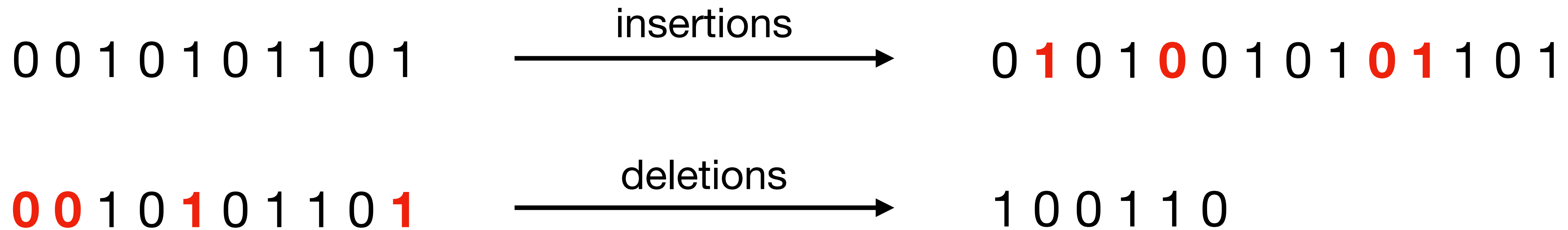
Simple models of errors that cause loss of synchronization.



Despite 50+ years of effort, we still don't know the capacity of channels with random insdels.

Insertions and deletions

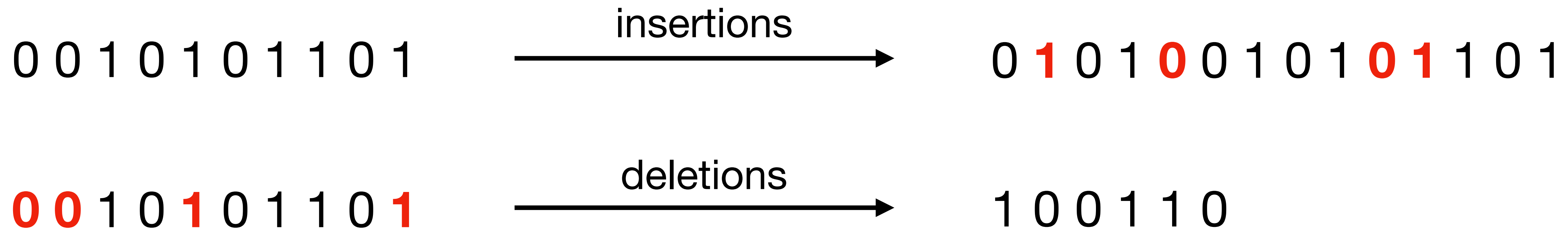
Simple models of errors that cause loss of synchronization.



Despite 50+ years of effort, we still don't know the capacity of channels with random insdels. We also know little about the limits of list-decoding from insdels.

Insertions and deletions

Simple models of errors that cause loss of synchronization.

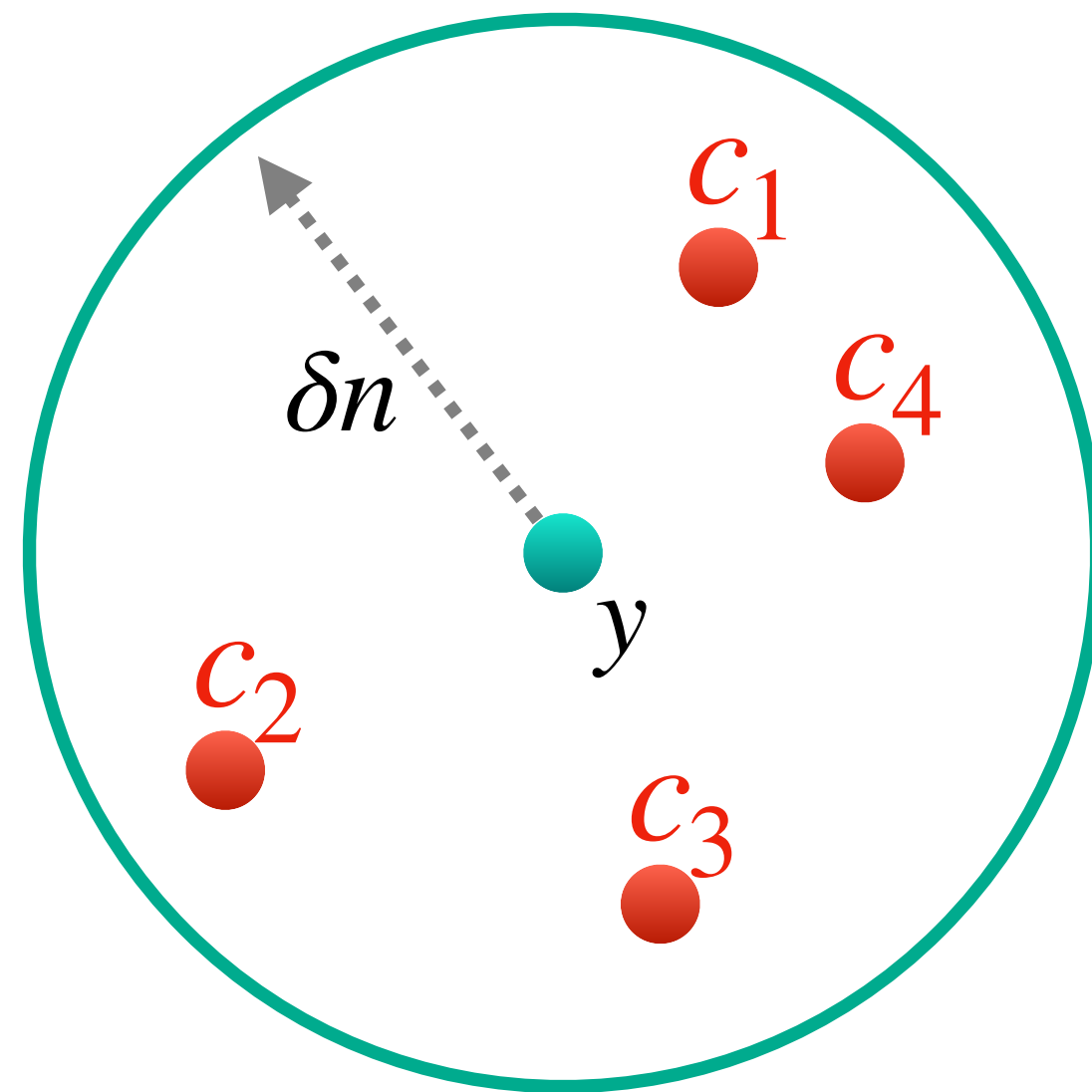


Despite 50+ years of effort, we still don't know the capacity of channels with random insdels.
We also know little about the limits of list-decoding from insdels.

Can we make progress by transferring insights between these problems?

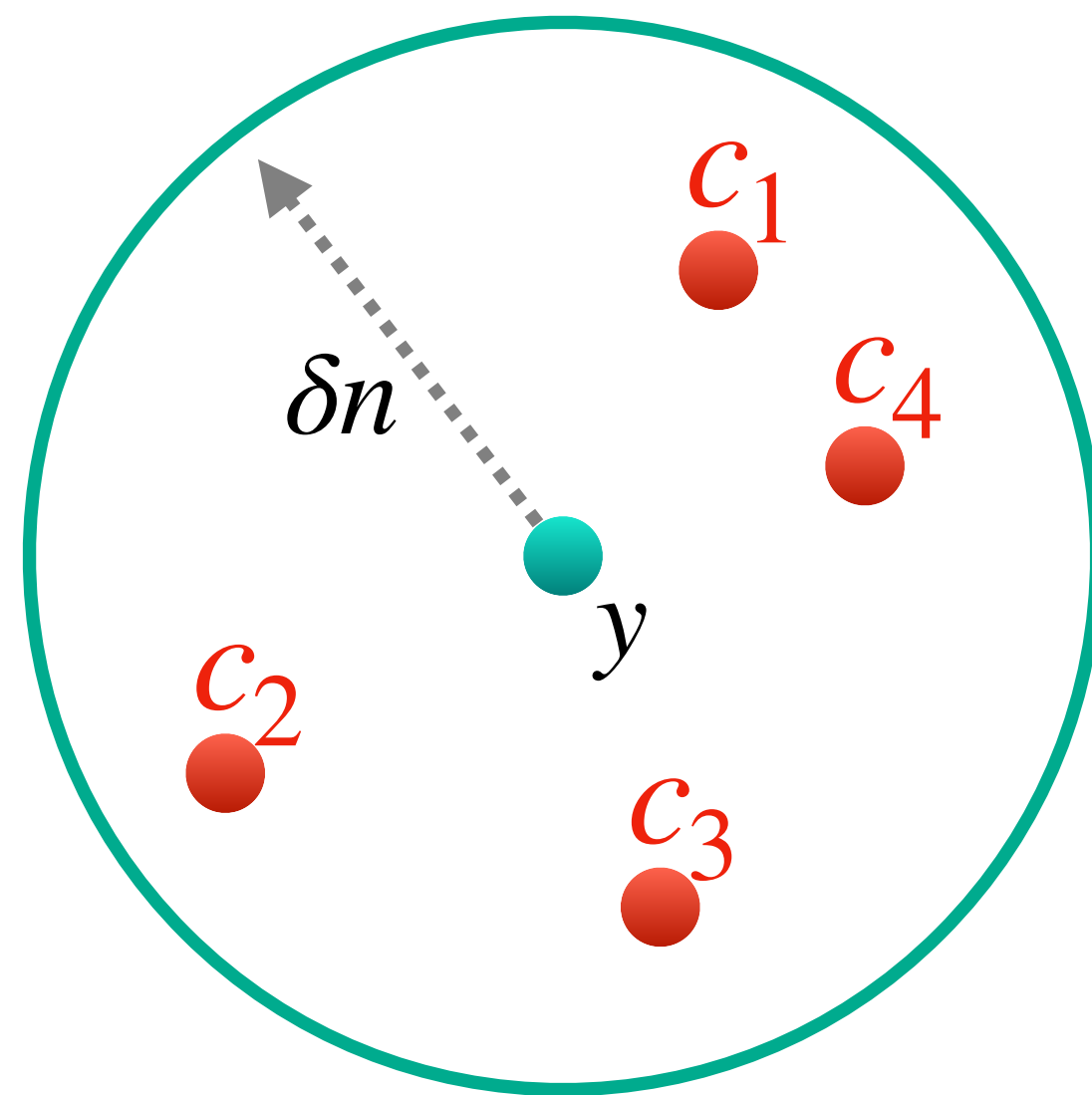
List-decoding from insertions

A useful relaxation of unique decoding.



List-decoding from insertions

A useful relaxation of unique decoding.



$C \subseteq \{0,1\}^n$ is (ρ, L) -list-decodable from insertions if
for all $y \in \{0,1\}^{(1+\rho)n}$

$$|\{c \in C : y \text{ is supersequence of } c\}| \leq L$$

Capacity of list-decoding from insertions — the story so far

$C_{\text{ins}}(\rho)$ = largest rate achievable by (ρ, L) -list-decodable code with small list size L .

Capacity of list-decoding from insertions — the story so far

$C_{\text{ins}}(\rho)$ = largest rate achievable by (ρ, L) -list-decodable code with small list size L .

As before, a natural starting point is to analyze a **uniformly random code**...

Capacity of list-decoding from insertions — the story so far

$C_{\text{ins}}(\rho)$ = largest rate achievable by (ρ, L) -list-decodable code with small list size L .

As before, a natural starting point is to analyze a **uniformly random code**...

[Haeupler-Shahrasbi-Sudan '18]: For $\rho \in [0, 1]$,

$$(1 + \rho) \left(1 - h \left(\frac{\rho}{1 + \rho} \right) \right) \geq C_{\text{ins}}(\rho) \geq \begin{cases} 1 - h(\rho), & \text{if } \rho < 1/2, \\ 0, & \text{if } \rho \geq 1/2. \end{cases}$$

Capacity of list-decoding from insertions — the story so far

$C_{\text{ins}}(\rho)$ = largest rate achievable by (ρ, L) -list-decodable code with small list size L .

As before, a natural starting point is to analyze a **uniformly random code**...

[Haeupler-Shahrasbi-Sudan '18]: For $\rho \in [0, 1]$,

$$(1 + \rho) \left(1 - h \left(\frac{\rho}{1 + \rho} \right) \right) \geq C_{\text{ins}}(\rho) \geq \begin{cases} 1 - h(\rho), & \text{if } \rho < 1/2, \\ 0, & \text{if } \rho \geq 1/2. \end{cases}$$

Not obvious whether $C_{\text{ins}}(\rho) > 0$ when $\rho \geq 1/2$.

Capacity of list-decoding from insertions — the story so far

$C_{\text{ins}}(\rho)$ = largest rate achievable by (ρ, L) -list-decodable code with small list size L .

As before, a natural starting point is to analyze a **uniformly random code**...

[Haeupler-Shahrasbi-Sudan '18]: For $\rho \in [0, 1]$,

$$(1 + \rho) \left(1 - h \left(\frac{\rho}{1 + \rho} \right) \right) \geq C_{\text{ins}}(\rho) \geq \begin{cases} 1 - h(\rho), & \text{if } \rho < 1/2, \\ 0, & \text{if } \rho \geq 1/2. \end{cases}$$

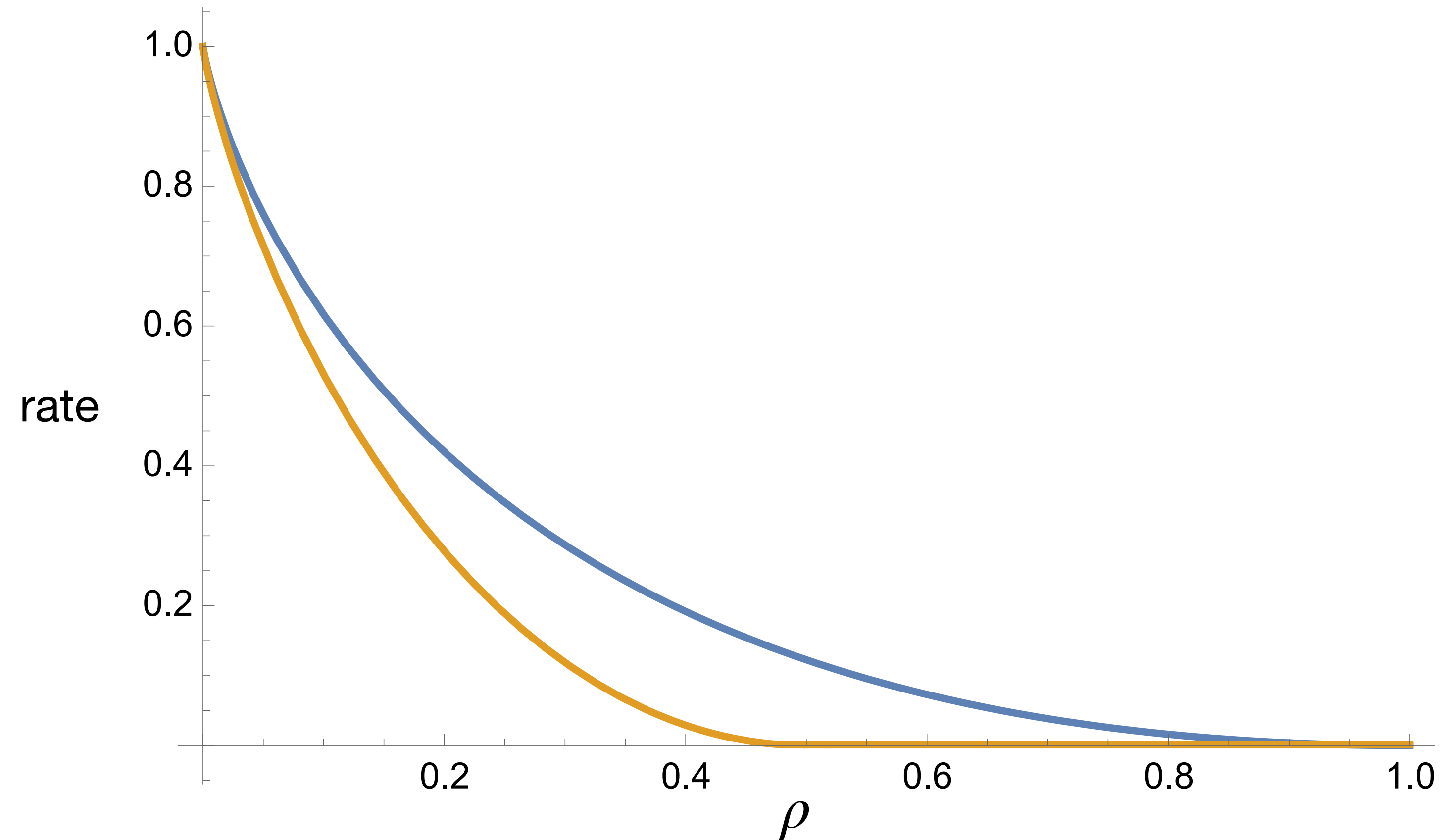
Not obvious whether $C_{\text{ins}}(\rho) > 0$ when $\rho \geq 1/2$.

[Guruswami-Haeupler-Shahrasbi '20]:

$$C_{\text{ins}}(\rho) > 0 \text{ for all } \rho < 1.$$

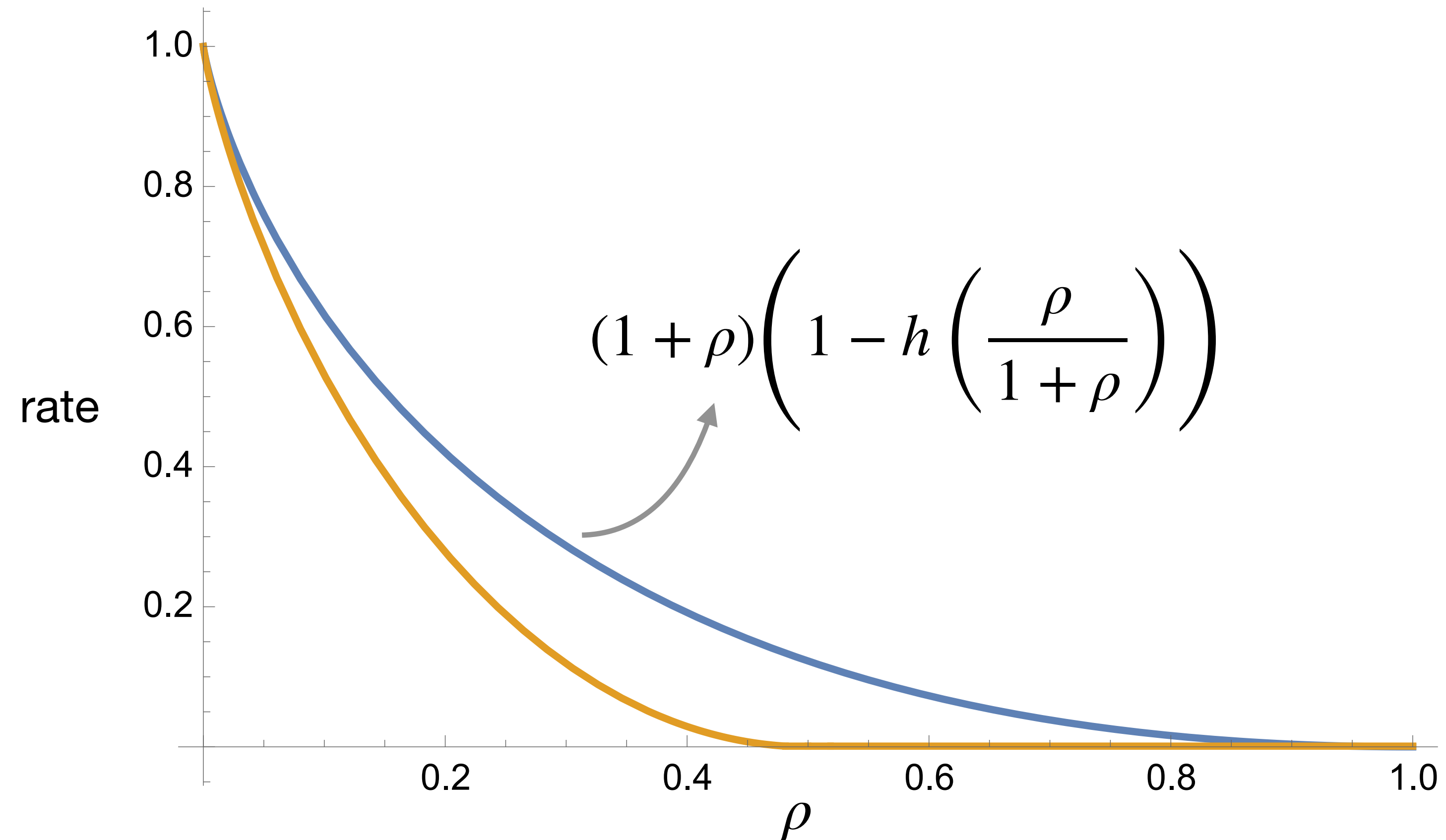
Capacity of list-decoding from insertions — the story so far

Summarizing:



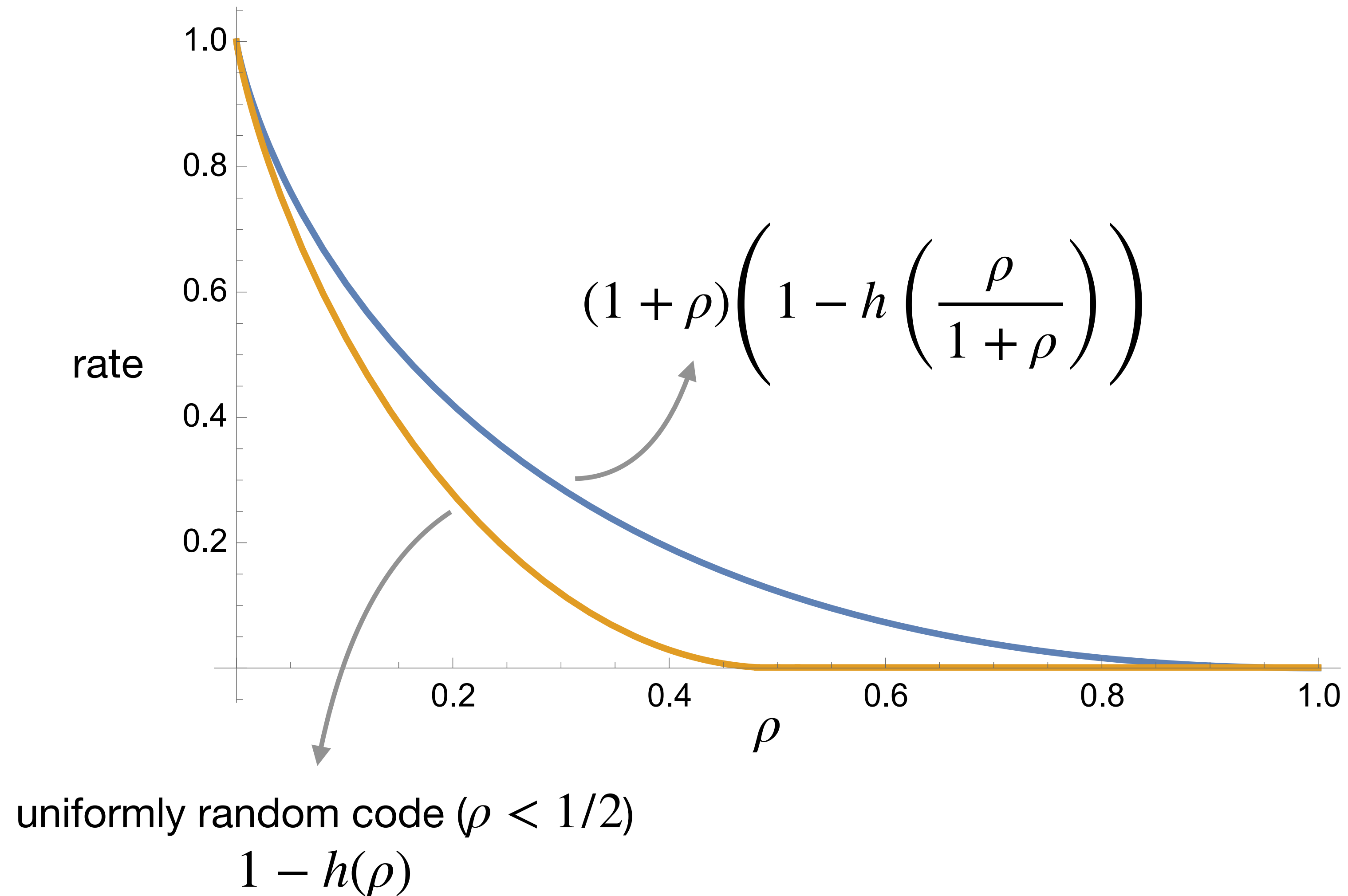
Capacity of list-decoding from insertions — the story so far

Summarizing:



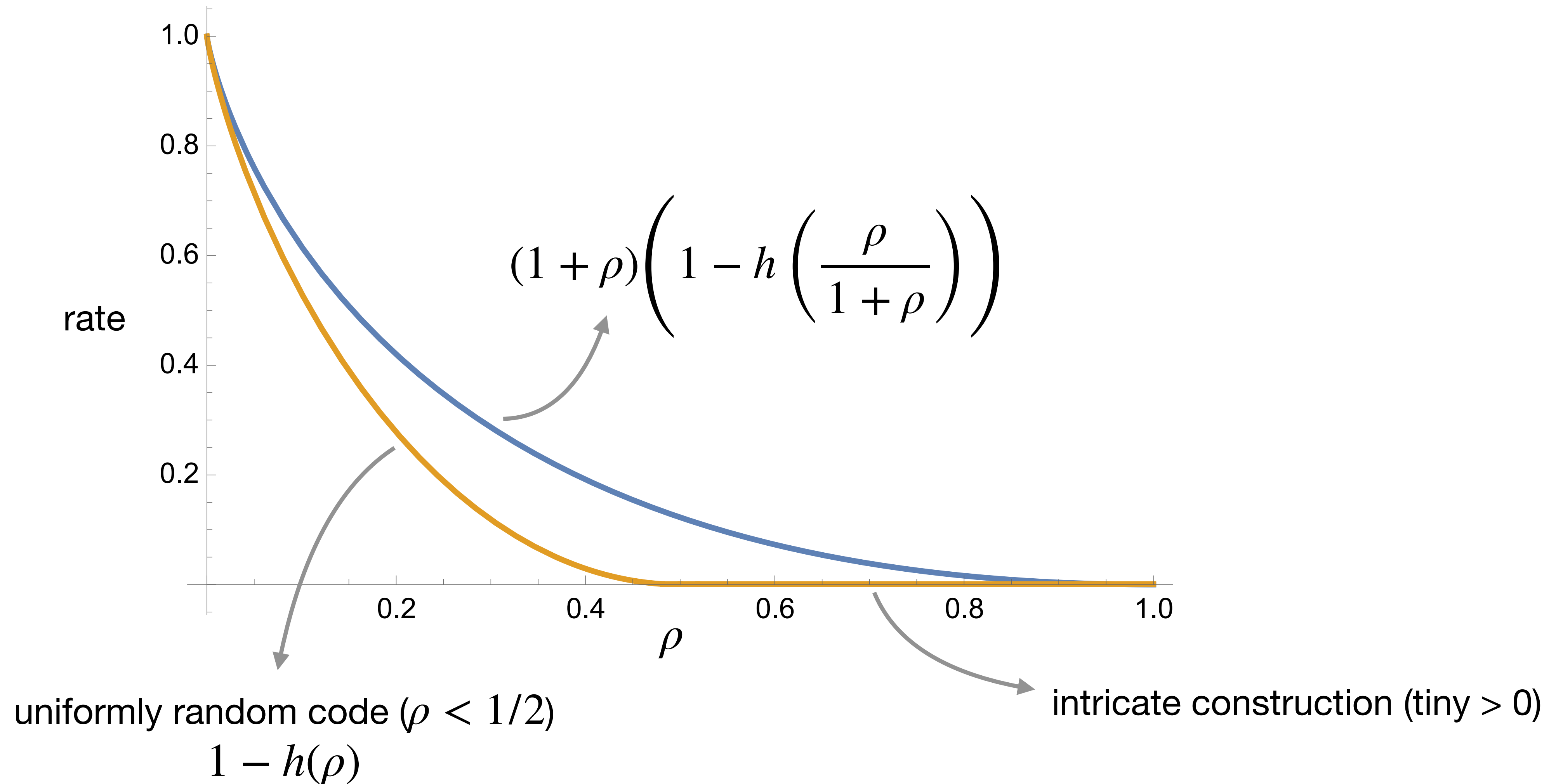
Capacity of list-decoding from insertions — the story so far

Summarizing:



Capacity of list-decoding from insertions — the story so far

Summarizing:



List-decoding capacity from Info Theory insights



Roni Con
Tel Aviv U



Dean Doron
Ben-Gurion U

List-decoding capacity from Info Theory insights



Roni Con
Tel Aviv U

$$C_{\text{ins}}(\rho) = (1 + \rho) \left(1 - h \left(\frac{\rho}{1 + \rho} \right) \right) \text{ for all } \rho \in [0, 1].$$



Dean Doron
Ben-Gurion U

List-decoding capacity from Info Theory insights



Roni Con
Tel Aviv U

$$C_{\text{ins}}(\rho) = (1 + \rho) \left(1 - h \left(\frac{\rho}{1 + \rho} \right) \right) \text{ for all } \rho \in [0, 1].$$

Uniformly random codes achieve capacity on the BSC, so it makes sense that they're good for list-decoding from bit-flips too.

But uniformly random codes aren't that good against random insdels.



Dean Doron
Ben-Gurion U

List-decoding capacity from Info Theory insights



Roni Con
Tel Aviv U

$$C_{\text{ins}}(\rho) = (1 + \rho) \left(1 - h \left(\frac{\rho}{1 + \rho} \right) \right) \text{ for all } \rho \in [0,1].$$

Uniformly random codes achieve capacity on the BSC, so it makes sense that they're good for list-decoding from bit-flips too.

But uniformly random codes aren't that good against random insdels.



Dean Doron
Ben-Gurion U

Codes sampled with memory are much better against random insdels!
[Diggavi-Grossglauser '06, Drinea-Mitzenmacher '07,...]

Markov codes and list-decoding from insertions

Markov codes and list-decoding from insertions

Idea: Instead of sampling codewords uniformly at random, sample each codeword independently according to a 2-state Markov chain $X_1, X_2, \dots, X_n$ with

Markov codes and list-decoding from insertions

Idea: Instead of sampling codewords uniformly at random, sample each codeword independently according to a 2-state Markov chain $X_1, X_2, \dots, X_n$ with

- X_1 a uniformly random bit;

Markov codes and list-decoding from insertions

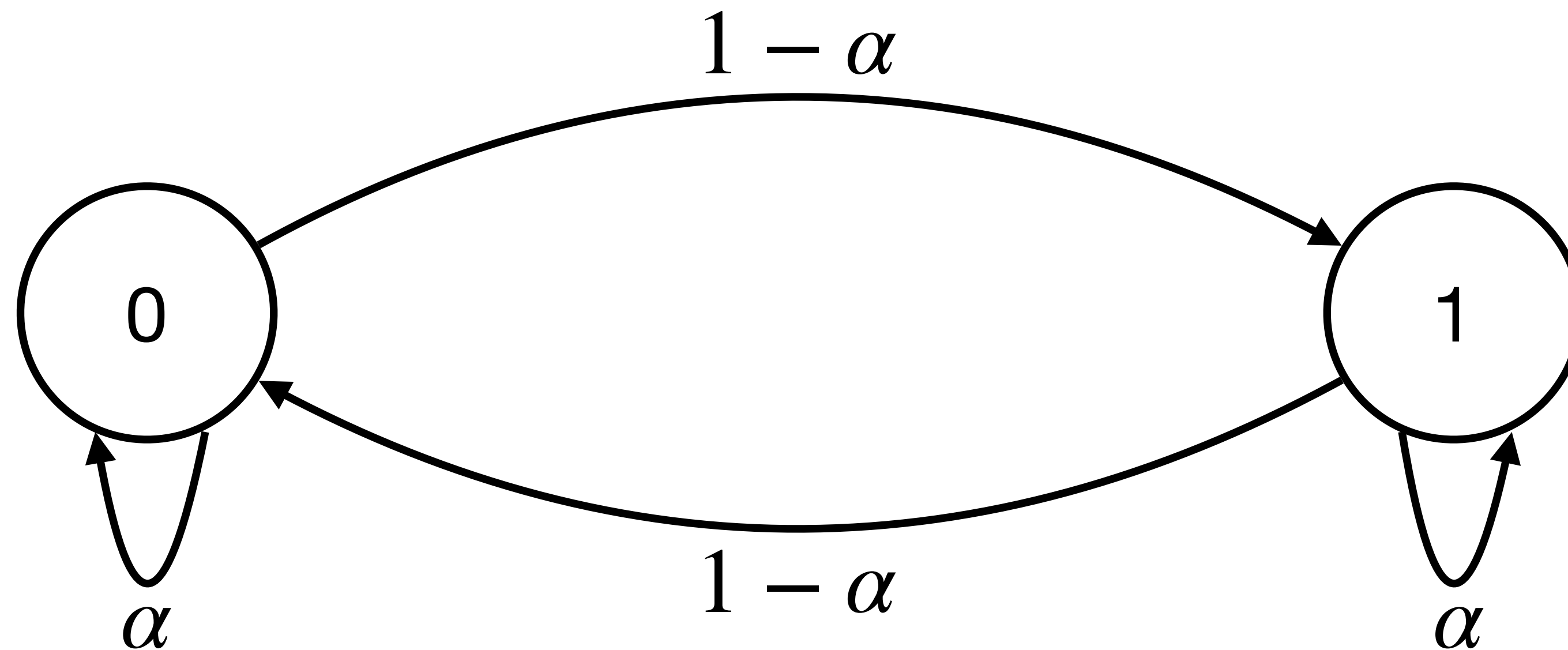
Idea: Instead of sampling codewords uniformly at random, sample each codeword independently according to a 2-state Markov chain $X_1, X_2, \dots, X_n$ with

- X_1 a uniformly random bit;
- $\Pr[X_i = X_{i-1}] = \alpha$ for all $i > 1$.

Markov codes and list-decoding from insertions

Idea: Instead of sampling codewords uniformly at random, sample each codeword independently according to a 2-state Markov chain $X_1, X_2, \dots, X_n$ with

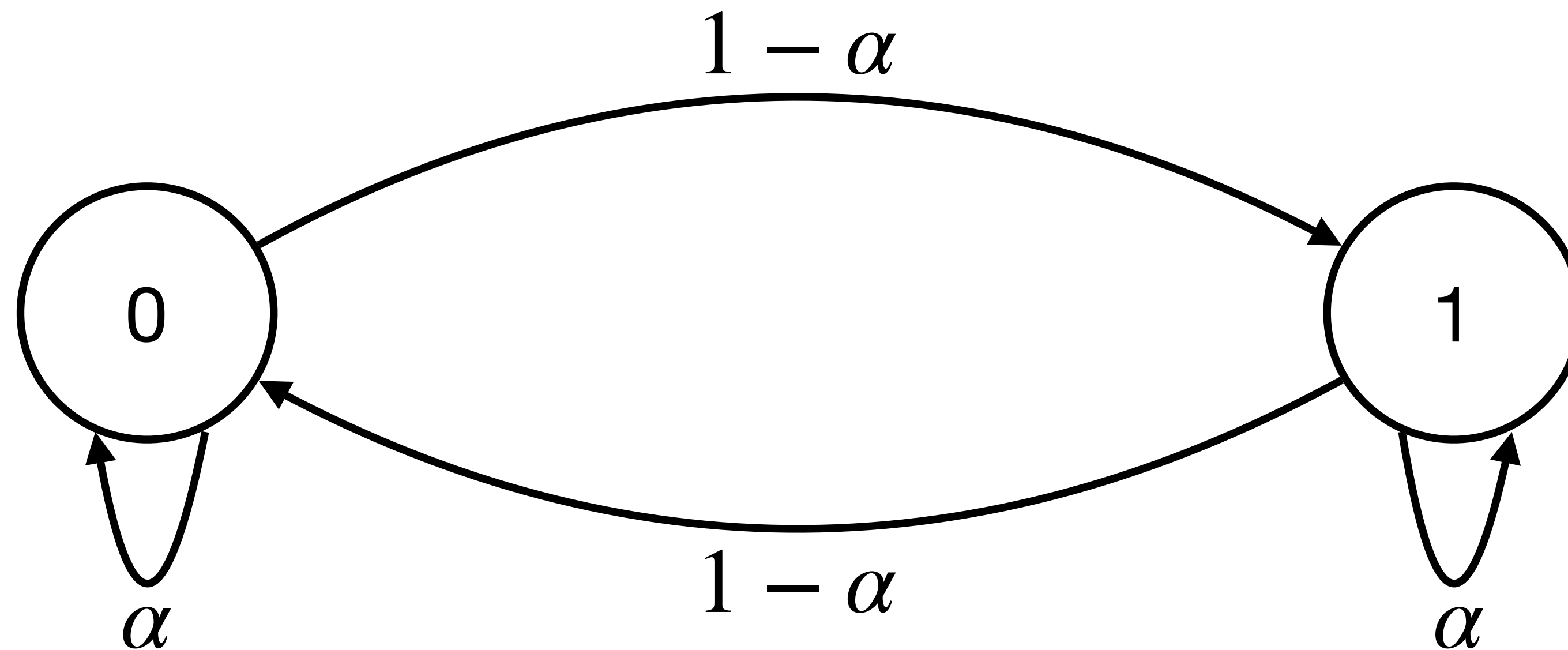
- X_1 a uniformly random bit;
- $\Pr[X_i = X_{i-1}] = \alpha$ for all $i > 1$.



Markov codes and list-decoding from insertions

Idea: Instead of sampling codewords uniformly at random, sample each codeword independently according to a 2-state Markov chain $X_1, X_2, \dots, X_n$ with

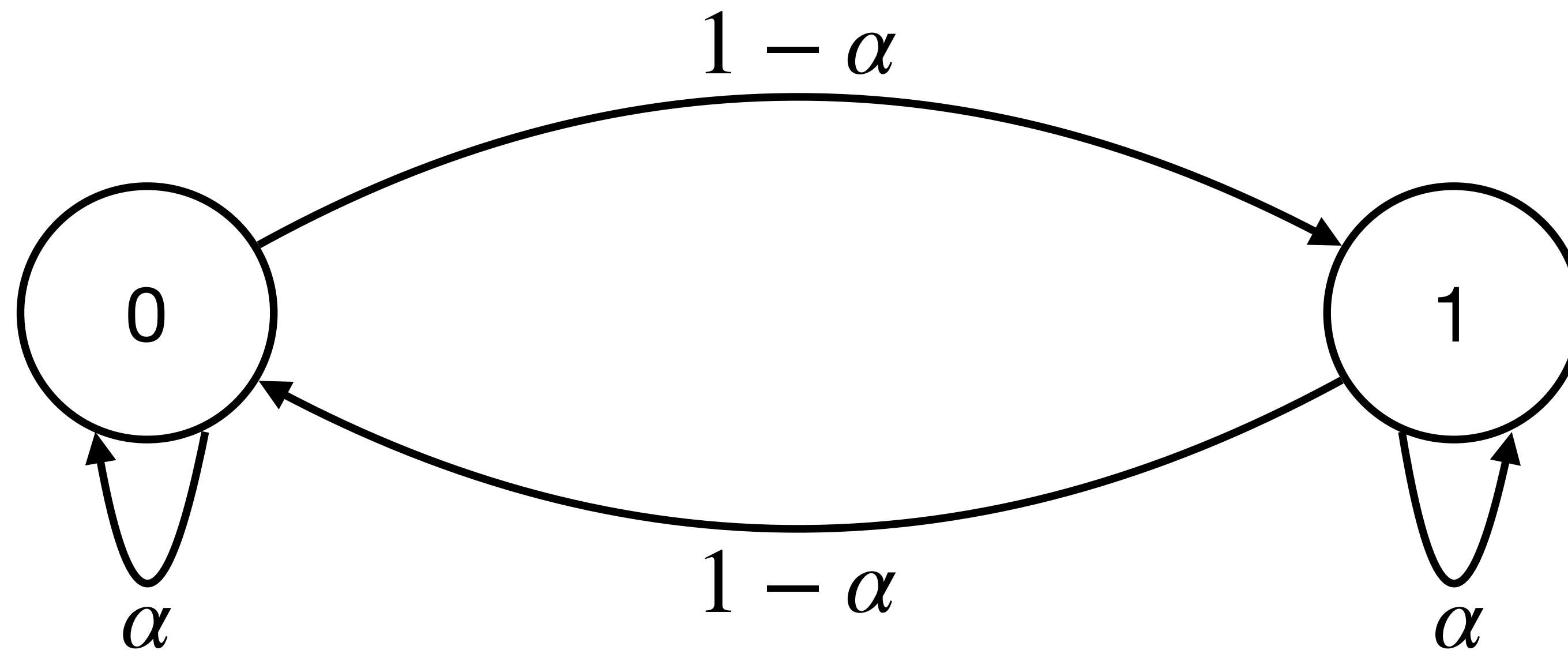
- X_1 a uniformly random bit;
- $\Pr[X_i = X_{i-1}] = \alpha$ for all $i > 1$.
- Choosing $\alpha = 1/2$ recovers uniformly random coding;



Markov codes and list-decoding from insertions

Idea: Instead of sampling codewords uniformly at random, sample each codeword independently according to a 2-state Markov chain $X_1, X_2, \dots, X_n$ with

- X_1 a uniformly random bit;
- $\Pr[X_i = X_{i-1}] = \alpha$ for all $i > 1$.
- Choosing $\alpha = 1/2$ recovers uniformly random coding;
- Optimal choice turns out to be $\alpha = \frac{1 + \rho}{2}$.



Philosophizing about our insertion list-decoding result

Philosophizing about our insertion list-decoding result

- We used the fact that sampling with memory works better than uniform for random insdels to give an exact answer about the capacity of list-decoding from adversarial insertions;

Philosophizing about our insertion list-decoding result

- We used the fact that sampling with memory works better than uniform for random insdels to give an exact answer about the capacity of list-decoding from adversarial insertions;
- I still don't really understand WHY this worked!

Philosophizing about our insertion list-decoding result

- We used the fact that sampling with memory works better than uniform for random insdels to give an exact answer about the capacity of list-decoding from adversarial insertions;
- I still don't really understand WHY this worked!
 - 2-state Markov chains are **not** optimal for random insdels;

Philosophizing about our insertion list-decoding result

- We used the fact that sampling with memory works better than uniform for random insdels to give an exact answer about the capacity of list-decoding from adversarial insertions;
- I still don't really understand WHY this worked!
 - 2-state Markov chains are **not** optimal for random insdels;
 - It's remarkable that we got THE answer, because 50+ years of studying insdels has led to very few exact answers so far (to the best of my knowledge, previously only in extremal settings).

Philosophizing about our insertion list-decoding result

- We used the fact that sampling with memory works better than uniform for random insdels to give an exact answer about the capacity of list-decoding from adversarial insertions;
- I still don't really understand WHY this worked!
 - 2-state Markov chains are **not** optimal for random insdels;
 - It's remarkable that we got THE answer, because 50+ years of studying insdels has led to very few exact answers so far (to the best of my knowledge, previously only in extremal settings).
- Making a parallel with bit-flips... What does this reveal about the capacity of channels with random insdels?

Wrapping up

Wrapping up

- Correcting errors in noisy communication boils down to constructing large collections of dots that are far apart;

Wrapping up

- Correcting errors in noisy communication boils down to constructing large collections of dots that are far apart;
- Placing dots *uniformly at random* is a great idea, and it's optimal for correcting random errors (fuzzy balls around dots) and for list-decoding (few dots in each ball);

Wrapping up

- Correcting errors in noisy communication boils down to constructing large collections of dots that are far apart;
- Placing dots *uniformly at random* is a great idea, and it's optimal for correcting random errors (fuzzy balls around dots) and for list-decoding (few dots in each ball);
- Placing dots according to a *random subspace* is also a great idea. This is a special case of the very active “random subspace vs. random set” line of research;

Wrapping up

- Correcting errors in noisy communication boils down to constructing large collections of dots that are far apart;
- Placing dots *uniformly at random* is a great idea, and it's optimal for correcting random errors (fuzzy balls around dots) and for list-decoding (few dots in each ball);
- Placing dots according to a *random subspace* is also a great idea. This is a special case of the very active “random subspace vs. random set” line of research;
- There's an intimate connection between coding against random bit-flips and list-decoding from adversarial bit-flips. What about for other types of errors?

Wrapping up

- Correcting errors in noisy communication boils down to constructing large collections of dots that are far apart;
- Placing dots *uniformly at random* is a great idea, and it's optimal for correcting random errors (fuzzy balls around dots) and for list-decoding (few dots in each ball);
- Placing dots according to a *random subspace* is also a great idea. This is a special case of the very active “random subspace vs. random set” line of research;
- There's an intimate connection between coding against random bit-flips and list-decoding from adversarial bit-flips. What about for other types of errors?
- **Determining the capacity of channels with random insertions/deletions is a major open problem in information theory:**

Wrapping up

- Correcting errors in noisy communication boils down to constructing large collections of dots that are far apart;
- Placing dots *uniformly at random* is a great idea, and it's optimal for correcting random errors (fuzzy balls around dots) and for list-decoding (few dots in each ball);
- Placing dots according to a *random subspace* is also a great idea. This is a special case of the very active “random subspace vs. random set” line of research;
- There's an intimate connection between coding against random bit-flips and list-decoding from adversarial bit-flips. What about for other types of errors?
- Determining the capacity of channels with random insertions/deletions is a major open problem in information theory:
 - Can we gain new insights by studying list-decoding from insertions/deletions?

Wrapping up

- Correcting errors in noisy communication boils down to constructing large collections of dots that are far apart;
- Placing dots *uniformly at random* is a great idea, and it's optimal for correcting random errors (fuzzy balls around dots) and for list-decoding (few dots in each ball);
- Placing dots according to a *random subspace* is also a great idea. This is a special case of the very active “random subspace vs. random set” line of research;
- There's an intimate connection between coding against random bit-flips and list-decoding from adversarial bit-flips. What about for other types of errors?
- Determining the capacity of channels with random insertions/deletions is a major open problem in information theory:
 - Can we gain new insights by studying list-decoding from insertions/deletions?
 - **We can use ideas from random errors (sampling with memory) to improve our understanding of list-decoding!**

Wrapping up

- Correcting errors in noisy communication boils down to constructing large collections of dots that are far apart;
- Placing dots *uniformly at random* is a great idea, and it's optimal for correcting random errors (fuzzy balls are like "fuzzy balls" (fuzzy balls are like fuzzy balls));
- Placing dots according to a specific distribution. This is a special case of the very active "random" research;
- There's an intimate connection between list-decoding from adversarial errors and list-decoding from flips and list-decoding from adversarial errors?
- Determining the capacity of channels with random insertions/deletions is a major open problem in information theory:
 - Can we gain new insights by studying list-decoding from insertions/deletions?
 - We can use ideas from random errors (sampling with memory) to improve our understanding of list-decoding!